



VERSION IMPRIMABLE

BATMAN & ROBIN × GODOT 4.5 — LE PAS-À-PAS

Parcours guidé, étape par étape, pour construire soi-même le niveau 1 de The Adventures of Batman & Robin (Mega Drive) dans Godot 4.5, en apprenant les méthodes du développement de jeu et le moteur Godot.

SOMMAIRE

ACCUEIL : LA MÉTHODE ET LE PARCOURS	6
01 · DÉCOUVRIR GODOT ET CRÉER LE PROJET	11
1.1 · Installer Godot et Git	11
1.2 · Créer le projet	12
1.3 · Faire le tour de l'éditeur	12
1.4 · Une scène, deux nœuds	13
1.5 · Un premier script	13
1.6 · Un premier signal	15
1.7 · Instancier une scène	16
1.8 · Régler le projet comme une Mega Drive	16
1.9 · Créer l'arborescence	18
1.10 · Le premier commit	18
02 · LES FONDATIONS	21
2.1 · Créer une action à la main	21
2.2 · Un script d'éditeur pour toutes les actions	22
2.3 · <code>_process</code> ou <code>_physics_process</code> ?	24
2.4 · Nommer les couches	25
2.5 · L'autoload Game	27
2.6 · Tester le patron Observateur	29
2.7 · Health, Hurtbox, Hitbox	30
2.8 · Un banc d'essai pour les dégâts	33
2.9 · State et StateMachine	34
2.10 · Tester la machine sur un feu tricolore	36
2.11 · Construire <code>test_room.tscn</code>	37
03 · PROTOTYPE : BATMAN EN UN SEUL SCRIPT	40
3.1 · Regarder Batman bouger	40
3.2 · Le corps de Batman	41
3.3 · Courir et sauter en trente lignes	41
3.4 · Régler le ressenti	43
3.5 · Ajouter l'accroupissement	43
3.6 · Compter ce qui reste à faire	44
04 · MACHINE À ÉTATS ET CAMÉRA	47
4.1 · Dessiner le graphe d'états	47
4.2 · Créer <code>player.tscn</code>	48
4.3 · <code>player.gd</code> , version 1	49
4.4 · Les états Idle et Run	51
4.5 · Les états Crouch et Air	53
4.6 · Voir l'état en direct	54
4.7 · Observer la caméra originale	56
4.8 · <code>camera_rig.gd</code>	56
4.9 · Une salle plus longue pour tester	58

05 · COMBAT, ACCROCHE ET DÉGÂTS	60
5.1 · Ce qui est un état, ce qui n'en est pas	60
5.2 · Ajouter les nœuds	61
5.3 · Une fiche par héros : HeroData	62
5.4 · player.gd, version de référence	62
5.5 · Mettre à jour Idle, Run, Crouch et Air	68
5.6 · L'état Melee	70
5.7 · L'état Hang et le rebord	71
5.8 · Hurt, Grabbed, Dead, Victory	73
5.9 · Une cible, un piège, une réapparition	75
5.10 · Les sprites (facultatif)	77
06 · ARMES, JAUGE ET BONUS	80
6.1 · Ce que fait le jeu original	80
6.2 · La fiche WeaponData	81
6.3 · Le registre des armes	83
6.4 · Le piège de la position mondiale	84
6.5 · Fx : faire apparaître, exploser, flasher	84
6.6 · Pool	87
6.7 · Le projectile	88
6.8 · WeaponSystem : tir, niveaux et jauge	90
6.9 · La base Pickup	94
6.10 · Le bonus d'arme à couleur cyclique	95
6.11 · Le crâne et le bonus de charge	96
07 · ENNEMIS ET ARÈNES À VAGUES	100
7.1 · Observer une arène	100
7.2 · Atelier : des formes de substitution pour les ennemis	101
7.3 · La classe Enemy	101
7.4 · Le clown	104
7.5 · Les vagues : WaveData	106
7.6 · L'arène : WaveArena	107
7.7 · La bulle « GO! » et une première arène	109
7.8 · Le costaud qui empoigne	110
7.9 · Le tireur et les balles ennemies	112
7.10 · La bombe	113
7.11 · Les formations de têtes à hélice	115
7.12 · Le ballon-mine	118
7.13 · Les objets cassables	119
08 · EFFETS MEGA DRIVE	122
8.1 · PaletteFx : le post-traitement 9 bits	122
8.2 · Brancher le crâne sur PaletteFx	125
8.3 · Le principe	125
8.4 · Une texture de test générée par script	128
8.5 · La façade pseudo-3D	129
8.6 · Le shader du feu	130
8.7 · HitFlash	133
8.8 · Changer l'ambiance sans redessiner	135
8.9 · Deux outils pour la suite	138

09 · SECTION 1 : RUE, BANQUE ET PREMIER BOSS	140
9.1 · La classe Section et SceneFlow	140
9.2 · Monter la scène s1_streets.tscn	142
9.3 · Placer les arènes	143
9.4 · Observer l'événement	145
9.5 · Préparer la façade « incendie »	145
9.6 · bank_event.gd	145
9.7 · La base commune des boss	147
9.8 · La machine d'Harley : le plan du combat	150
9.9 · La machine d'Harley : le code	150
9.10 · Monter la machine dans la rue	154
9.11 · Jouer la section de bout en bout	155
10 · HUD, ÉCRANS ET FLUX DE JEU	157
10.1 · Le jeu est une machine à états	157
10.2 · SceneFlow, version complète	158
10.3 · Bus audio et autoload Audio	159
10.4 · Un thème pour tout le jeu	161
10.5 · Le médaillon et la jauge, dessinés à la main	161
10.6 · hud.tscn et hud.gd	163
10.7 · L'intro, une frise temporelle	166
10.8 · L'écran titre	167
10.9 · Les options et un réglage qui persiste	169
10.10 · La sélection du héros	171
10.11 · Les interludes et le continue	172
10.12 · Le jeu de bout en bout	173
11 · SECTION 2 : GEM EXPO ET GRAPPIN	176
11.1 · Observer la répétition	176
11.2 · Deux salles	177
11.3 · La section et la skyline	178
11.4 · Les vagues de têtes à hélice	178
11.5 · Peindre des caisses avec une TileMapLayer	179
11.6 · Harley passe : une annonce en pleine partie	181
11.7 · La projection perspective	183
11.8 · Harley, son fauteuil et le mini-boss	187
12 · SECTION 3 : CONVOI ET JOKER	191
12.1 · Le monde ne bouge pas, la rue si	191
12.2 · La section et la rue qui défile	192
12.3 · Le véhicule, une plateforme mobile	193
12.4 · Tomber du convoi	194
12.5 · Ballons-mines et paquet géant	195
12.6 · Observer un passage	196
12.7 · joker_balloon.gd	196
12.8 · Monter la montgolfière	199
12.9 · Le niveau complet	200

13 · FINITIONS, EXPORT ET SUITE	202
13.1 · Le tremblement de caméra	202
13.2 · Le hit stop	203
13.3 · Comparer à l'original, section par section	204
13.4 · Ranger les outils de débogage	205
13.5 · Mesurer les performances	205
13.6 · Exporter ton jeu	206
13.7 · Ta rétrospective	206
13.8 · Et après ?	207

CONSTRUIRE LE NIVEAU 1, DE ZÉRO, DE TES PROPRES MAINS

Le [guide vérifié](#) explique *ce que fait le jeu* et *pourquoi*. Ce pas-à-pas te dit **quoi faire, dans quel ordre, et comment vérifier que ça marche**. En 13 étapes, tu passes d'un projet vide au niveau 1 complet, et tu apprends en chemin les méthodes du développement de jeu et le moteur Godot 4.

Godot 4.5+ · GDScript typé

13 étapes · ~50 h

Aucune expérience de Godot requise

Tout se fait à la main

À QUI S'ADRESSE CE PARCOURS

À quelqu'un qui sait déjà programmer (variables, fonctions, conditions, boucles, classes), mais qui n'a jamais fait de jeu, ou jamais utilisé Godot. Tu n'as pas besoin de connaître la Mega Drive : on l'explique quand elle sert.

Les quatre documents du dépôt se complètent :

Document	Rôle	Quand l'ouvrir
Secrets de programmation	Première doc, conservée telle quelle	Par curiosité historique
Tutoriel en 7 tomes	Deuxième doc, conservée telle quelle	Par curiosité historique
Guide vérifié (12 tomes)	La référence : ce que montre la vidéo, comment faisait la console, le code complet commenté	Quand tu veux comprendre le « pourquoi » d'une mécanique, ou revoir la version de référence d'un script
Ce pas-à-pas	Le parcours : l'ordre des manipulations, les tests, les erreurs fréquentes, les exercices	Tout le temps, en travaillant

UN SEUL CODE

Les scripts finaux de ce parcours sont **identiques** à ceux du guide. Quand le pas-à-pas fait d'abord une version simplifiée (pour apprendre), il le dit, puis la remplace par la version de référence. Chaque étape renvoie au tome du guide correspondant.

LA MÉTHODE

Chaque étape suit la même boucle. C'est la boucle de travail d'un développeur de jeu, à toutes les échelles : d'un saut de personnage à un jeu entier.

1**Comprendre**

Lire le concept : quel problème on résout, quelle idée générale on applique.

2**Construire**

Créer les nœuds et les scènes dans l'éditeur.

3**Coder**

Taper le script, puis lire « Comprendre le code ».

4**Tester**

Lancer, observer, comparer au résultat attendu.

5**Déboguer**

Si ça coince : l'encadré « Si ça ne marche pas ».

6**Valider**

Cocher l'étape, faire un commit Git.

Six principes reviennent dans tout le parcours. Ce sont eux que tu emporteras sur tes propres jeux :

1. **Toujours avoir un jeu qui tourne.** Chaque étape se termine sur quelque chose de jouable. On n'écrit jamais deux heures de code sans lancer le jeu.
2. **Des formes de substitution d'abord.** Un rectangle bleu suffit pour régler un saut. Les sprites viennent quand la mécanique est bonne.
3. **Mesurer la référence avant de coder.** Ici, la vidéo du jeu original : vitesse, durées, comportements. Un chiffre mesuré vaut mieux qu'une impression.
4. **Un nœud, une responsabilité.** La vie, les dégâts, le déplacement, la caméra : chacun dans son coin, reliés par des signaux.
5. **Les réglages dans des données, pas dans le code.** `@export` et les `Resource` permettent d'ajuster sans réécrire.
6. **Des petits pas vérifiés.** Un bug trouvé cinq minutes après l'avoir écrit se corrige en une minute ; trouvé trois semaines plus tard, il coûte une soirée.

LES ENCADRÉS

CONCEPT

L'idée générale de développement de jeu, réutilisable dans n'importe quel moteur.

GODOT 4

Une notion propre au moteur : nœud, signal, ressource, réglage de l'éditeur...

À FAIRE DANS L'ÉDITEUR

1. Les manipulations, dans l'ordre, avec les menus : `≡ Scène → Nouvelle scène`, les touches : `F6`.

TESTE

Ce que tu dois voir en lançant le jeu. Si tu ne le vois pas, ne passe pas à la suite.

SI ÇA NE MARCHE PAS

Les erreurs les plus fréquentes à cet endroit, et comment les corriger.

MÉTHODE

Une habitude de travail de développeur de jeu.

DANS LA VIDÉO

Un moment du jeu original à regarder : [▶ 2:52](#).

Chaque titre d'étape a un bouton « **Marquer comme fait** ». Ta progression est mémorisée dans ce navigateur et s'affiche sur les cartes ci-dessous.

LE PARCOURS

01

~2 h

Découvrir Godot

Installer, comprendre nœuds, scènes, scripts et signaux. Créer et régler le projet 320×224.

02

~3 h

Les fondations

Entrées, couches de collision, autoload, composants de vie et de dégâts, première machine à états.

03

~1 h 30

Prototype

Batman en un seul script, pour sentir le personnage, puis comprendre pourquoi il faut mieux.

04

~3 h

Machine à états & caméra

Réécrire Batman proprement : un état par fichier. Caméra lissée qui anticipe.

05

~3 h

Combat & accroche

Coup de poing, coup de pied sauté, accroche au balcon, dégâts, invincibilité, mort, victoire.

06

~3 h

Armes & bonus

Armes en données, réserve de projectiles, jauge de charge, bonus cycliques, crâne.

07

~5 h

Ennemis & arènes

Classe de base, clowns, costauds, tireurs, formations, arènes à vagues et « GO! ».

08

~4 h

Effets Mega Drive

Fondus par paliers, façade raster, feu ondulant, flash de dégâts, couleurs indexées.

09

~6 h

Section 1

La rue complète, l'explosion de la banque et la machine d'Harley en trois phases.

10

~5 h

HUD, écrans & flux

HUD fidèle, intro, titre, options, sélection, continue, audio, enchaînement des scènes.

11

~5 h

Section 2

Salles réutilisables, TileMapLayer de caisses, et le grappin en sphères pseudo-3D.

12

~4 h

Section 3

Le convoi en plateformes mobiles, les ballons-mines et la montgolfière du Joker.

13

~3 h

Finitions & suite

Ressenti, fidélité, profilage, export, boîte à outils de débogage, et la suite.

POURQUOI CET ORDRE

On construit **du bas vers le haut** : d'abord les briques que tout le monde utilise (02), puis le héros (03–05), ce qu'il tire (06), ce qu'il combat (07), l'habillage (08), et seulement ensuite les vrais niveaux (09–12). Chaque étape n'utilise que ce qui existe déjà. Les étapes 11 et 12 sont indépendantes l'une de l'autre : tu peux les inverser. Le HUD et les écrans (10) viennent juste après la première section, pour que ton jeu ressemble vite à un jeu complet.

AVANT DE COMMENCER

- **Godot 4.5 ou plus récent**, version standard (pas la version .NET) : godotengine.org/download. L'étape 01 détaille l'installation.
- **Git**, pour sauvegarder chaque étape validée. Si tu ne l'as jamais utilisé, les commandes sont données.
- Un éditeur d'images simple (Aseprite, LibreSprite, Krita, Paint.NET, GIMP...) pour les formes de substitution et les découpes de sprites.
- La [vidéo de référence](#) dans un onglet : on y revient souvent.
- Les planches de sprites du dépôt : [batman-and-robin/spritesheets/](https://github.com/batman-and-robin/spritesheets/). Elles ne sont pas obligatoires pour avancer.

DROITS SUR LES RESSOURCES

Sprites, musique et logos appartiennent à DC Comics, Warner Bros. et Sega. Ton projet est un exercice personnel : ne publie pas de build qui les contient.

COMMENT VRAIMENT ASSIMILER

- **Tape le code, ne le colle pas.** Le bouton « Copier » existe pour dépanner. Taper force à lire chaque ligne, et l'autocomplétion de Godot t'apprend l'API au passage.
- **Lis « Comprendre le code » après avoir tapé,** pas avant : tu sauras quelles lignes t'ont posé question.
- **Fais les expériences « Et si... ».** Changer une valeur et observer le résultat est la façon la plus rapide de comprendre un système.
- **Réponds aux questions de fin d'étape** avant d'ouvrir la réponse.
- **Tiens un journal de bord** : un fichier texte où tu notes les erreurs rencontrées et leur solution. Au bout de quelques étapes, c'est ton aide-mémoire personnel.
- **Refais de mémoire.** Une fois une étape finie, recrée une petite partie (par exemple l'état ) sans regarder. Ce qui bloque, c'est ce qu'il reste à apprendre.

DÉCOUVRIR GODOT ET CRÉER LE PROJET

Avant Batman, un bac à sable : un carré qui bouge et change de couleur. Il suffit à comprendre les quatre idées sur lesquelles repose tout Godot. Ensuite, on crée le vrai projet et on le règle comme une Mega Drive.

 ~2 h

Prérequis : aucun

Guide : [tome 02](#)

CE QUE TU VAS APPRENDRE

GODOT 4

- Nœud, scène, script, signal
- L'éditeur : arbre de scène, inspecteur, système de fichiers
- `_ready()`, `_process()`, `delta`, `@export`
- Les paramètres du projet

DÉVELOPPEMENT DE JEU

- La boucle de jeu
- La résolution virtuelle et le pixel net
- Le bac à sable pour apprendre sans risque
- Le versionnage dès la première minute

INSTALLER

1.1 · Installer Godot et Git



À FAIRE

1. Télécharge **Godot 4.5** (ou plus récent) sur godotengine.org/download, version **standard**. Sur macOS, glisse `Godot.app` dans `Applications`. Godot n'a pas d'installateur : c'est un seul exécutable.
2. Vérifie que Git est installé : dans un terminal, `git --version`. Sinon : git-scm.com (sur macOS, `xcode-select --install` l'installe aussi).
3. Crée un dossier pour tes projets Godot, par exemple `~/Godot/`.

GODOT 4

Godot est un moteur libre (licence MIT) : pas de compte, pas de redevance, et le jeu t'appartient entièrement. Son langage, **GDScript**, ressemble à Python : indentation significative, pas d'accolades. Ce parcours utilise le **GDScript typé** (`var vitesse: float`, `func f() → void`) : l'éditeur détecte plus d'erreurs avant même de lancer le jeu, et l'autocomplétion est meilleure.

1.2 · Créer le projet



À FAIRE

1. Lance Godot. Le **gestionnaire de projets** s'ouvre. Clique sur **Créer** (*Create*).
2. Nom : `batman-robin-11`. Chemin : ton dossier `~/Godot/`. Coche « Créer un dossier ».
3. Moteur de rendu : **Compatibilité** (*Compatibility*). C'est le plus léger, il suffit largement à un jeu 2D en pixel art, et il tourne partout.
4. Métadonnées de contrôle de version : **Git**. Godot crée alors un `.gitignore` adapté.
5. Clique sur **Créer et modifier**.

SI ÇA NE MARCHE PAS

Sur macOS, si le système refuse d'ouvrir Godot (« développeur non identifié ») : clic droit sur `Godot.app` → **Ouvrir**, puis confirme. Il ne le redemandera plus.

1.3 · Faire le tour de l'éditeur



Prends cinq minutes pour repérer les zones. Tu y passeras des dizaines d'heures.

Zone	Où	À quoi elle sert
Scène	en haut à gauche	L'arbre des nœuds de la scène ouverte. On y ajoute (<code>Ctrl A</code> / <code>⌘ A</code>), renomme (<code>F2</code>), déplace et supprime des nœuds.
Système de fichiers	en bas à gauche	Les fichiers du projet. Tout chemin commence par <code>res://</code> , la racine du projet.
Vue 2D	au centre	Pour placer les nœuds à la souris. Onglets en haut : 2D, 3D, Script, AssetLib.
Inspecteur	à droite	Les propriétés du nœud sélectionné. Onglet voisin : Nœud (signaux et groupes).
Panneau du bas	en bas	Sortie (les <code>print</code>), Débogueur (erreurs, moniteurs), Audio, Animation.
Lancement	en haut à droite	Lancer le projet <code>F5</code> (<code>⌘ B</code> sur Mac), lancer la scène ouverte <code>F6</code> (<code>⌘ R</code>), arrêter <code>F8</code> (<code>⌘ .</code>).

LANGUE DE L'ÉDITEUR

Ce parcours donne les menus en français, avec l'anglais entre parenthèses quand c'est utile. Pour changer la langue : **Éditeur** → **Paramètres de l'éditeur** → **Interface** → **Éditeur** → **Langue de l'éditeur**. Beaucoup de tutoriels en ligne sont en anglais : savoir lire les deux t'aidera.

LE BAC À SABLE : LES QUATRE IDÉES DE GODOT

CONCEPT : LES QUATRE IDÉES

- **Nœud** : la brique de base. Chaque nœud a *un* rôle : afficher une image (`Sprite2D`), détecter un contact (`Area2D`), jouer un son, compter le temps (`Timer`)...
- **Scène** : un arbre de nœuds enregistré dans un fichier `.tscn`. Une scène peut être *instanciée* dans une autre, autant de fois qu'on veut : Batman est une scène, chaque clown est une instance de la scène « clown ».
- **Script** : du code attaché à un nœud, qui en étend le comportement.
- **Signal** : un message qu'un nœud émet (« j'ai été touché », « le temps est écoulé ») et que d'autres reçoivent, sans se connaître.

1.4 · Une scène, deux nœuds



À FAIRE

1. Dans le Système de fichiers, clic droit sur `res://` → **Nouveau dossier** → `sandbox`. Ce dossier contiendra tous les essais jetables.
2. ≡ **Scène** → **Nouvelle scène**. Dans le panneau Scène, choisis **Scène 2D** : la racine est un `Node2D`. Renomme-le `Box` (double-clic ou `F2`).
3. Sélectionne `Box`, puis `Ctrl A` (`⌘ A`) : cherche `ColorRect` et ajoute-le comme enfant.
4. Dans l'Inspecteur du `ColorRect` : **Color** = un rouge vif ; dans **Layout** → **Transform**, **Size** = (16, 16) et **Position** = (-8, -8) pour que le carré soit centré sur son parent.
5. Sélectionne `Box` et, dans l'Inspecteur, **Transform** → **Position** = (160, 112).
6. `Ctrl S` (`⌘ S`) : enregistre sous `res://sandbox/box.tscn`.

POSITION RELATIVE AU PARENT

La position d'un nœud est **relative à son parent**. Le carré est à (-8, -8) *par rapport à* `Box` : quand `Box` bouge, le carré suit. Toute la hiérarchie fonctionne ainsi. C'est ce qui permet de déplacer Batman d'un bloc avec son sprite, ses zones de collision et ses effets.

1.5 · Un premier script



À FAIRE

1. Sélectionne `Box`, clique sur l'icône de parchemin « + » en haut du panneau Scène (**Attacher un script**).
2. Langage GDScript, hérite de `Node2D`, chemin `res://sandbox/box.gd`, modèle « Object: Empty » si proposé. **Créer**.
3. L'éditeur de script s'ouvre. Tape ce code (sans le coller) :

res://sandbox/box.gd

```
extends Node2D
## Mon premier script : un carré qui se déplace avec les flèches.

@export var speed := 120.0 # pixels par seconde, modifiable dans l'inspecteur

func _ready() -> void:
    print("Bonjour depuis ", name)

func _process(delta: float) -> void:
    var dir := Input.get_vector("ui_left", "ui_right", "ui_up", "ui_down")
    position += dir * speed * delta
```

TESTE

F6 (⌘ R) lance la scène ouverte. Une fenêtre grise s'ouvre avec le carré ; les flèches le déplacent. Dans le panneau **Sortie**, en bas : « Bonjour depuis Box ». Ferme la fenêtre ou F8.

Comprendre le code

- `extends Node2D` : le script est un `Node2D` amélioré. Il a donc accès à `position`, `rotation`, `name` ...
- `## ...` : un commentaire de documentation, affiché dans l'aide de l'éditeur. `# ...` est un commentaire ordinaire.
- `@export` : la variable apparaît dans l'Inspecteur. Sélectionne `Box` : tu y vois **Speed**. On peut régler un objet sans toucher au code.
- `:=` : déclare la variable et *déduit* son type de la valeur (ici `float`). Équivaut à `var speed: float = 120.0`.
- `_ready()` : appelée une fois, quand le nœud entre dans la scène et que ses enfants sont prêts.
- `_process(delta)` : appelée **à chaque image affichée**. `delta` est la durée écoulée depuis l'image précédente, en secondes ($\approx 0,0167$ à 60 i/s).
- `Input.get_vector(...)` : renvoie un `Vector2` de longueur ≤ 1 selon les touches enfoncées. Les actions `ui_*` existent par défaut dans tout projet.
- `position += dir * speed * delta` : vitesse $\times$ temps = distance. C'est la ligne la plus importante de la page.

CONCEPT : LA BOUCLE DE JEU

Un jeu est une boucle infinie : *lire les entrées* $\rightarrow$ *mettre à jour le monde* $\rightarrow$ *dessiner*, environ 60 fois par seconde. Godot fait tourner la boucle pour toi et appelle `_process()` à chaque tour. Ton travail se résume souvent à répondre à une question : « que fait cet objet pendant ce petit instant `delta` ? ».

ET SI...

- ...tu enlèves `* delta` ? Le carré part à 120 pixels *par image*, soit 7 200 px/s : il disparaît. Et sa vitesse dépendrait de l'écran (un écran 144 Hz le ferait aller 2,4 fois plus vite). `delta` rend la vitesse indépendante de la machine.
- ...tu mets **Speed** à 400 dans l'Inspecteur sans toucher au script ? Relance : c'est pris en compte.
- ...tu tapes `positon` au lieu de `position` ? L'éditeur souligne en rouge avant même de lancer. C'est l'intérêt du typage.

1.6 · Un premier signal



À FAIRE

1. Ajoute à `Box` un enfant `Timer`. Dans l'Inspecteur : **Wait Time** = 1, **Autostart** coché.
2. Avec le `Timer` sélectionné, ouvre l'onglet **Nœud** (à côté d'Inspecteur) → **Signaux**. Double-clique sur `timeout()`.
3. Dans la fenêtre, choisis `Box` comme destinataire. La méthode proposée s'appelle `_on_timer_timeout`. **Connecter**.
4. Godot ajoute la fonction au script, avec une icône verte dans la marge (connexion faite dans l'éditeur). Complète-la :

```
res://sandbox/box.gd (ajout en bas du fichier)
```

```
func _on_timer_timeout() -> void:
    $ColorRect.color = Color(randf(), randf(), randf())
```

TESTE

`F6` : le carré change de couleur chaque seconde, et bouge toujours avec les flèches.

Comprendre le code

- `$ColorRect` : raccourci de `get_node("ColorRect")`. Le chemin est relatif au nœud qui porte le script. Si tu renomes le nœud, le chemin casse : on verra comment s'en protéger.
- Le `Timer` ne sait pas qui l'écoute. Il « crie » `timeout`, et `Box` a choisi d'écouter. C'est le **patron Observateur**. On peut aussi connecter en code : `$Timer.timeout.connect(_on_timer_timeout)`. Ce parcours utilise surtout cette forme, plus visible quand on relit le script.

CONCEPT : LE DÉCOUPLAGE

Grâce aux signaux, un émetteur n'a pas besoin de connaître ses récepteurs. Plus tard, quand Batman perd de la vie, il émettra un signal : le HUD, le son et l'effet de clignotement réagiront chacun de leur côté. On pourra supprimer le HUD sans que Batman plante.

1.7 · Instancier une scène



À FAIRE

1. Nouvelle scène 2D, racine renommée `Sandbox`, enregistrée en `res://sandbox/sandbox.tscn`.
2. Glisse `box.tscn` depuis le Système de fichiers vers la vue 2D, deux fois. Tu obtiens `Box` et `Box2`, placés où tu as lâché la souris.
3. Sélectionne `Box2` et mets sa **Speed** à 40.
4. `F6`.

TESTE

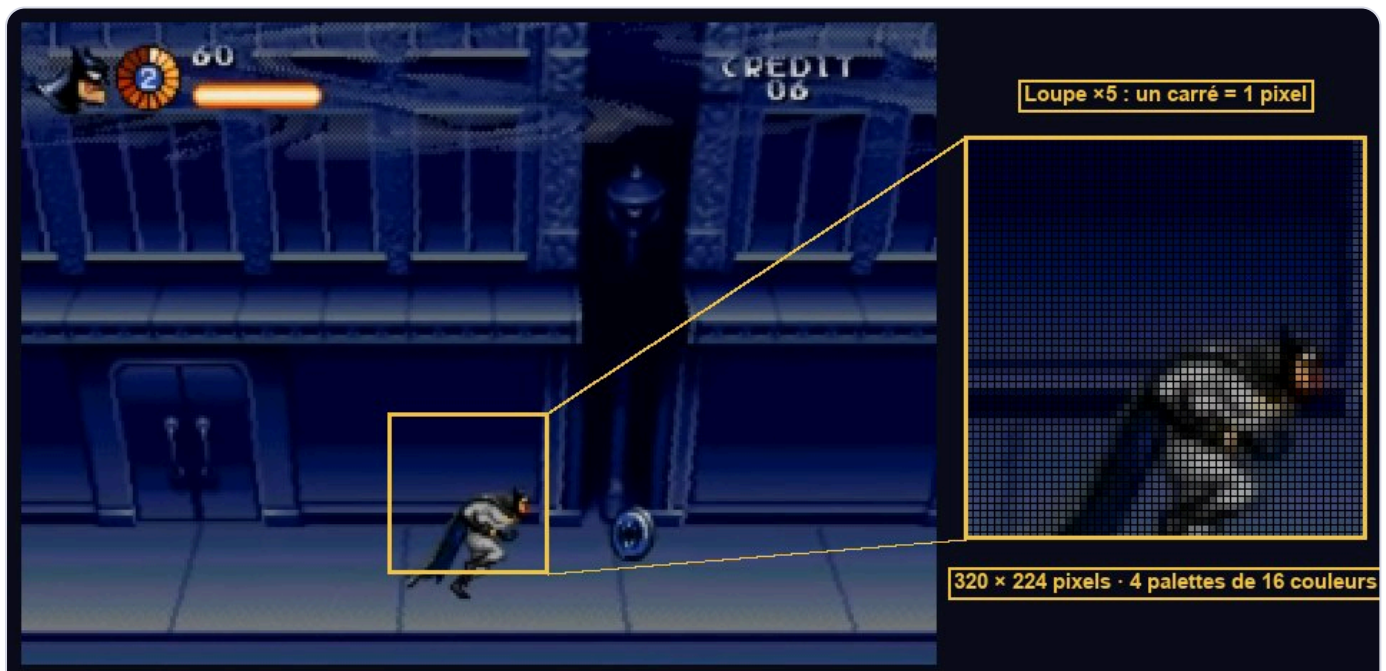
Deux carrés bougent ensemble, l'un trois fois plus lentement, et changent de couleur chacun de leur côté. Ouvre maintenant `box.tscn`, agrandis le `ColorRect` à 24×24, enregistre, relance `sandbox.tscn` : **les deux** instances ont changé.

INSTANCE ET SURCHARGE

Une instance reprend tout de sa scène d'origine. Ce que tu modifies sur l'instance (ici la vitesse de `Box2`) est une **surcharge**, marquée d'une flèche circulaire dans l'Inspecteur ; un clic dessus revient à la valeur de la scène. C'est exactement ainsi qu'on fera un clown « rapide » à partir du clown de base.

LE VRAI PROJET

1.8 · Régler le projet comme une Mega Drive



320 × 224 pixels. Le jeu original affiche cette grille. On veut la même dans Godot : chaque pixel du jeu devient un carré net de 4×4 pixels de ton écran.

À FAIRE

1. **≡ Projet → Paramètres du projet** (*Project → Project Settings*). En haut à droite, active **Paramètres avancés** (*Advanced Settings*).
2. Section **Affichage → Fenêtre** (*Display → Window*) :
 - **Size → Viewport Width** = 320, **Viewport Height** = 224
 - **Size → Window Width Override** = 1280, **Window Height Override** = 896
 - **Stretch → Mode** = `viewport`, **Aspect** = `keep`, **Scale Mode** = `integer`
3. **Rendu → Textures → Canvas Textures → Default Texture Filter** = `Nearest`.
4. **Rendu → 2D → Snap** : coche **Snap 2D Transforms to Pixel** et **Snap 2D Vertices to Pixel**.
5. **Physique → Commun** : **Physics Ticks per Second** = 60, **Max Physics Steps per Frame** = 8 (valeurs par défaut, vérifie-les).
6. **Physique → 2D → Default Gravity** = 900.
7. Ferme la fenêtre : c'est enregistré automatiquement dans `project.godot`.

Pour vérifier, ouvre `project.godot` avec un éditeur de texte : tu dois y trouver ces lignes.

project.godot (extrait)

```
[display]
window/size/viewport_width=320
window/size/viewport_height=224
window/size/window_width_override=1280
window/size/window_height_override=896
window/stretch/mode="viewport"
window/stretch/aspect="keep"
window/stretch/scale_mode="integer"

[physics]
2d/default_gravity=900.0

[rendering]
textures/canvas_textures/default_texture_filter=0
2d/snap/snap_2d_transforms_to_pixel=true
2d/snap/snap_2d_vertices_to_pixel=true
```

Les valeurs restées par défaut (ici les 60 ticks et les 8 pas) n'y figurent pas : Godot n'écrit que ce qui diffère du réglage standard.

CONCEPT : LA RÉOLUTION VIRTUELLE

Le jeu est **calculé et dessiné en 320×224**, puis l'image entière est agrandie ×4. Un sprite ne peut jamais tomber entre deux pixels, et un effet qui travaille « ligne par ligne » (étape 08) voit exactement 224 lignes, comme la console. Le mode `viewport` fait ça ; le mode `canvas_items` dessinerait en haute résolution et lisserait les pixels. L'échelle `integer` interdit les agrandissements non entiers (×3,5), qui déforment les pixels.

TESTE

Relance `sandbox.tscn` : la fenêtre fait 1280×896, et les carrés sont énormes, aux bords parfaitement nets. Agrandis la fenêtre : l'image passe à ×5 seulement quand la place le permet, avec des bandes noires autour.

1.9 · Créer l'arborescence

Tous les chemins `res://` du parcours s'y réfèrent. Crée maintenant ces dossiers vides (clic droit → Nouveau dossier) :

```
res://
├─ autoload/      les singletons (état de la partie, transitions, audio)
├─ core/          les briques génériques réutilisables
├─ player/        Batman et Robin
│   └─ states/
├─ weapons/
│   └─ data/
├─ pickups/       les bonus
├─ enemies/
├─ level/         caméra, arènes, objets de décor
├─ fx/            effets visuels et shaders
├─ bossess/
├─ sections/      les trois sections du niveau
├─ screens/        intro, titre, menus...
├─ ui/            le HUD
├─ art/           tes images
├─ audio/
├─ tools/         scripts d'éditeur
└─ sandbox/       les essais jetables
```

MÉTHODE : RANGER PAR FONCTION

On range par *ce que fait* un fichier (ennemis, armes), pas par *type* (tous les scripts ensemble, toutes les scènes ensemble). Quand tu travailles sur les ennemis, tout ce qu'il te faut est au même endroit.

1.10 · Le premier commit

À FAIRE

1. Ouvre un terminal dans le dossier du projet (celui qui contient `project.godot`).
2. Si Godot ne l'a pas fait : `git init`.
3. Regarde ce qui va être enregistré : `git status`. Le dossier `.godot/` ne doit pas apparaître (c'est un cache que Godot reconstruit ; le `.gitignore` l'exclut).
4. Enregistre :

```
git git add -A && git commit -m "Étape 01 : projet 320x224, bac à sable"
```

MÉTHODE : UN COMMIT PAR ÉTAPE VALIDÉE

Un commit est un point de sauvegarde. Si une modification casse tout, `git diff` montre ce qui a changé depuis le dernier point qui marchait, et `git restore .` y revient. Ce parcours te propose un commit à la fin de chaque étape. Prends l'habitude d'en faire aussi entre deux, dès que quelque chose marche.

SI ÇA NE MARCHE PAS

Les dossiers vides n'existent pas pour Git : ils n'apparaîtront dans tes commits que quand ils contiendront un fichier. Ce n'est pas grave.

POINT DE CONTRÔLE

TU AS FINI L'ÉTAPE QUAND

- ☐ Le bac à sable fonctionne : deux carrés qui bougent et changent de couleur.
- ☐ La fenêtre du jeu s'ouvre en 1280×896, avec des pixels nets.
- ☐ L'arborescence existe et le premier commit est fait.
- ☐ Tu sais expliquer avec tes mots : nœud, scène, instance, script, signal, `delta`.

CE QUE TU RETIENS

? Quelle différence entre une scène et une instance ?

La scène est le modèle, enregistré dans un fichier `.tscn`. Une instance est un exemplaire de ce modèle placé dans une autre scène. Modifier la scène modifie toutes ses instances ; modifier une instance (surcharge) ne touche qu'elle.

? Pourquoi multiplier une vitesse par `delta` ?

Pour exprimer la vitesse en unités par **seconde** et non par image. Sans `delta`, l'objet irait plus vite sur un écran à haute fréquence et ralentirait quand le jeu rame.

? Que se passe-t-il si tu renommes `ColorRect` en `Square` sans changer le script ?

`$ColorRect` ne trouve plus rien et renvoie `null`. Au premier `timeout`, le jeu s'arrête sur l'erreur « Invalid set index 'color' (on base: 'null instance') ». Retiens ce message : c'est presque toujours un chemin de nœud faux.

? Pourquoi le mode d'étirement `viewport` plutôt que `canvas_items` ?

Avec `viewport`, le jeu est réellement rendu en 320×224 puis agrandi : pixels nets, positions toujours entières, et des effets par ligne exacts. `canvas_items` dessine en haute résolution et casserait ces trois propriétés.

POUR ALLER PLUS LOIN

- Empêche le carré de sortir de l'écran avec `position = position.clamp(Vector2.ZERO, Vector2(320, 224))`.
- Ajoute un `@export var color_period := 1.0` et applique-le au `Timer` dans `_ready()` (`$Timer.wait_time = color_period`).
- Lis la page [Nœuds et scènes](#) de la documentation officielle (en français).

LES FONDATIONS

Les briques que tout le reste du jeu utilisera : des entrées abstraites, des couches de collision nommées, un état de partie global, des composants de vie et de dégâts, et une machine à états. Chaque brique est testée seule avant de servir.

~3 h

Prérequis : étape 01

Guide : tome 02

CE QUE TU VAS APPRENDRE

GODOT 4

- La carte des entrées et les scripts d'éditeur (`@tool`)
- `_physics_process` et les ticks physiques
- Couches et masques de collision, `Area2D`
- Autoload, `class_name`, signaux typés
- `@onready`, `owner`, `await`

DÉVELOPPEMENT DE JEU

- Séparer les touches et les actions
- La boucle à pas fixe
- La composition par composants
- Hitbox et hurtbox
- La machine à états finis
- Le banc d'essai : tester une brique seule

LES ENTRÉES

2.1 - Créer une action à la main



CONCEPT : TOUCHES CONTRE ACTIONS

Le code ne doit jamais demander « la touche K est-elle enfoncée ? », mais « le joueur veut-il sauter ? ». On déclare des **actions** (`jump`, `attack` ...) et on leur associe des touches et des boutons de manette. Changer de clavier, brancher une manette ou laisser le joueur reconfigurer ne demande alors aucune modification du code.

À FAIRE

1. ≡ **Projet** → **Paramètres du projet** → **Contrôles** (*Input Map*).
2. Dans le champ « Ajouter une nouvelle action », tape `jump`, puis **Ajouter**.
3. Clique sur le **+** à droite de `jump`, appuie sur `K`, valide. Recommence avec `Espace`.
4. Observe l'option « Physical Keycode » : elle désigne la **position** de la touche, pas la lettre imprimée dessus.

Il faut ainsi sept actions, chacune avec plusieurs touches et boutons. Les saisir à la main est fastidieux et source d'oublis : on va l'automatiser.

2.2 · Un script d'éditeur pour toutes les actions



Action	Clavier (position physique)	Manette	Mega Drive
<code>move_left</code> / <code>move_right</code> / <code>move_up</code> / <code>move_down</code>	Flèches ou W A S D (= Z Q S D sur AZERTY)	Croix + stick gauche	Croix
<code>attack</code>	J et L	X et B (Xbox)	A et C
<code>jump</code>	K et Espace	A (Xbox)	B
<code>start</code>	Entrée	Start / Menu	Start

À FAIRE

1. Dans le Système de fichiers, clic droit sur `res://tools/` → **Nouveau script**. Hérite de `EditorScript`, nom `setup_inputs.gd`.
2. Remplace son contenu par le code ci-dessous.
3. Script ouvert dans l'éditeur de script : ≡ **Fichier** → **Exécuter** (`Ctrl Maj X` / `⌘ Maj X`). La Sortie affiche « Actions créées : ... ».
4. ≡ **Projet** → **Recharger le projet actuel**, puis rouvre la carte des entrées : les sept actions y sont.

```

@tool
extends EditorScript

## Crée les actions d'entrée du projet. À exécuter une seule fois (Fichier > Exécuter),
## puis Projet > Recharger le projet en cours.

func _run() -> void:
    var actions := {
        "move_left": [_key(KEY_LEFT), _key(KEY_A), _pad(JOY_BUTTON_DPAD_LEFT),
            _axis(JOY_AXIS_LEFT_X, -1.0)],
        "move_right": [_key(KEY_RIGHT), _key(KEY_D), _pad(JOY_BUTTON_DPAD_RIGHT),
            _axis(JOY_AXIS_LEFT_X, 1.0)],
        "move_up": [_key(KEY_UP), _key(KEY_W), _pad(JOY_BUTTON_DPAD_UP), _axis(JOY_AXIS_LEFT_Y,
            -1.0)],
        "move_down": [_key(KEY_DOWN), _key(KEY_S), _pad(JOY_BUTTON_DPAD_DOWN),
            _axis(JOY_AXIS_LEFT_Y, 1.0)],
        "attack": [_key(KEY_J), _key(KEY_L), _pad(JOY_BUTTON_X), _pad(JOY_BUTTON_B)],
        "jump": [_key(KEY_K), _key(KEY_SPACE), _pad(JOY_BUTTON_A)],
        "start": [_key(KEY_ENTER), _pad(JOY_BUTTON_START)],
    }
    for action_name: String in actions:
        ProjectSettings.set_setting("input/" + action_name, {
            "deadzone": 0.4,
            "events": actions[action_name],
        })
    ProjectSettings.save()
    print("Actions créées : ", actions.keys())

func _key(code: Key) -> InputEventKey:
    var e := InputEventKey.new()
    e.physical_keycode = code # position physique : WASD devient ZQSD sur AZERTY
    return e

func _pad(button: JoyButton) -> InputEventJoypadButton:
    var e := InputEventJoypadButton.new()
    e.button_index = button
    return e

func _axis(axis: JoyAxis, value: float) -> InputEventJoypadMotion:
    var e := InputEventJoypadMotion.new()
    e.axis = axis
    e.axis_value = value
    return e

```

Comprendre le code

- `@tool` + `extends EditorScript` : ce script ne tourne pas dans le jeu mais **dans l'éditeur**, quand tu demandes à l'exécuter. Sa fonction d'entrée est `_run()`.
- Une action est enregistrée dans les paramètres du projet sous la clé `input/nom`, sous forme d'un dictionnaire : une zone morte et une liste d'événements. Le script écrit exactement ce que l'interface écrirait.
- `for action_name: String in actions` : on parcourt les clés du dictionnaire, en typant la variable de boucle.
- `physical_keycode` : la touche à l'emplacement de W sur un clavier QWERTY, c'est-à-dire Z sur un AZERTY. Les commandes restent bien placées quelle que soit la langue du clavier.

MÉTHODE : AUTOMATISER CE QUI SE RÉPÈTE

Dès qu'une manipulation de l'éditeur est longue, répétitive ou facile à rater, un petit script d'éditeur la rend fiable et reproductible. Tu en écriras un autre à l'étape 08, pour convertir des images en couleurs indexées.

LA BOUCLE À PAS FIXE

2.3 · `_process` ou `_physics_process` ?



CONCEPT : LE PAS DE TEMPS FIXE

`_process(delta)` suit l'écran : son `delta` varie (0,016 s à 60 Hz, 0,007 s à 144 Hz, 0,05 s si le jeu rame). `_physics_process(delta)` est appelé à **intervalle fixe** : 60 fois par seconde avec nos réglages, quoi qu'il arrive. Les collisions, les sauts et l'IA y sont reproductibles : un saut a toujours la même hauteur. La Mega Drive faisait pareil : un « pas » de jeu par image (interruption VBlank, 60 Hz).

Règle du parcours : tout le gameplay va dans `_physics_process()`. `_process()` ne sert qu'aux effets purement visuels (clignotements, interface).

Les vitesses du jeu original se mesurent en **pixels par image**. Comme un tick physique dure une image, la conversion est directe :

Conversion (exemple, à ne pas créer)

```
# Mesuré dans la vidéo : la caméra défile à ~1,95 px par image quand Batman court.
const MD_FPS := 60.0
const RUN_SPEED := 1.95 * MD_FPS # ≈ 117 px/s

func _physics_process(delta: float) -> void:
    position.x += RUN_SPEED * delta # 117 px/s × 1/60 s = 1,95 px par tick
```

EXERCICE

1. Dans `res://sandbox/box.gd`, renomme `_process` en `_physics_process`. Lance : rien ne change en apparence.
2. Ouvre le **Débugueur** → **Moniteurs** pendant que le jeu tourne et coche **Time** → **Physics Process** et **Time** → **FPS**. Les ticks physiques restent à 60 par seconde.

LES COUCHES DE COLLISION

2.4 · Nommer les couches



Les couches de collision, posées sur une vraie image du jeu. Le sol (1), le balcon traversable (2), le rebord d'accroche (8), le corps de Batman (3), ceux des clowns (4) et un bonus (7).

CONCEPT : COUCHES ET MASQUES

Chaque objet physique **est** sur certaines couches (`collision_layer`) et **regarde** certaines couches (`collision_mask`). A détecte B si le masque de A contient une couche de B. Un batarang regarde seulement les zones vulnérables des ennemis : il ignore les murs, les autres batarangs et Batman. Bien régler les masques évite des bugs et des calculs inutiles.

N°	Nom	Qui est dessus	Qui la regarde
1	monde	Sols, murs	Corps du joueur et des ennemis
2	plateformes	Balcon, plateaux de camions (traversables par-dessous)	Corps du joueur et des ennemis
3	corps_joueur	Le corps de Batman	Bonus, déclencheurs, prise du costaud, zone de chute
4	corps_ennemis	Le corps des ennemis	Personne dans le niveau 1
5	hurtbox_joueur	Zone vulnérable de Batman	Coups, balles et bombes ennemis
6	hurtbox_ennemis	Zones vulnérables des ennemis, boss, objets cassables	Poing et projectiles de Batman
7	bonus	Zone de ramassage des bonus	Personne (c'est le bonus qui regarde la couche 3)
8	rebords	Zones d'accroche sous le balcon	La sonde « mains » de Batman

À FAIRE

1. **≡ Projet → Paramètres du projet → Noms de calques → Physique 2D** (*Layer Names → 2D Physics*).
2. Saisis les huit noms du tableau (`monde`, `plateformes` ...). Désormais, l'Inspecteur affiche ces noms au survol des cases de couche.
3. Crée `res://core/layers.gd` :

```
res://core/layers.gd
```

```
class_name Layers
## Masques de bits des couches physiques (couche n => bit n-1).

const WORLD := 1 << 0
const PLATFORMS := 1 << 1
const PLAYER_BODY := 1 << 2
const ENEMY_BODY := 1 << 3
const PLAYER_HURTBOX := 1 << 4
const ENEMY_HURTBOX := 1 << 5
const PICKUPS := 1 << 6
const LEDGES := 1 << 7
```

`CLASS_NAME`

`class_name Layers` enregistre le script comme un **type global** : partout dans le projet, on peut écrire `Layers.PICKUPS` sans rien importer. Sur un script qui étend un nœud, `class_name` le fait aussi apparaître dans la fenêtre « Ajouter un nœud », comme un nœud natif. Un masque est un entier dont chaque bit vaut une couche : la couche 3 est le bit 2, soit `1 << 2 = 4`.

L'ÉTAT DE LA PARTIE

2.5 · L'autoload Game



AUTOLOAD

Quand on passe d'une scène à une autre, l'ancienne est détruite, avec toutes ses variables. Le score, les vies et l'arme en main doivent survivre : on les range dans un **autoload**, un nœud que Godot crée au démarrage, place à la racine, et ne détruit jamais. Son nom devient global : `Game.score` marche depuis n'importe quel script.

À FAIRE

1. Crée `res://autoload/game.gd` (hérite de `Node`) avec le code ci-dessous.
2. **Projet** → **Paramètres du projet** → **Globaux** → **Autoload** : chemin `res://autoload/game.gd`, nom `Game`, **Ajouter**. Vérifie que « Activer » est coché.

res://autoload/game.gd (Autoload « Game »)

`extends Node`

État global de la partie. Le HUD l'écoute via ses signaux (patron Observateur).

`signal score_changed(score: int)`

`signal lives_changed(lives: int)`

`signal credits_changed(credits: int)`

`signal weapon_changed(weapon_id: StringName, level: int)`

`enum Hero { BATMAN, ROBIN }`

`const START_LIVES := 2` *# constaté : « 2 » dans le médaillon en début de partie*

`const START_CREDITS := 6` *# constaté : « CREDIT 06 »*

`const MAX_WEAPON_LEVEL := 7` *# TASVideos : niveau maximal 7*

À CALIBRER : les vies montent à 3 puis 4 pendant la section 2 (palier de score ?).

`var extra_life_every := 20000`

`var hero: Hero = Hero.BATMAN`

`var score := 0`

`var lives := START_LIVES`

`var credits := START_CREDITS`

Crédits accordés à chaque nouvelle partie. Réglé par l'écran OPTIONS (tome 10).

`var credits_setting := START_CREDITS`

`var weapon_id: StringName = &"batarang"`

`var weapon_level := 1`

`var current_section := ""` *## chemin de la section en cours : point de reprise du « continue »*

`var _next_extra_life := 0`

`func new_game(selected: Hero) -> void:`

`hero = selected`

`credits = credits_setting`

`_reset_run()`

```

func continue_game() -> bool:
    ## Renvoie false s'il ne reste plus de crédit.
    if credits <= 0:
        return false
    credits -= 1
    credits_changed.emit(credits)
    _reset_run()
    return true

func _reset_run() -> void:
    score = 0
    lives = START_LIVES
    weapon_id = &"batarang"
    weapon_level = 1
    _next_extra_life = extra_life_every
    score_changed.emit(score)
    lives_changed.emit(lives)
    weapon_changed.emit(weapon_id, weapon_level)

func add_score(points: int) -> void:
    score += points
    score_changed.emit(score)
    while extra_life_every > 0 and score >= _next_extra_life:
        _next_extra_life += extra_life_every
        add_life()

func add_life() -> void:
    lives += 1
    lives_changed.emit(lives)

## Appelé à la mort du joueur. Renvoie true s'il peut réapparaître.
func lose_life() -> bool:
    if lives <= 0:
        return false
    lives -= 1
    lives_changed.emit(lives)
    return true

func set_weapon(id: StringName, level: int) -> void:
    weapon_id = id
    weapon_level = clampi(level, 1, MAX_WEAPON_LEVEL)
    weapon_changed.emit(weapon_id, weapon_level)

```

Comprendre le code

- `signal score_changed(score: int)` : un signal déclaré dans le script, avec un paramètre typé. On l'émet avec `score_changed.emit(score)`.
- `enum Hero { BATMAN, ROBIN }` : des noms pour des entiers (0 et 1). `Game.Hero.ROBIN` se lit mieux que `1`.
- `&"batarang"` : un `StringName`, une chaîne « internée » que Godot compare très vite. On l'utilise pour les identifiants (armes, états, animations).

- `credits` et `credits_setting` : le premier est l'état de la partie (consommé par les continues), le second un réglage (choisi dans les options). Les séparer évite qu'un nouveau départ efface le réglage.
- `while ... add_life()` : un `while` plutôt qu'un `if`, au cas où un gros bonus franchirait deux paliers d'un coup.
- Aucune fonction ne touche à l'affichage. `Game` annonce ce qui change ; c'est au HUD (étape 10) d'écouter.

2.6 · Tester le patron Observateur



À FAIRE

1. Dans `res://sandbox/`, crée une scène 2D `observer_test.tscn` avec un script sur la racine :

```
res://sandbox/observer_test.gd
```

```
extends Node2D
## Banc d'essai jetable : on écoute Game, puis on le fait changer.

func _ready() -> void:
    Game.score_changed.connect(func(s: int) -> void: print("Score : ", s))
    Game.lives_changed.connect(func(l: int) -> void: print("Vies : ", l))
    Game.add_score(15000)
    Game.add_score(6000)    # franchit le palier de 20 000 : une vie de plus
```

TESTE

F6. La Sortie affiche « Score : 15000 », puis « Score : 21000 » et « Vies : 3 ». Pendant que le jeu tourne, ouvre l'onglet **Distant** (*Remote*) au-dessus de l'arbre de scène : tu vois le nœud `Game` à la racine, à côté de ta scène. C'est l'arbre réel du jeu en cours ; clique sur `Game` pour lire ses variables en direct.

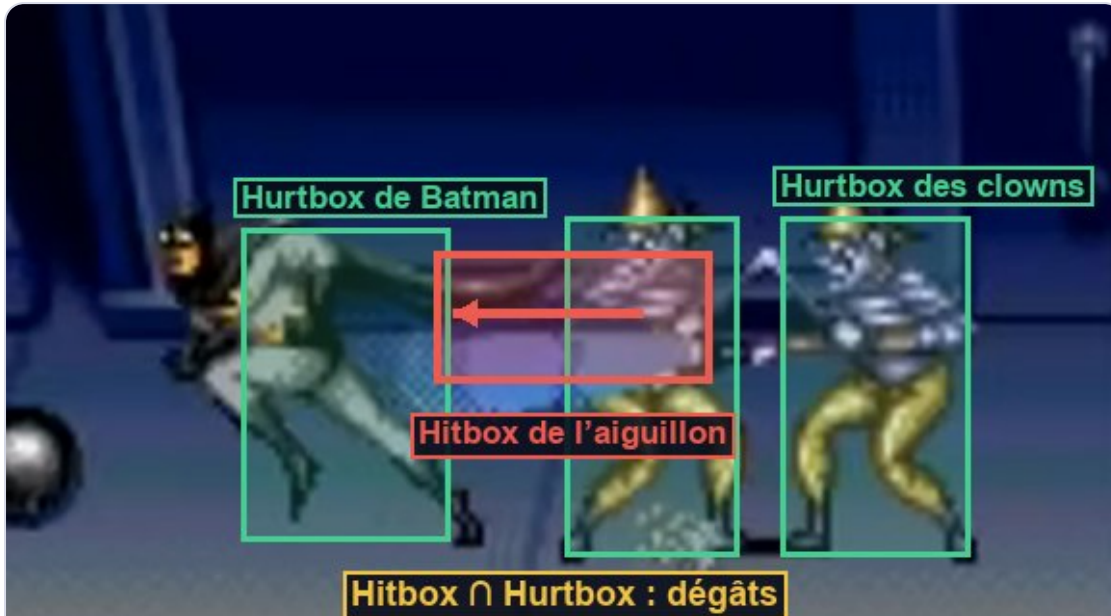
LES FONCTIONS ANONYMES

`func(s: int) -> void: print(...)` est une **lambda** : une fonction sans nom, définie sur place. Pratique pour une réaction d'une ligne. Pour plus long, on écrit une vraie méthode et on la passe : `Game.score_changed.connect(_on_score)`.

LES COMPOSANTS

CONCEPT : COMPOSITION PLUTÔT QU'HÉRITAGE

Batman, un clown, une poubelle et la montgolfière du Joker ont tous de la vie et peuvent être touchés. Plutôt que d'écrire quatre fois la même logique, ou d'imaginer une hiérarchie de classes géante, on fabrique de petits **composants** (un nœud, un rôle) qu'on assemble dans chaque scène, comme des pièces de Lego. Besoin de vie ? On ajoute un nœud `Health`.



Hitbox contre Hurtbox. L'aiguillon du clown (rouge) est une Hitbox ; il ne fait mal que s'il chevauche la Hurtbox de Batman (vert). Le corps qui sert à marcher est une troisième forme, séparée.

Crée ces trois fichiers dans `res://core/`.

`res://core/health.gd`

```
class_name Health
extends Node
## Points de vie génériques : joueur, ennemis et boss.

signal changed(current: int, maximum: int)
signal damaged(amount: int)
signal died

@export var max_health := 8
var current := 0
var invincible := false

func _ready() -> void:
    current = max_health

func is_dead() -> bool:
    return current <= 0

func damage(amount: int) -> void:
    if invincible or is_dead() or amount <= 0:
        return
    current = maxi(current - amount, 0)
    damaged.emit(amount)
    changed.emit(current, max_health)
    if current == 0:
        died.emit()
```

```

func heal(amount: int) -> void:
    if is_dead():
        return
    current = mini(current + amount, max_health)
    changed.emit(current, max_health)

```

```

func reset() -> void:
    current = max_health
    changed.emit(current, max_health)

```

res://core/hurtbox.gd

```

class_name Hurtbox
extends Area2D
## Zone vulnérable. Ne détecte rien elle-même : ce sont les Hitbox qui la trouvent.

signal hurt(hitbox: Hitbox)

@export var health: Health
## Pendant l'invincibilité (clignotement), les coups sont ignorés.
var invincible := false

func _ready() -> void:
    monitoring = false # une Hurtbox est « vue », elle ne regarde pas

func take_hit(hitbox: Hitbox) -> bool:
    ## Renvoie true si le coup a porté.
    if invincible or (health != null and health.is_dead()):
        return false
    if health != null:
        health.damage(hitbox.damage)
    hurt.emit(hitbox)
    return true

```

```

class_name Hitbox
extends Area2D
## Zone qui inflige des dégâts aux Hurtbox présentes dans son masque de collision.

signal landed(hurtbox: Hurtbox)

@export var damage := 1
## Poussée appliquée à la cible (x orienté vers l'avant de l'attaquant).
@export var knockback := Vector2(120, -80)
## Pour un projectile : se désactive après le premier impact.
@export var single_hit := false
## Sens de l'attaquant : 1 vers la droite, -1 vers la gauche (mis à jour par le porteur).
var facing := 1
var _already_hit: Array[Hurtbox] = []

func _ready() -> void:
    monitorable = false # personne n'a besoin de « voir » une hitbox
    area_entered.connect(_on_area_entered)

## À appeler à chaque nouvelle attaque (coup de poing suivant, projectile recyclé).
func reset_hits() -> void:
    _already_hit.clear()

func _on_area_entered(area: Area2D) -> void:
    var hurtbox := area as Hurtbox
    if hurtbox == null or hurtbox in _already_hit:
        return
    _already_hit.append(hurtbox)
    if hurtbox.take_hit(self):
        landed.emit(hurtbox)
        if single_hit:
            set_deferred("monitoring", false)

func get_knockback() -> Vector2:
    return Vector2(knockback.x * facing, knockback.y)

```

Comprendre le code

- **Qui regarde qui** : une `Area2D` a deux interrupteurs. `monitoring` = « je détecte les autres », `monitorable` = « les autres peuvent me détecter ». La Hitbox détecte, la Hurtbox est détectée : chacun coupe l'interrupteur dont il n'a pas besoin.
- `@export var health: Health` : un **export de nœud**. Dans l'Inspecteur, tu désignes le nœud `Health` en cliquant ou en le glissant. Pas de chemin écrit en dur : renommer le nœud ne casse rien.
- `area as Hurtbox` : renvoie l'objet s'il est bien une `Hurtbox`, sinon `null`. On ignore ainsi toutes les autres zones.
- `_already_hit` : sans cette liste, un poing qui reste 10 images sur un ennemi le toucherait 10 fois. Chaque attaque touche une cible au plus une fois, jusqu'au prochain `reset_hits()`.
- `set_deferred("monitoring", false)` : pendant un signal de collision, Godot interdit de modifier l'état physique (erreur « Function blocked during in/out signal »). `set_deferred` reporte la modification à la fin de l'image.

2.8 · Un banc d'essai pour les dégâts



MÉTHODE : LE BANC D'ESSAI

Avant d'utiliser une brique dans le vrai jeu, on la teste seule, dans une scène minuscule qui ne sert qu'à ça. Si quelque chose cloche plus tard, tu sauras que ce n'est pas elle. Les professionnels font la même chose avec des tests automatisés et des « niveaux de test » remplis de cas limites.

À FAIRE

1. Nouvelle scène 2D `res://sandbox/hit_test.tscn`, racine `HitTest`. Construis cet arbre (les types `Health`, `Hurtbox`, `Hitbox` apparaissent dans « Ajouter un nœud » grâce à `class_name`) :

```
HitTest (Node2D, hit_test.gd)
├── Target (Node2D)
│   ├── Health
│   │   └── max_health = 3
│   ├── Hurtbox
│   │   ├── CollisionShape2D
│   │   │   └── RectangleShape2D 16×32
│   │   └── ColorRect
│   │       └── 16×32 · position (-8, -16) · rouge
│   └── Fist (Hitbox)
│       ├── position (20, 112) · couche vide · masque 6 seul
│       └── CollisionShape2D
│           └── CircleShape2D rayon 6
```

2. Pour régler couches et masques : Inspecteur → **Collision** → clique sur les cases numérotées. Une case allumée = couche active.
3. Pour `health` sur la Hurtbox : clique sur « Assigner... » et choisis `Health`.
4. Attache ce script à `HitTest` :

res://sandbox/hit_test.gd

```
extends Node2D
## Banc d'essai : un « poing » traverse une cible en boucle.

@onready var fist: Hitbox = $Fist
@onready var health: Health = $Target/Health

func _ready() -> void:
    health.damaged.connect(func(a: int) -> void: print("Touché ! -", a, " → reste ",
        health.current))
    health.died.connect(func() -> void: print("Cible détruite"))

func _physics_process(delta: float) -> void:
    fist.position.x += 80.0 * delta
    if fist.position.x > 300.0:
        fist.position.x = 20.0
        fist.reset_hits() # nouvelle « attaque » : le poing peut retoucher
```

TESTE

Active ≡ **Déboguer** → **Formes de collision visibles** (*Debug* → *Visible Collision Shapes*), puis `F6`. Le cercle traverse la cible trois fois : « reste 2 », « reste 1 », « reste 0 » puis « Cible détruite ». Ensuite, plus rien : une cible morte n'encaisse plus.

SI ÇA NE MARCHE PAS

- **Rien ne s'affiche** : vérifie que le masque du `Fist` contient la couche 6 et que la `Hurtbox` est sur la couche 6. C'est l'erreur la plus fréquente de tout le projet.
- **« Invalid get index 'damaged' (on base: 'null instance') »** : `$Target/Health` ne trouve pas le nœud. Vérifie les noms exacts, majuscules comprises.
- **Touché, mais « reste » ne baisse pas** : le champ `health` de la `Hurtbox` n'est pas assigné.

@ONREADY

Une variable `@onready` est initialisée juste avant `_ready()`, quand les enfants existent. Écrire `var fist: Hitbox = $Fist` sans `@onready` échouerait : à la création du script, l'enfant n'est pas encore là.

LA MACHINE À ÉTATS

2.9 • State et StateMachine



CONCEPT : LA MACHINE À ÉTATS FINIS

Un personnage est toujours dans **un seul état** à la fois : au sol, en l'air, suspendu, touché... Chaque état a son propre script avec trois moments : `enter()` à l'entrée, `physics_update()` à chaque tick, `exit()` à la sortie. Un état décide lui-même quand passer au suivant. On évite ainsi les forêts de `if is_jumping and not is_hurt and not is_hanging...`. Tu sentiras concrètement le problème à l'étape 03.

res://core/state.gd

```
class_name State
extends Node
## Classe de base d'un état. Les états concrets surchargent les fonctions utiles.

## Émis pour demander une transition. Toujours émis via transition_to().
signal finished(next_state: StringName, data: Dictionary)

func enter(_previous: StringName, _data: Dictionary) -> void:
    pass

func exit() -> void:
    pass

func update(_delta: float) -> void:
    pass

func physics_update(_delta: float) -> void:
    pass

func transition_to(next_state: StringName, data: Dictionary = {}) -> void:
    finished.emit(next_state, data)
```

```

class_name StateMachine
extends Node
## Gère des états enfants. Le propriétaire appelle physics_update() depuis son _physics_process.

signal state_changed(previous: StringName, current: StringName)

@export var initial_state: State
var state: State
var _states: Dictionary = {} # StringName -> State

func _ready() -> void:
    for child in get_children():
        if child is State:
            _states[child.name] = child
            child.finished.connect(_on_state_finished)
    if initial_state == null and get_child_count() > 0:
        initial_state = get_child(0) as State
    # On attend que le propriétaire soit prêt (ses @onready sont résolus après nous).
    await owner.ready
    state = initial_state
    state.enter("", {})

func physics_update(delta: float) -> void:
    if state:
        state.physics_update(delta)

func update(delta: float) -> void:
    if state:
        state.update(delta)

func force(next_state: StringName, data: Dictionary = {}) -> void:
    ## Transition imposée de l'extérieur (dégâts, mort, cinématique).
    _on_state_finished(next_state, data)

func current_name() -> StringName:
    return state.name if state else ""

func _on_state_finished(next_state: StringName, data: Dictionary) -> void:
    if not _states.has(next_state):
        push_error("État inconnu : %s" % next_state)
        return
    var previous: StringName = state.name if state else ""
    if state:
        state.exit()
    state = _states[next_state]
    state.enter(previous, data)
    state_changed.emit(previous, next_state)

```

Comprendre le code

- Les états sont des **nœuds enfants** de la machine. Le **nom du nœud** sert d'identifiant : `transition_to(&"Air")` active l'enfant nommé `Air`.
- Un état ne change pas l'état lui-même : il *demande*, via le signal `finished`. Seule la machine fait `exit()` puis `enter()`, toujours dans cet ordre.
- `data` : un dictionnaire pour passer une information au prochain état (par exemple la hauteur du rebord auquel s'accrocher).
- `owner` : la racine de la scène enregistrée qui contient ce nœud. Pour un état de Batman, `owner` est Batman.
- `await owner.ready` : dans Godot, les enfants sont prêts *avant* leur parent. La machine suspend son `_ready()` jusqu'à ce que le propriétaire soit prêt, pour que le premier `enter()` trouve tout en place.

AWAIT

`await signal` met la fonction en pause jusqu'à l'émission du signal, sans bloquer le jeu : le reste continue de tourner. La fonction devient une **coroutine**. C'est très pratique, mais dangereux pour la logique de jeu : si l'objet change d'avis pendant l'attente, la fonction se réveille quand même. On l'utilisera surtout pour des séquences (cinématiques, transitions).

2.10 · Tester la machine sur un feu tricolore



Une machine à états se teste mieux sur un objet trivial que sur un personnage. Un feu tricolore a trois états et une règle simple.

À FAIRE

- Crée `res://sandbox/light_state.gd` :

`res://sandbox/light_state.gd`

```
extends State
## Un état de feu tricolore : une couleur, une durée, l'état suivant.

@export var color := Color.GREEN
@export var duration := 2.0
@export var next: StringName = &"Orange"
var _t := 0.0

func enter(previous: StringName, _data: Dictionary) -> void:
    _t = 0.0
    (owner.get_node("Lamp") as ColorRect).color = color
    print("%s → %s" % [previous, name])

func physics_update(delta: float) -> void:
    _t += delta
    if _t >= duration:
        transition_to(next)
```

```
res://sandbox/traffic_light.gd
```

```
extends Node2D
## Le propriétaire fait avancer la machine à chaque tick.

@onready var fsm: StateMachine = $StateMachine

func _physics_process(delta: float) -> void:
    fsm.physics_update(delta)
```

À FAIRE (SUITE)

2. Nouvelle scène `res://sandbox/traffic_light.tscn` :

```
TrafficLight (Node2D, traffic_light.gd)    position (160, 112)
├─ Lamp (ColorRect)                       24×24 · position (-12, -12)
├─ StateMachine (StateMachine)             initial_state = Green
│   ├─ Green (Node, light_state.gd)         color vert · duration 2 · next = Orange
│   ├─ Orange (Node, light_state.gd)        color orange · duration 0.7 · next = Red
│   └─ Red (Node, light_state.gd)           color rouge · duration 2 · next = Green
```

3. Pour attacher le même script à trois nœuds : sélectionne-les tous (**Maj** + clic), puis glisse `light_state.gd` depuis le Système de fichiers sur la sélection. Règle ensuite chaque nœud dans l'Inspecteur.

TESTE

F6 . Le carré passe vert → orange → rouge → vert... La Sortie affiche « → Green », « Green → Orange », « Orange → Red »... Change `next` de `Red` en `Blue` : au passage, la Sortie affiche « État inconnu : Blue » en rouge et le feu reste bloqué. Remets `Green` .

CE QUE MONTRE CET EXEMPLE

Un seul script sert trois états, parce que ces états ne diffèrent que par des **données** (couleur, durée, suivant). Garde ce réflexe : avant d'écrire un nouveau script, demande-toi si ce n'est pas le même comportement avec d'autres chiffres.

LA SALLE DE TEST

2.11 · Construire `test_room.tscn`



Batman y fera ses premiers pas à l'étape suivante. Elle te servira jusqu'à l'étape 08.

À FAIRE

1. Nouvelle scène 2D `res://level/test_room.tscn`, racine `TestRoom`.
2. Ajoute un `StaticBody2D` nommé `Floor` : couche 1 seule, masque vide. Enfants : un `CollisionShape2D` avec un `RectangleShape2D` de 320×16, et un `ColorRect` gris de 320×16 en position (-160, -8). Place `Floor` en (160, 208).
3. Ajoute un `StaticBody2D` nommé `Balcony` : couche 2 seule. Enfants : `CollisionShape2D` rectangle 200×8 avec **One Way Collision** coché, et un `ColorRect` brun 200×8 en (-100, -4). Place `Balcony` en (200, 112).
4. Ajoute une `Camera2D` en (160, 112).

```
TestRoom (Node2D)
├── Floor (StaticBody2D)           couche 1 « monde » · (160, 208)
│   ├── CollisionShape2D         rectangle 320×16
│   └── ColorRect                320×16, gris
├── Balcony (StaticBody2D)       couche 2 « plateformes » · (200, 112)
│   ├── CollisionShape2D         rectangle 200×8, One Way Collision activé
│   └── ColorRect                200×8, brun
└── Camera2D                    (160, 112)
```

LES CORPS PHYSIQUES 2D

- `StaticBody2D` : ne bouge pas (sol, murs).
- `CharacterBody2D` : déplacé par ton code avec `move_and_slide()` (Batman, ennemis).
- `AnimatableBody2D` : déplacé par ton code ou une animation, et emporte ce qui est posé dessus (camions, étape 12).
- `RigidBody2D` : entièrement géré par la simulation physique. Jamais utilisé ici : un jeu d'arcade veut un contrôle exact, pas une simulation réaliste.
- `Area2D` : ne bloque rien, détecte seulement (hitbox, déclencheurs).

TESTE

`F6` : un sol gris en bas et un balcon brun au milieu. Rien ne bouge, c'est normal.

POINT DE CONTRÔLE

TU AS FINI L'ÉTAPE QUAND

- ☐ Les actions `move_*`, `jump`, `attack` et `start` existent dans la carte des entrées.
- ☐ `Game` est accessible depuis n'importe quel script, et le banc d'essai observateur affiche le score et les vies.
- ☐ Le banc d'essai des dégâts : trois coups puis « Cible détruite ».
- ☐ Le feu tricolore change d'état en boucle.
- ☐ Les scripts de `res://core/` se chargent sans erreur et la salle de test existe.

`git` `git add -A && git commit -m "Étape 02 : entrées, couches, Game, composants, machine à états"`

CE QUE TU RETIENS

? Un batarang ne touche pas un clown. Quelles sont les deux premières choses à vérifier ?

Que le **masque** du batarang contient la couche 6, et que la **Hurtbox** du clown est sur la couche 6. Ensuite seulement : que la Hitbox est bien `monitoring`, et que les formes de collision existent (Débuguer → Formes de collision visibles).

? Pourquoi le gameplay dans `_physics_process` et pas dans `_process` ?

Parce que `_physics_process` tourne à intervalle fixe (60 Hz). Les calculs de saut, de collision et d'IA donnent donc toujours le même résultat, quelle que soit la machine. `_process` suit l'affichage et son `delta` varie.

? À quoi sert de séparer Hitbox, Hurtbox et forme de collision du corps ?

Chacune a un rôle différent : le corps sert à marcher et à buter contre les murs, la Hurtbox à encaisser, la Hitbox à frapper. On peut ainsi rendre Batman invincible (Hurtbox ignorée) sans qu'il traverse le sol, ou faire un poing qui dépasse du corps.

? Dans une machine à états, qui a le droit d'appeler `enter()` et `exit()` ?

Uniquement la machine. Les états demandent une transition (`transition_to`) ; l'extérieur peut l'imposer (`force`). Ainsi, la sortie de l'ancien état précède toujours l'entrée du nouveau.

POUR ALLER PLUS LOIN

- Ajoute au feu tricolore un état `Blink` (orange clignotant) atteint quand on appuie sur `start`. Indice : la touche se lit dans `physics_update` d'un état, ou le propriétaire appelle `fsm.force(&"Blink")`.
- Dans le banc d'essai des dégâts, connecte `Hitbox.landed` pour afficher le nom de la cible touchée.
- Lis le tome 02 du guide, section « De la Mega Drive à Godot » : quelle pièce de la console correspond à quel outil de Godot.

PROTOTYPE : BATMAN EN UN SEUL SCRIPT

On ne commence pas par l'architecture parfaite : on commence par *sentir* le personnage. Un rectangle bleu, trente lignes de code, et on règle la course et le saut jusqu'à ce que ça « tombe juste ». Puis on essaie d'aller plus loin, et on voit le code devenir ingérable.

⌚ ~1 h 30

Prérequis : étape 02

Guide : [tome 01](#) , [tome 03](#)

CE QUE TU VAS APPRENDRE

GODOT 4

- `CharacterBody2D` , `velocity` , `move_and_slide()`
- `is_on_floor()` , collisions à sens unique
- `Input.get_axis()` , `is_action_just_pressed/released`
- `@export_range`

DÉVELOPPEMENT DE JEU

- Observer et mesurer la référence
- Le prototype jetable
- La physique « arcade » : gravité, saut à hauteur variable
- Le réglage du ressenti (*game feel*)
- Reconnaître le moment de restructurer

OBSERVER AVANT DE CODER

3.1 · Regarder Batman bouger



MÉTHODE : OBSERVER, MESURER, CODER, COMPARER

Quand on recrée un jeu, la référence est sous tes yeux. Avant d'écrire une ligne, regarde le personnage à vitesse normale, puis au ralenti (sur YouTube, `,` et `.` avancent image par image quand la vidéo est en pause). Note ce que tu vois. Pour ton propre jeu, la « référence » sera un jeu que tu aimes, ou une maquette.

DANS LA VIDÉO

- `▶ 1:47` : Batman court. Il démarre et s'arrête net, sans glisser.
- `▶ 2:05` : un saut. Observe la hauteur par rapport au balcon.
- `▶ 3:30` : suspendu sous le balcon. C'est comme ça qu'on y monte.
- `▶ 4:42` : une longue course dans une rue vide, celle qui a servi aux mesures.

Le guide a mesuré ces valeurs image par image ([tome 01, mesures](#)) :

Mesure	Valeur	Dans Godot
Vitesse de course	≈ 1,95 px par image	1,95 × 60 = 117 px/s
Accélération	aucune visible	vitesse pleine dès le premier tick
Hauteur de saut	non mesurable avec précision	≈ 60 px, à régler à l'œil

LE PROTOTYPE

3.2 · Le corps de Batman



À FAIRE

1. Nouvelle scène. Racine : **Autre nœud** → `CharacterBody2D`, renommé `PlayerProto`. Enregistre en `res://sandbox/player_proto.tscn`.
2. Inspecteur → **Collision** : couche 3 seule (`corps_joueur`), masque 1 et 2 (`monde` , `plateformes`).
3. Enfant `CollisionShape2D` : `RectangleShape2D` de 14×36, position (0, -18).
4. Enfant `ColorRect` nommé `Placeholder` : couleur bleu nuit, taille 14×36, position (-7, -36).

CONCEPT : L'ORIGINE AUX PIEDS

On place l'origine du personnage **sous ses pieds**, d'où la forme décalée de -18 et le rectangle de -36. Avantage : « Batman est à y = 200 » veut dire « ses pieds touchent le sol à y = 200 ». Tous les calculs de hauteur (sol, balcon, rebord) deviennent lisibles.

CONCEPT : LA FORME DE SUBSTITUTION

Le rectangle bleu est un *placeholder*. Il a la taille du futur sprite, et c'est tout ce qui compte pour régler un saut. Les sprites peuvent attendre que la mécanique soit bonne. Beaucoup de jeux célèbres ont passé des mois en rectangles gris.

3.3 · Courir et sauter en trente lignes



Attache un script à `PlayerProto` (`res://sandbox/player_proto.gd`) :

res://sandbox/player_proto.gd

```
extends CharacterBody2D

## PROTOTYPE jetable : tout dans un seul script, pour sentir le personnage.

@export var run_speed := 117.0          ## px/s, mesuré : ~1,95 px/image
@export var jump_velocity := -330.0     ## px/s vers le haut (y négatif = vers le haut)
@export var gravity := 900.0            ## px/s²
@export var max_fall_speed := 330.0
@export_range(0.0, 1.0) var jump_cut := 0.45  ## part de vitesse gardée si on relâche tôt

func _physics_process(delta: float) -> void:
    # 1. Gravité : la vitesse verticale augmente à chaque tick, plafonnée.
    velocity.y = minf(velocity.y + gravity * delta, max_fall_speed)

    # 2. Course : démarrage et arrêt secs, comme dans le jeu original.
    var x := Input.get_axis("move_left", "move_right")
    velocity.x = signf(x) * run_speed

    # 3. Saut, seulement depuis le sol.
    if Input.is_action_just_pressed("jump") and is_on_floor():
        velocity.y = jump_velocity
    # Saut variable : relâcher pendant la montée coupe l'élan.
    if Input.is_action_just_released("jump") and velocity.y < 0.0:
        velocity.y *= jump_cut

    # 4. Le moteur déplace le corps selon velocity et gère les collisions.
    move_and_slide()
```

À FAIRE

1. Ouvre `res://level/test_room.tscn` et glisse `player_proto.tscn` dedans. Place-le en (60, 200) : ses pieds sur le sol.
2. `F6`.

TESTE

- Les flèches (ou Q/D sur AZERTY) le font courir et s'arrêter net.
- `Espace` ou `K` : une tape donne un petit saut, une pression longue un grand saut.
- Sous le balcon, en sautant, sa tête **traverse** le balcon par en dessous, puis il retombe : le saut ne suffit pas pour y monter. C'est voulu, et c'est fidèle au jeu : on y montera en s'accrochant (étape 05).

Comprendre le code

- `velocity` : une propriété de `CharacterBody2D`, en pixels par seconde. On la règle, et `move_and_slide()` l'applique en tenant compte des collisions : il fait glisser le corps le long des murs et du sol, et met à jour `is_on_floor()`. **Pas besoin de `delta` ici** : `move_and_slide()` le gère lui-même.
- La gravité, elle, se multiplie par `delta` : c'est une *accélération* (px/s²), elle s'ajoute à une vitesse.
- `Input.get_axis("move_left", "move_right")` renvoie une valeur entre -1 et 1. Avec un stick analogique, elle peut valoir 0,3 ; `signf()` la ramène à -1, 0 ou 1, car le jeu original n'a pas de marche lente.

- `is_action_just_pressed` n'est vrai qu'**au tick où la touche s'enfonce**. Avec `is_action_pressed`, maintenir la touche ferait sauter en boucle.
- En Godot, l'axe y pointe **vers le bas** : une vitesse négative fait monter.

LA COLLISION À SENS UNIQUE

La case **One Way Collision** de la forme du balcon ne bloque que ce qui arrive *par le dessus*. On peut donc traverser le balcon en sautant depuis la rue, et se poser dessus en arrivant d'en haut. C'est le comportement classique des plateformes de jeux d'arcade.

3.4 · Régler le ressenti

CONCEPT : LA PHYSIQUE ARCADE

On ne cherche pas le réalisme, mais le contrôle. La hauteur d'un saut se calcule : $h = v^2 / (2 \times g)$. Avec $v = 330$ et $g = 900$: $330^2 / 1800 \approx 60$ px. La durée de montée vaut $v / g \approx 0,37$ s. Pour un saut plus haut mais aussi rapide, on augmente à la fois la vitesse et la gravité.

EXERCICES DE RÉGLAGE

1. Sélectionne l'instance `PlayerProto` dans la salle de test et modifie ses valeurs dans l'Inspecteur, sans toucher au script.
2. Mets `jump_velocity` à -450 : $h = 450^2 / 1800 \approx 112$ px. Il atteint maintenant le balcon (surface à $y = 108$, soit 92 px au-dessus des pieds) et se pose dessus. Remets -330.
3. Mets `gravity` à 600 puis à 1 400, en gardant la même hauteur (calcule `jump_velocity` = $-\sqrt{2 \times g \times 60}$). Lequel ressemble le plus à la vidéo ?
4. Mets `jump_cut` à 1 : le saut variable disparaît. À 0,1 : une tape donne un saut minuscule.

MÉTHODE : TOUT CE QUI SE RÈGLE EN `@EXPORT`

Chaque chiffre de ressenti est exposé dans l'Inspecteur. Tu peux le modifier pendant que le jeu tourne : arbre **Distant** → sélectionne le nœud → change la valeur dans l'Inspecteur, et l'effet est immédiat. Régler un saut se fait en jouant, pas en relisant du code.

LE MUR DU PROTOTYPE

3.5 · Ajouter l'accroupissement

Ajoutons une seule capacité : accroupi, Batman ne bouge pas et ne saute pas. Modifie le prototype :

res://sandbox/player_proto.gd (version 2)

```
extends CharacterBody2D
## PROTOTYPE jetable, version 2 : avec l'accroupissement.

@export var run_speed := 117.0
@export var jump_velocity := -330.0
@export var gravity := 900.0
@export var max_fall_speed := 330.0
@export_range(0.0, 1.0) var jump_cut := 0.45

var is_crouching := false

func _physics_process(delta: float) -> void:
    velocity.y = minf(velocity.y + gravity * delta, max_fall_speed)

    is_crouching = is_on_floor() and Input.is_action_pressed("move_down")
    $Placeholder.size.y = 22.0 if is_crouching else 36.0
    $Placeholder.position.y = -22.0 if is_crouching else -36.0

    var x := Input.get_axis("move_left", "move_right")
    if is_crouching:
        velocity.x = 0.0
    else:
        velocity.x = signf(x) * run_speed

    if Input.is_action_just_pressed("jump") and is_on_floor() and not is_crouching:
        velocity.y = jump_velocity
    if Input.is_action_just_released("jump") and velocity.y < 0.0:
        velocity.y *= jump_cut

    move_and_slide()
```

TESTE

Bas : le rectangle s'écrase et Batman s'immobilise ; le saut ne marche plus tant qu'il est accroupi. Ça fonctionne.

3.6 · Compter ce qui reste à faire



Voici ce que Batman doit encore savoir faire, d'après la vidéo : frapper au contact (et s'arrêter pour frapper), tirer en courant, en sautant et accroupi, donner un coup de pied sauté, s'accrocher sous le balcon, grimper, se laisser tomber, encaisser un coup (et perdre le contrôle un instant), clignoter, se faire empoigner, mourir, faire sa pose de victoire.

Avec des booléens, chaque capacité ajoute une variable et des conditions partout :

Ce qu'on veut éviter (ne pas taper)

```
if Input.is_action_just_pressed("jump") and is_on_floor() and not is_crouching \
    and not is_attacking and not is_hurt and not is_grabbed and not is_dead \
    and not is_celebrating:
    if is_hanging:
        if Input.is_action_pressed("move_down"):
            is_hanging = false           # et si on était aussi is_attacking ?
        else:
            position.y = ledge_y         # et on oublie de remettre is_hanging à false...
    else:
        velocity.y = jump_velocity
```

CONCEPT : L'EXPLOSION COMBINATOIRE

Huit booléens donnent $2^8 = 256$ combinaisons, dont presque toutes sont absurdes (« mort ET en train de grimper ET accroupi »). Chaque bug est une combinaison oubliée. Or Batman n'est jamais que dans **une seule** situation à la fois. Une machine à états le garantit par construction : un état actif, des transitions explicites, et chaque état ne contient que le code qui le concerne. C'est l'objet de l'étape suivante.

MÉTHODE : LE PROTOTYPE EST JETABLE

Ce prototype a fait son travail : tu sais ce que doivent valoir la vitesse et le saut, tu as senti le personnage, et tu as vu où le code allait casser. On ne le transforme pas petit à petit : on le garde dans `sandbox/` comme référence, et on réécrit proprement. Réécrire 30 lignes coûte dix minutes ; traîner une mauvaise structure coûte des semaines.

À FAIRE

1. Retire l'instance `PlayerProto` de la salle de test (sélectionne-la, `Suppr`). Le fichier reste dans `sandbox/`.
2. Note dans ton journal les valeurs que tu as préférées.

POINT DE CONTRÔLE

TU AS FINI L'ÉTAPE QUAND

- ☐ Le prototype court à 117 px/s, s'arrête net et fait des sauts de hauteur variable.
- ☐ Tu sais calculer la hauteur d'un saut à partir de `jump_velocity` et `gravity`.
- ☐ Tu peux expliquer pourquoi des booléens ne suffisent pas pour Batman.

```
git git add -A && git commit -m "Étape 03 : prototype de déplacement"
```

CE QUE TU RETIENS

? Pourquoi la gravité est-elle multipliée par `delta`, mais pas `velocity` avant `move_and_slide()` ?

La gravité est une accélération : elle modifie la vitesse d'une quantité proportionnelle au temps écoulé, d'où `gravity * delta`. `velocity` est une vitesse en px/s que `move_and_slide()` convertit lui-même en déplacement avec le `delta` physique.

? Que faut-il changer pour un saut deux fois plus haut ?

$h = v^2 / (2g)$. Pour doubler h à gravité constante, multiplier la vitesse par $\sqrt{2} \approx 1,41$: `jump_velocity` passe de -330 à environ -467.

? Quel est le rôle d'un prototype, et que devient-il ensuite ?

Valider vite une idée ou un ressenti, avec le moins de code possible. Une fois la réponse obtenue, il est mis de côté, et on écrit la version définitive avec une structure adaptée.

POUR ALLER PLUS LOIN

- Ajoute au prototype une petite inertie : `velocity.x = move_toward(velocity.x, signif(x) * run_speed, 900 * delta)`. Compare avec la vidéo : le jeu original n'en a pas.
- Lis la section « [Le graphe d'états](#) » du tome 03 : c'est la carte de ce que tu vas construire.

MACHINE À ÉTATS ET CAMÉRA

On réécrit Batman proprement : un script principal qui offre des « verbes » (courir, sauter...), et un petit script par état qui décide quoi en faire. Puis une caméra qui suit Batman comme dans le jeu original : en douceur, en anticipant, sans jamais reculer.

~3 h

Prérequis : étape 03

Guide : [tome 03](#)

CE QUE TU VAS APPRENDRE

GODOT 4

- `class_name` sur un personnage, `@export_group`
- Les groupes de nœuds
- `AnimatedSprite2D` et `SpriteFrames`
- `Camera2D`, `make_current()`
- Modifier les couches de collision par le code

DÉVELOPPEMENT DE JEU

- La machine à états appliquée à un personnage
- Séparer les données, les verbes et les décisions
- Coyote time et mémoire de saut
- L'affichage de débogage
- La caméra lissée avec anticipation

LE PLAN

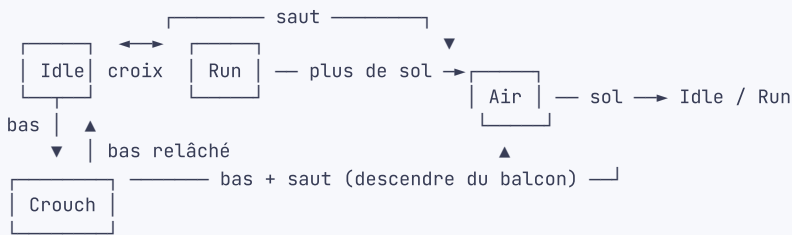
4.1 · Dessiner le graphe d'états



Chaque état de la machine, tel qu'il apparaît dans le jeu. Idle, Run, Air, Crouch, Hang, attaque aérienne, coup de pied sauté, Grabbed, Victory.

MÉTHODE : LE SCHÉMA AVANT LE CODE

Avant de coder une machine à états, on la dessine : des boîtes (les états) et des flèches (les transitions, avec leur condition). Sur papier, une flèche manquante se voit tout de suite ; dans le code, elle devient un bug « le personnage reste bloqué ». Recopie ce schéma dans ton journal.



Cette étape construit ces quatre états. L'étape 05 ajoutera Melee, Hang, Hurt, Grabbed, Dead et Victory.

CONCEPT : DONNÉES, VERBES, DÉCISIONS

On sépare trois choses. Les **données** (vitesse, gravité...) et les **verbes** (`run()`, `jump()`, `apply_gravity()`) vivent dans `player.gd`. Les **décisions** (« si le joueur appuie sur saut et que je suis au sol, alors je saute et je passe en l'air ») vivent dans les états. Un état ne réimplémente jamais un saut : il appelle `p.jump()`. Si tu modifies le saut, tu le fais à un seul endroit.

LA SCÈNE

4.2 · Créer `player.tscn`



À FAIRE

- Nouvelle scène, racine `CharacterBody2D` renommée `Player`. Couche 3 seule, masque 1 et 2. Enregistre en `res://player/player.tscn`.
- Construis l'arbre ci-dessous. Pour `Sprite` : un `AnimatedSprite2D`, offset y = -24, laissé sans `SpriteFrames` pour l'instant. Le rectangle `Placeholder` est un **enfant** du `Sprite` : quand le sprite sera masqué (clignotement d'invincibilité, étape 05), le rectangle le sera aussi.
- Sous `StateMachine`, crée quatre nœuds `Node` nommés exactement `Idle`, `Run`, `Crouch`, `Air`. Les noms comptent : ils servent d'identifiants.
- Sélectionne `StateMachine` : dans l'Inspecteur, **Initial State** = `Idle`.
- Sélectionne `Player` → onglet **Nœud** → **Groupes** → ajoute le groupe `player`.

```
Player (CharacterBody2D)      couche 3 · masque 1+2 · groupe « player »
├─ Sprite (AnimatedSprite2D)  offset y = -24 (vide pour l'instant)
│   └─ Placeholder (ColorRect) 14×36 · position (-7, -36) · bleu nuit
├─ Body (CollisionShape2D)    rectangle 14×36, position (0, -18)
├─ StateDebug (Label)         position (-30, -52) · taille 60×10 · voir 4.6
└─ StateMachine (StateMachine) initial_state = Idle
    ├─ Idle (Node)
    ├─ Run (Node)
    ├─ Crouch (Node)
    └─ Air (Node)
```

LES GROUPES

Un groupe est une étiquette posée sur des nœuds. N'importe quel script peut ensuite demander « donne-moi le nœud du groupe `player` » : `get_tree().get_first_node_in_group("player")`. Les ennemis trouveront Batman ainsi, sans connaître sa place dans l'arbre.

LE CODE

4.3 · `player.gd` , version 1



Attache ce script à `Player`. C'est une version réduite au déplacement ; l'étape 05 la complètera jusqu'à la version de référence du guide.

res://player/player.gd (version 1 – étape 04)

```
class_name Player
extends CharacterBody2D

## Batman ou Robin. Les décisions sont prises par les états (StateMachine) ;
## ce script porte les réglages et les actions communes.

@export_group("Déplacement")
@export var run_speed := 117.0          ## px/s, mesuré : ~1,95 px/image
@export var jump_velocity := -330.0     ## px/s, À CALIBRER (sommet ≈ 60 px avec gravity = 900)
@export var gravity := 900.0
@export var max_fall_speed := 330.0
@export_range(0.0, 1.0) var jump_cut := 0.45 ## saut variable : part de vitesse gardée au
relâchement, À CALIBRER
@export var coyote_time := 0.06         ## tolérance moderne ; 0 = comportement strict
@export var jump_buffer := 0.10

var facing := 1
var _coyote := 0.0
var _buffer := 0.0
var _drop_left := 0.0

@onready var sprite: AnimatedSprite2D = $Sprite
@onready var fsm: StateMachine = $StateMachine

func _physics_process(delta: float) -> void:
    _coyote = coyote_time if is_on_floor() else maxf(_coyote - delta, 0.0)
    if Input.is_action_just_pressed("jump"):
        _buffer = jump_buffer
    else:
        _buffer = maxf(_buffer - delta, 0.0)
    if _drop_left > 0.0:
        _drop_left -= delta
        if _drop_left <= 0.0:
            set_collision_mask_value(2, true)

    fsm.physics_update(delta) # l'état courant règle velocity
    move_and_slide()

# --- Déplacement -----
```

```

func apply_gravity(delta: float) -> void:
    velocity.y = minf(velocity.y + gravity * delta, max_fall_speed)

func read_move_x() -> float:
    return Input.get_axis("move_left", "move_right")

func run(dir_x: float) -> void:
    # Démarrage et arrêt secs : pas d'accélération, comme dans le jeu original.
    velocity.x = signf(dir_x) * run_speed
    if dir_x != 0.0:
        set_facing(int(signf(dir_x)))

func set_facing(dir: int) -> void:
    if dir == 0 or dir == facing:
        return
    facing = dir
    sprite.flip_h = facing < 0

func wants_jump() -> bool:
    return _buffer > 0.0

func can_jump() -> bool:
    return is_on_floor() or _coyote > 0.0

func consume_jump() -> void:
    _buffer = 0.0
    _coyote = 0.0

func jump() -> void:
    velocity.y = jump_velocity
    consume_jump()

func drop_through_platform() -> void:
    ## Bas + saut sur le balcon : on ignore la couche « plateformes » un instant.
    set_collision_mask_value(2, false)
    position.y += 2.0
    _drop_left = 0.25

# --- Animation -----

## Joue l'animation demandée si elle existe ; sinon garde l'animation en cours.
func play_anim(base: StringName) -> void:
    if sprite.sprite_frames == null:
        return
    if sprite.sprite_frames.has_animation(base) and sprite.animation != base:
        sprite.play(base)

```

Comprendre le code

- `class_name Player` : les états pourront écrire `var p: Player = owner` et profiter de l'autocomplétion sur tous les verbes.
- `@export_group("Déplacement")` : range les exports sous un titre repliable dans l'Inspecteur.
- `_physics_process` fait trois choses, toujours dans cet ordre : mettre à jour les minuteries, laisser l'état courant décider de `velocity`, puis déplacer avec `move_and_slide()`. Aucun état n'appelle `move_and_slide()` : il n'y a qu'un seul endroit où Batman bouge.
- **Coyote time** (`_coyote`) : pendant 0,06 s après avoir quitté un bord, le saut est encore accepté. **Mémoire de saut** (`_buffer`) : un appui 0,1 s avant l'atterrissage est gardé en mémoire et exécuté au contact du sol. Ce sont deux comforts modernes ; mets-les à 0 pour un ressenti strict de 1995.
- `set_collision_mask_value(2, false)` : retire la couche 2 du masque, donc Batman traverse le balcon. Un compteur la remet après 0,25 s. On décale aussi Batman de 2 px vers le bas, pour qu'il ne soit plus posé sur la plateforme quand le masque revient.
- `play_anim()` ne plante pas s'il n'y a pas encore d'animations : on peut avancer avec le rectangle bleu.

4.4 · Les états `Idle` et `Run`



Crée `res://player/states/`, puis ces deux scripts. Attache chacun au nœud du même nom sous `StateMachine`.

`res://player/states/idle.gd` (version 1)

```
extends State

@onready var p: Player = owner

func enter(_previous: StringName, _data: Dictionary) -> void:
    p.velocity.x = 0.0

func physics_update(delta: float) -> void:
    p.apply_gravity(delta)
    if not p.is_on_floor():
        transition_to(&"Air")
        return
    if Input.is_action_pressed("move_down"):
        transition_to(&"Crouch")
        return
    if p.wants_jump() and p.can_jump():
        p.jump()
        transition_to(&"Air")
        return
    if p.read_move_x() != 0.0:
        transition_to(&"Run")
        return
    p.velocity.x = 0.0
    p.play_anim(&"idle")
```

```
extends State
```

```
@onready var p: Player = owner
```

```
func physics_update(delta: float) -> void:
    p.apply_gravity(delta)
    if not p.is_on_floor():
        transition_to(&"Air")    # le temps « coyote » permet encore de sauter
        return
    var x := p.read_move_x()
    if x == 0.0:
        transition_to(&"Idle")
        return
    if p.wants_jump() and p.can_jump():
        p.jump()
        transition_to(&"Air")
        return
    p.run(x)
    p.play_anim(&"run")
```

Comprendre le code

- **Le motif « vérifier puis sortir »** : chaque condition de sortie est testée dans l'ordre de priorité ; dès qu'une transition est demandée, `return` . Ce qui reste en bas de la fonction, c'est le comportement « normal » de l'état.
- L'ordre est une décision de game design : dans `Idle` , « tomber » passe avant « s'accroupir », qui passe avant « sauter ».
- Chaque fichier fait moins de 30 lignes et ne parle que de sa situation. Compare avec le prototype version 2.

```
res://player/states/crouch.gd (version 1)
```

```
extends State

## Accroupi : esquive les tirs hauts et tire au ras du sol.
## Bas + saut sur une plateforme traversable = descendre (hypothèse de confort).

@onready var p: Player = owner

func enter(_previous: StringName, _data: Dictionary) -> void:
    p.velocity.x = 0.0

func physics_update(delta: float) -> void:
    p.apply_gravity(delta)
    p.velocity.x = 0.0
    if not p.is_on_floor():
        transition_to(&"Air")
        return
    if p.wants_jump():
        p.consume_jump()
        p.drop_through_platform()
        transition_to(&"Air")
        return
    if not Input.is_action_pressed("move_down"):
        transition_to(&"Idle")
        return
    var dir := p.read_move_x()
    if dir != 0.0:
        p.set_facing(int(signf(dir)))
    p.play_anim(&"crouch")
```

res://player/states/air.gd (version 1)

extends State

Saut et chute. Contrôle aérien total, saut à hauteur variable.

@onready var p: Player = owner

func physics_update(delta: float) -> void:

1. Saut variable : relâcher le bouton pendant la montée coupe l'élan.

if Input.is_action_just_released("jump") **and** p.velocity.y < 0.0:

 p.velocity.y *= p.jump_cut

p.apply_gravity(delta)

2. Saut retardé (coyote) : on vient de quitter le bord.

if p.wants_jump() **and** p.can_jump():

 p.jump()

3. Contrôle aérien complet, typique des run & gun.

p.run(p.read_move_x())

4. Atterrissage.

if p.is_on_floor() **and** p.velocity.y >= 0.0:

 transition_to(&"Run" **if** p.read_move_x() != 0.0 **else** &"Idle")

return

p.play_anim(&"jump" **if** p.velocity.y < 0.0 **else** &"fall")

POURQUOI L'ATTERRISSAGE TESTE `VELOCITY.Y >= 0`

Au tick du saut, Batman est encore au sol (`is_on_floor()` vaut vrai jusqu'au prochain `move_and_slide()`). Sans cette condition, l'état `Air` déciderait aussitôt qu'il a atterri. On exige donc qu'il soit en train de descendre.

4.6 · Voir l'état en direct



MÉTHODE : RENDRE L'INVISIBLE VISIBLE

Une machine à états est invisible : quand Batman se comporte mal, il faut savoir *dans quel état* il est. Les développeurs ajoutent des affichages de débogage partout : état courant, vitesse, zones de collision. On les retire (ou on les cache) à la fin.

À FAIRE

1. Sur le `Label` `StateDebug` : **Horizontal Alignment** = Center, **Theme Overrides** → **Font Sizes** → **Font Size** = 8.
2. Attache-lui ce script, puis, dans l'Inspecteur, désigne `Player` dans le champ **Player** :

```
res://player/state_debug.gd
```

```
extends Label
```

```
## Outil de débogage : affiche l'état courant et le sens du regard au-dessus du personnage.
```

```
@export var player: Player
```

```
func _process(_delta: float) -> void:
```

```
    text = "%s %s" % [player.fsm.current_name(), "→" if player.facing > 0 else "←"]
```

À FAIRE (SUITE)

3. Ouvre `test_room.tscn`, glisse `player.tscn` dedans, en (200, 100) : **sur le balcon**.

4. `F6`.

TESTE

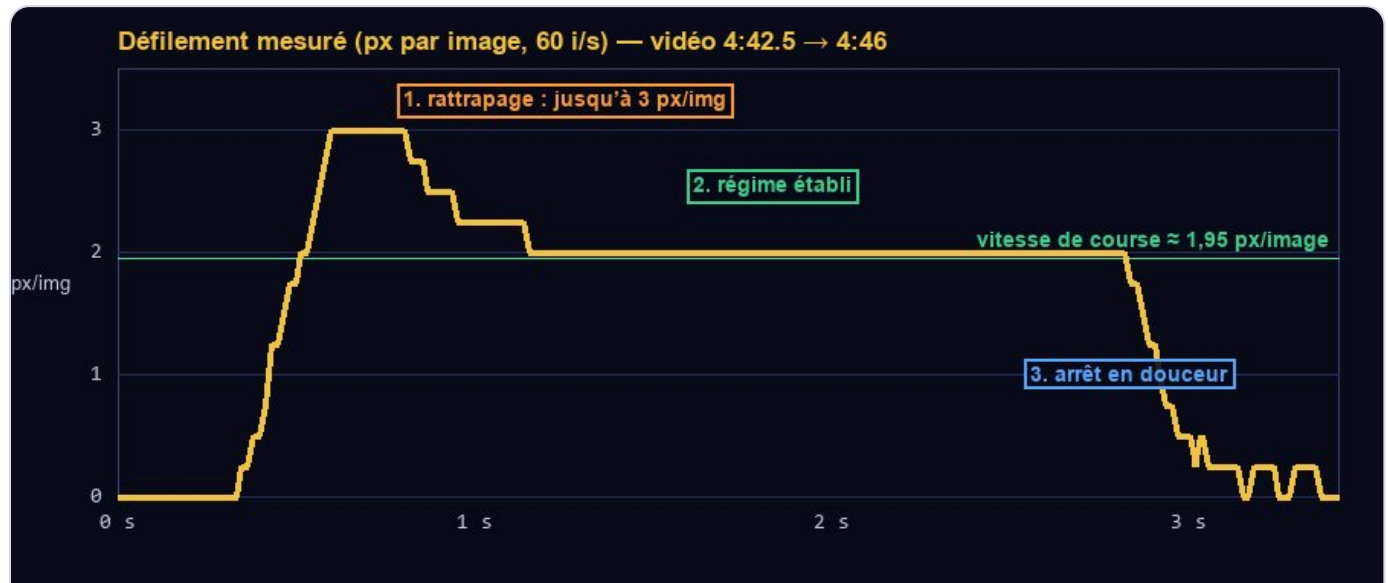
- Au départ : « Idle → », sur le balcon.
- Cours : « Run ». Tombe du bord du balcon : « Air », puis « Idle » au sol.
- Juste après avoir quitté le bord, un saut reste possible (coyote time).
- Remonte sur le balcon en modifiant temporairement `jump_velocity` à -450 dans l'Inspecteur. Bas : « Crouch ». Bas + saut : Batman traverse le balcon et retombe dans la rue. Remets -330.
- Bas sur le sol de la rue + saut : rien ne se passe, la couche « monde » ne se traverse pas.

SI ÇA NE MARCHE PAS

- « **Invalid get index 'apply_gravity' (on base: 'null instance')** » : un état n'arrive pas à récupérer Batman. Le script d'état est-il attaché à un enfant de `StateMachine` dans `player.tscn`, et non dans la salle de test ? `owner` désigne la racine de la scène où le nœud a été créé.
- « **État inconnu : Air** » : le nœud ne s'appelle pas exactement `Air` (majuscule, espace en trop).
- **Batman tombe à travers le sol** : son masque ne contient pas la couche 1, ou le sol n'est pas sur la couche 1.
- **Batman ne bouge pas du tout** : `Initial State` est vide et le premier enfant n'est pas un état, ou le script `player.gd` n'est pas attaché.

LA CAMÉRA

4.7 · Observer la caméra originale



La vitesse de la caméra, mesurée image par image dans la vidéo. Elle dépasse d'abord la vitesse de course pour rattraper Batman (1), se stabilise (2), puis ralentit en douceur à l'arrêt (3).

DANS LA VIDÉO

Regarde ▶ 4:42 au ralenti : quand Batman se met à courir, l'écran accélère progressivement ; quand il s'arrête, l'écran continue un peu et freine en douceur. Batman n'est pas au centre de l'écran : il y a plus de place devant lui que derrière. Et l'écran ne revient jamais en arrière.

CONCEPT : LA CAMÉRA LISSÉE AVEC ANTICIPATION

Une caméra collée au joueur donne mal au cœur et cache ce qui arrive. La caméra vise un point **devant** le joueur (*look-ahead*) et s'en rapproche à chaque image d'une fraction de la distance restante (*lissage exponentiel*). Elle rattrape vite quand elle est loin, et ralentit en approchant.

4.8 · camera_rig.gd

Crée `res://level/camera_rig.gd` (hérite de `Camera2D`) :

`res://level/camera_rig.gd`

```
class_name CameraRig
extends Camera2D

## Caméra de run & gun : suit le joueur avec anticipation, ne revient jamais en arrière,
## peut être verrouillée sur une arène ou défiler toute seule (convoi).

@export var target: Player
@export var look_ahead := 40.0      ## px d'avance dans la direction du regard
@export var follow_rate := 5.0      ## plus grand = rattrapage plus rapide
@export var forward_only := true    ## l'écran ne recule pas
@export var fixed_y := 112.0        ## les sections du niveau 1 ne défilent pas verticalement
```

```

## px/s : défilement forcé de l'écran. Non utilisé dans le niveau 1 (au convoi, c'est la rue
## qui défile derrière un convoi immobile, tome 09), mais prêt pour un niveau à défilement imposé.
var autoscroll_speed := 0.0
var _locked := false
var _lock_center_x := 0.0

func _ready() -> void:
    make_current()
    if target:
        target.camera = self
        global_position = Vector2(target.global_position.x, fixed_y)

func _physics_process(delta: float) -> void:
    if target == null:
        return
    var x := global_position.x
    if _locked:
        x = move_toward(x, _lock_center_x, 240.0 * delta)
    else:
        var desired := target.global_position.x + look_ahead * target.facing
        # Lissage exponentiel indépendant du nombre d'images par seconde.
        var t := 1.0 - exp(-follow_rate * delta)
        var next := lerp(x, desired, t) + autoscroll_speed * delta
        x = maxf(next, x) if forward_only else next
    global_position = Vector2(x, fixed_y)

## Fige l'écran sur une arène (les vagues d'ennemis du tome 05).
func lock_on(center_x: float) -> void:
    _locked = true
    _lock_center_x = center_x

func unlock() -> void:
    _locked = false

func view_rect() -> Rect2:
    var size := get_viewport_rect().size / zoom
    return Rect2(global_position - size / 2.0, size)

func clamp_to_view(body: Node2D, margin := 10.0) -> void:
    var r := view_rect()
    body.global_position.x = clampf(body.global_position.x, r.position.x + margin, r.end.x - margin)

```

Ce script écrit `target.camera` : il faut ajouter cette variable à `player.gd`, et faire en sorte que Batman reste dans l'écran. Deux ajouts :

```
res://player/player.gd (ajouts)
```

```
# Sous « var facing := 1 » :  
var camera: CameraRig  
  
# À la fin de _physics_process(), après move_and_slide() :  
if camera:  
    camera.clamp_to_view(self)
```

SI L'ÉDITEUR AFFICHE UNE ERREUR ROUGE

`Player` parle de `CameraRig` et `CameraRig` parle de `Player`. Tant que l'un des deux fichiers n'existe pas ou n'est pas enregistré, l'autre affiche « Could not find type ». Enregistre les deux (`Ctrl Maj S` / `⌘ Maj S` : tout enregistrer) : l'erreur disparaît. Les références croisées entre classes sont permises dans Godot 4.

Comprendre le code

- `1.0 - exp(-follow_rate * delta)` : la fraction de distance parcourue à chaque tick. `lerp(x, cible, 0.1)` à chaque image irait deux fois plus vite à 120 Hz qu'à 60 Hz ; cette forme donne la même courbe quelle que soit la fréquence. Garde ce réflexe chaque fois que tu « rattrapes » une valeur.
- `maxf(next, x)` : on garde la plus grande des deux positions, donc la caméra n'avance que vers la droite.
- `lock_on()` et `unlock()` serviront aux arènes de l'étape 07 : l'écran se fige pendant le combat.
- `clamp_to_view()` : la caméra ne reculant pas, Batman ne doit pas pouvoir sortir par la gauche.
- `fixed_y` : les trois sections du niveau 1 ne défilent jamais verticalement. Une seule ligne suffit à le garantir.

4.9 · Une salle plus longue pour tester



À FAIRE

- Dans `test_room.tscn`, agrandis le sol : `Floor` en (1000, 208), rectangle de collision 2000×16, `ColorRect` 2000×16 en (-1000, -8).
- Ajoute un `Node2D` nommé `Markers` avec quelques `ColorRect` verticaux (4×60) tous les 160 px, posés sur le sol : ce sont tes « réverbères » pour voir défiler l'écran.
- Sélectionne la `Camera2D`, attache-lui `camera_rig.gd`, et mets **Target** = `Player`.
- Remplace `Player` au sol, en (60, 200).
- `F6`.

TESTE

- En courant à droite, l'écran accélère en douceur et Batman se retrouve un peu à gauche du centre.
- À l'arrêt, l'écran glisse encore un peu puis s'arrête.
- En courant à gauche, l'écran ne recule pas, et Batman bute contre le bord gauche.
- Expériences : `follow_rate` à 20 (caméra collée), à 1 (caméra molle) ; `look_ahead` à 0 ; `forward_only` décoché.

Grâce au *snap au pixel* réglé à l'étape 01, la position à virgule de la caméra ne fait pas trembler les pixels.

POINT DE CONTRÔLE

TU AS FINI L'ÉTAPE QUAND

- ☐ Batman court à ~117 px/s, s'arrête net, saute plus ou moins haut selon la durée d'appui.
- ☐ Il s'accroupit, et descend du balcon avec bas + saut.
- ☐ L'étiquette de débogage affiche le bon état à tout moment.
- ☐ La caméra anticipe dans le sens de la course, ne recule pas, et Batman ne peut pas sortir par la gauche.

```
git add -A && git commit -m "Étape 04 : Batman en machine à états, caméra"
```

CE QUE TU RETIENS

? Pourquoi aucun état n'appelle-t-il `move_and_slide()` ?

Pour qu'il n'y ait qu'un seul endroit où Batman se déplace : `player.gd`, après que l'état a décidé de `velocity`. Si deux états l'appelaient, on risquerait de déplacer deux fois par tick lors d'une transition.

? Que fait le `coyote time`, et pourquoi existe-t-il ?

Il accepte un saut quelques centièmes de seconde après que le personnage a quitté le bord. Le joueur a l'impression d'avoir sauté à temps ; sans lui, le jeu paraît injuste, car l'œil humain ne perçoit pas une image de retard.

? Pourquoi `1 - exp(-k * delta)` plutôt qu'une constante comme 0,1 ?

Une constante appliquée à chaque image dépend du nombre d'images par seconde : la caméra irait plus vite sur un écran 144 Hz. La forme exponentielle donne la même trajectoire quel que soit `delta`.

? Tu ajoutes un état `Slide`. Quels fichiers modifies-tu ?

Tu crées `slide.gd` et un nœud `Slide` sous `StateMachine`, puis tu ajoutes la transition dans l'état d'où on y entre (par exemple `Run`). Aucun autre état ne change. C'est tout l'intérêt de la machine à états.

POUR ALLER PLUS LOIN

- Affiche aussi `velocity` dans l'étiquette de débogage, arrondie à l'entier.
- Mets `coyote_time` et `jump_buffer` à 0, joue deux minutes, puis remets-les. Note la différence dans ton journal.
- Lis [la section caméra du tome 03](#).

COMBAT, ACCROCHE ET DÉGÂTS

Batman apprend à frapper, à s'accrocher sous le balcon, à encaisser, à mourir et à fêter sa victoire. On branche les composants de l'étape 02 sur lui, on ajoute six états, et on construit une cible d'entraînement pour tout vérifier sans attendre les ennemis.

~3 h

Prérequis : étape 04

Guide : [tome 03](#)

CE QUE TU VAS APPRENDRE

GODOT 4

- `Resource` personnalisée et fichiers `.tres`
- `get_overlapping_areas()`, `has_overlapping_areas()`
- Scripts qui héritent d'un script (`super`)
- L'éditeur `SpriteFrames` et les `AtlasTexture`

DÉVELOPPEMENT DE JEU

- État contre capacité
- L'attaque contextuelle
- Préparation, images actives, récupération
- Recul et invincibilité temporaire
- Compteurs plutôt que minuteries asynchrones

PRÉPARER BATMAN

5.1 · Ce qui est un état, ce qui n'en est pas



CONCEPT : ÉTAT CONTRE CAPACITÉ

Batman tire en courant, en sautant, accroupi ou suspendu. Si « tirer » était un état, il en faudrait « courir-en-tirant », « sauter-en-tirant »... Le tir est donc une **capacité** : une fonction (`try_attack()`) que les états autorisés appellent. En revanche, la **mêlée au sol** est un état : Batman s'arrête pour frapper, il ne peut rien faire d'autre pendant ce temps. La question à se poser : « pendant cette action, le personnage peut-il faire autre chose ? »

CONCEPT : L'ATTAQUE CONTEXTUELLE

D'après le manuel du jeu, un seul bouton fait deux choses : un coup au contact si un ennemi est tout près, un projectile sinon. Il faut donc une zone qui répond à la question « un ennemi est-il à portée de poing ? » : `MeleeSense`.



MeleeSense touche une Hurtbox : Melee



Sinon : WeaponSystem.try_fire()

Un seul bouton, deux attaques. Si `MeleeSense` détecte une Hurtbox ennemie, c'est un coup ; sinon, un projectile.

5.2 · Ajouter les nœuds



À FAIRE

1. Ouvre `player.tscn` et ajoute les nœuds marqués ★ :

```
Player (CharacterBody2D)
├─ Sprite (AnimatedSprite2D)    player.gd · couche 3 · masque 1+2 · groupe « player »
│   └─ Placeholder (ColorRect)  offset y = -24
│       └─ 14×36 · position (-7, -36)
├─ Body (CollisionShape2D)      rectangle 14×36, position (0, -18)
├─ StateDebug (Label)
├─ ★ Health (Health)            max_health = 8 (À CALIBRER : segments de l'anneau)
├─ ★ Hurtbox (Hurtbox)          couche 5 · masque aucun · health = Health
│   └─ CollisionShape2D         rectangle 12×32, position (0, -18)
├─ ★ MeleeHitbox (Hitbox)       couche aucune · masque 6 · damage = 2 · knockback (140, -60)
│   └─ CollisionShape2D         rectangle 22×20, position (16, -22)
├─ ★ MeleeSense (Area2D)        couche aucune · masque 6
│   └─ CollisionShape2D         rectangle 26×34, position (16, -18)
├─ ★ HandProbe (Area2D)         couche aucune · masque 8 « rebords »
│   └─ CollisionShape2D         rectangle 10×4, position (0, -42) : au-dessus de la tête
├─ StateMachine (StateMachine) initial_state = Idle
│   └─ Idle ── Run ── Crouch ── Air
│       └─ ★ Melee ── ★ Hang ── ★ Hurt ── ★ Grabbed
│           └─ ★ Dead ── ★ Victory
```

2. Les six nouveaux états sont de simples `Node` pour l'instant ; leurs scripts arrivent plus bas.
3. Astuce : pour une forme de collision, clique sur **Shape** → **Nouveau RectangleShape2D**, puis clique sur la forme créée pour régler sa **Size**.

PIÈGE : LES FORMES PARTAGÉES

Si tu dupliques un `CollisionShape2D` (`Ctrl D`), la copie **partage** la même ressource de forme : redimensionner l'une redimensionne l'autre. Pour les séparer : clic droit sur la forme dans l'Inspecteur → **Rendre unique**.

5.3 · Une fiche par héros : HeroData



LES RESSOURCES PERSONNALISÉES

Une `Resource` est un objet de données qu'on enregistre dans un fichier `.tres` et qu'on édite dans l'Inspecteur. En écrivant une classe qui hérite de `Resource`, on crée son propre type de fiche. Ici, une fiche par héros : nom, animations, portrait, vitesse.

```
res://player/hero_data.gd
```

```
class_name HeroData
extends Resource
## Différences entre Batman et Robin. Crée batman.tres et robin.tres avec l'inspecteur.

@export var display_name := "Batman"
@export var frames: SpriteFrames
@export var portrait: Texture2D
@export var run_speed := 117.0
@export var jump_velocity := -330.0
```

À FAIRE

1. Clic droit sur `res://player/` → **Nouveau → Ressource...** → cherche `HeroData` → nom `batman.tres`.
2. Double-clique dessus : l'Inspecteur affiche ses champs. Laisse `frames` et `portrait` vides pour l'instant.
3. Recommence pour `robin.tres`, avec `display_name` = « Robin ».

BATMAN OU ROBIN

La vidéo ne montre que Batman. Les deux fiches gardent donc les mêmes valeurs ; seules les animations changeront. Si tu mesures une différence dans un émulateur, il suffira de modifier `robin.tres`.

LE SCRIPT COMPLET

5.4 · `player.gd`, version de référence



Remplace tout le contenu de `player.gd`. Avant de taper, repère les nouveautés par rapport à la version 1 :

- le chargement de la fiche du héros dans `_ready()` ;
- la section **Combat** : visée, attaque contextuelle, mêlée, invincibilité, dégâts, mort, réapparition ;
- `find_ledge()` pour l'accroche ;
- `play_anim()` qui sait choisir une variante « en train de tirer ».

```
res://player/player.gd
```

```
class_name Player
extends CharacterBody2D
## Batman ou Robin. Les décisions sont prises par les états (StateMachine) ;
```

```

## ce script porte les réglages et les actions communes.

signal died ## émis quand l'animation de mort est terminée

@export var hero: HeroData

@export_group("Déplacement")
@export var run_speed := 117.0 ## px/s, mesuré : ~1,95 px/image
@export var jump_velocity := -330.0 ## px/s, À CALIBRER (sommet ≈ 60 px avec gravity = 900)
@export var gravity := 900.0
@export var max_fall_speed := 330.0
@export_range(0.0, 1.0) var jump_cut := 0.45 ## saut variable : part de vitesse gardée au relâchement, À CALIBRER
@export var coyote_time := 0.06 ## tolérance moderne ; 0 = comportement strict
@export var jump_buffer := 0.10

@export_group("Combat")
@export var invincibility_time := 1.5
@export var hurt_time := 0.35

## Fiches des héros, dans l'ordre de l'enum Game.Hero (BATMAN, ROBIN).
const HERO_FILES := ["res://player/batman.tres", "res://player/robin.tres"]

## Position du canon selon la visée (droite = +x). Clé : (|x| > 0, signe de y).
const MUZZLES := {
    Vector2i(1, 0): Vector2(14, -24),
    Vector2i(1, -1): Vector2(12, -36),
    Vector2i(0, -1): Vector2(2, -44),
    Vector2i(1, 1): Vector2(12, -12),
    Vector2i(0, 1): Vector2(0, -6),
}

var facing := 1
var aim := Vector2.RIGHT
var camera: CameraRig
var _coyote := 0.0
var _buffer := 0.0
var _invincible_left := 0.0
var _shoot_pose := 0.0
var _drop_left := 0.0

@onready var sprite: AnimatedSprite2D = $Sprite
@onready var health: Health = $Health
@onready var hurtbox: Hurtbox = $Hurtbox
@onready var melee_hitbox: Hitbox = $MeleeHitbox
@onready var melee_sense: Area2D = $MeleeSense
@onready var hand_probe: Area2D = $HandProbe
@onready var fsm: StateMachine = $StateMachine
## Non typé : le WeaponSystem n'arrive qu'au tome 04. Sans lui, Batman ne tire pas.
@onready var weapons: Node = get_node_or_null("WeaponSystem")

func _ready() -> void:
    if hero == null:
        hero = load(HERO_FILES[Game.hero]) as HeroData # choix de l'écran PLAYER SELECT
    if hero:
        sprite.sprite_frames = hero.frames
        run_speed = hero.run_speed

```

```

        jump_velocity = hero.jump_velocity
        melee_hitbox.monitoring = false
        hurtbox.hurt.connect(_on_hurt)
        health.died.connect(_on_died)

func _physics_process(delta: float) -> void:
    _coyote = coyote_time if is_on_floor() else maxf(_coyote - delta, 0.0)
    if Input.is_action_just_pressed("jump"):
        _buffer = jump_buffer
    else:
        _buffer = maxf(_buffer - delta, 0.0)
    _shoot_pose = maxf(_shoot_pose - delta, 0.0)
    _tick_invincibility(delta)
    if _drop_left > 0.0:
        _drop_left -= delta
        if _drop_left <= 0.0:
            set_collision_mask_value(2, true)

    fsm.physics_update(delta) # l'état courant règle velocity
    move_and_slide()
    if camera:
        camera.clamp_to_view(self)

# --- Déplacement -----

func apply_gravity(delta: float) -> void:
    velocity.y = minf(velocity.y + gravity * delta, max_fall_speed)

func read_move_x() -> float:
    return Input.get_axis("move_left", "move_right")

func run(dir_x: float) -> void:
    # Démarrage et arrêt secs : pas d'accélération, comme dans le jeu original.
    velocity.x = signf(dir_x) * run_speed
    if dir_x != 0.0:
        set_facing(int(signf(dir_x)))

func set_facing(dir: int) -> void:
    if dir == 0 or dir == facing:
        return
    facing = dir
    sprite.flip_h = facing < 0
    melee_hitbox.scale.x = facing
    melee_sense.scale.x = facing
    melee_hitbox.facing = facing

func wants_jump() -> bool:
    return _buffer > 0.0

func can_jump() -> bool:
    return is_on_floor() or _coyote > 0.0

```

```

func consume_jump() -> void:
    _buffer = 0.0
    _coyote = 0.0

func jump() -> void:
    velocity.y = jump_velocity
    consume_jump()

func drop_through_platform() -> void:
    ## Bas + saut sur le balcon : on ignore la couche « plateformes » un instant.
    set_collision_mask_value(2, false)
    position.y += 2.0
    _drop_left = 0.25

func find_ledge() -> float:
    ## Renvoie la hauteur du rebord touché par les mains, ou NAN.
    for area in hand_probe.get_overlapping_areas():
        return area.global_position.y
    return NAN

# --- Combat -----

func compute_aim(crouching := false) -> Vector2:
    var x := signf(read_move_x())
    var y := signf(Input.get_axis("move_up", "move_down"))
    if crouching or (is_on_floor() and y > 0.0):
        y = 0.0 # au sol, « bas » veut dire s'accroupir, pas viser ses pieds
    if x == 0.0 and y == 0.0:
        x = facing
    aim = Vector2(x, y).normalized()
    return aim

func enemy_in_reach() -> bool:
    return melee_sense.has_overlapping_areas()

## Bouton d'attaque. Renvoie &"melee" si l'état doit jouer un coup au contact.
func try_attack(crouching := false, allow_up := true) -> StringName:
    if not Input.is_action_pressed("attack"):
        return &""
    compute_aim(crouching)
    if not allow_up and aim.y < 0.0:
        aim = Vector2(facing, 0.0)
    if Input.is_action_just_pressed("attack") and enemy_in_reach() and aim.y == 0.0:
        return &"melee"
    if weapons and weapons.try_fire(muzzle_position(crouching), aim):
        _shoot_pose = 0.15
    return &""

func muzzle_position(crouching := false) -> Vector2:

```

```

var key := Vector2i(1 if absf(aim.x) > 0.1 else 0, int(signf(aim.y)))
var offset: Vector2 = MUZZLES.get(key, MUZZLES[Vector2i(1, 0)])
if crouching:
    offset.y += 12.0
offset.x *= facing
return global_position + offset

func melee_on() -> void:
    melee_hitbox.reset_hits()
    melee_hitbox.set_deferred("monitoring", true)

func melee_off() -> void:
    melee_hitbox.set_deferred("monitoring", false)

func start_invincibility(duration: float) -> void:
    _invincible_left = duration
    hurtbox.invincible = true

func _tick_invincibility(delta: float) -> void:
    if _invincible_left <= 0.0:
        return
    _invincible_left -= delta
    # Clignotement « une image sur deux », typique de la Mega Drive.
    sprite.visible = int(_invincible_left * 30.0) % 2 == 0
    if _invincible_left <= 0.0:
        hurtbox.invincible = false
        sprite.visible = true

func _on_hurt(hitbox: Hitbox) -> void:
    if health.is_dead():
        return
    velocity = hitbox.get_knockback()
    start_invincibility(invincibility_time)
    fsm.force(&"Hurt")

func _on_died() -> void:
    fsm.force(&"Dead")

func respawn() -> void:
    ## Réapparition : Batman retombe du haut de l'écran, comme dans les run & gun d'arcade.
    health.reset()
    sprite.visible = true
    if camera:
        var view := camera.view_rect()
        global_position = Vector2(view.position.x + view.size.x * 0.35, view.position.y + 24.0)
    velocity = Vector2.ZERO
    start_invincibility(2.5)
    fsm.force(&"Air")

func celebrate() -> void:

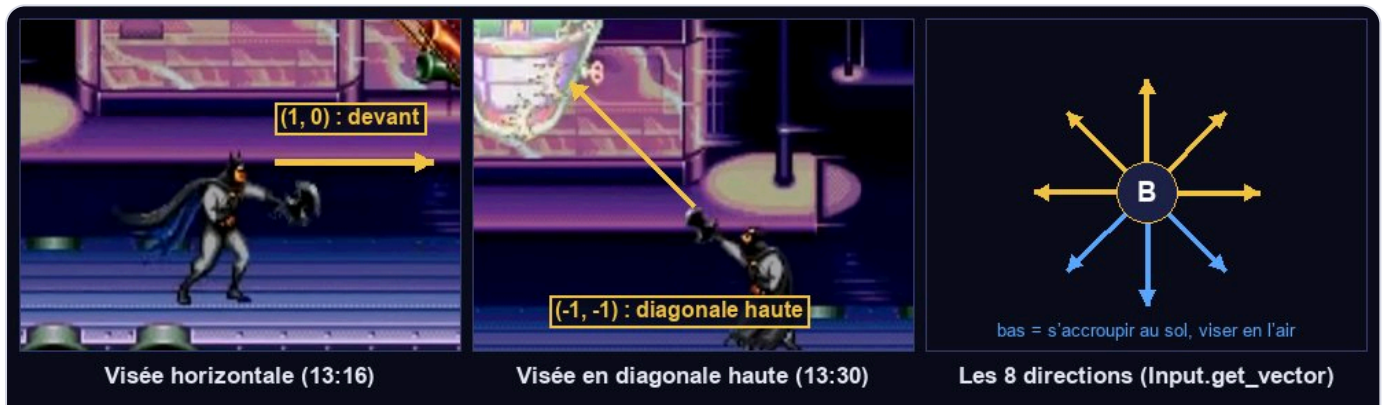
```

```
fsm.force("Victory")

# --- Animation -----

## Joue l'animation « base », ou sa variante de tir (« run_shoot_diag_up »...) si elle existe.
func play_anim(base: StringName) -> void:
    if sprite.sprite_frames == null:
        return
    var anim := base
    if _shoot_pose > 0.0:
        var shoot := StringName("%s_shoot_%s" % [base, _aim_suffix()])
        if sprite.sprite_frames.has_animation(shoot):
            anim = shoot
    if sprite.sprite_frames.has_animation(anim) and sprite.animation != anim:
        sprite.play(anim)

func _aim_suffix() -> String:
    if aim.y < -0.1:
        return "up" if absf(aim.x) < 0.1 else "diag_up"
    if aim.y > 0.1:
        return "down" if absf(aim.x) < 0.1 else "diag_down"
    return "fwd"
```



Le tir dans 8 directions. `compute_aim()` combine la croix (x, y) ; au sol, « bas » sert à s'accroupir et n'est pas une direction de tir.

Comprendre le code

- `@onready var weapons: Node = get_node_or_null("WeaponSystem")` : le système d'armes n'existe pas encore (étape 06). `get_node_or_null` renvoie `null` sans erreur, et `try_attack()` vérifie `if weapons`. Le code est prêt pour la suite sans planter aujourd'hui.
- `melee_hitbox.scale.x = facing` : une échelle de -1 retourne la zone, donc le poing passe de l'autre côté du corps.
- `compute_aim()` : la croix donne x et y entre -1 et 1 ; `normalized()` transforme (1, -1) en une diagonale de longueur 1. Sans direction, on tire devant soi.
- `try_attack()` renvoie `&"melee"` au lieu de lancer la mêlée lui-même : c'est l'état qui décide s'il veut passer en `Melee` (au sol) ou donner un coup de pied sans changer d'état (en l'air).
- `_tick_invincibility()` : un compteur décrémenté à chaque tick. `int(t × 30) % 2` alterne entre 0 et 1 toutes les 1/30 s : c'est le clignotement.

- `_on_hurt()` : on vérifie d'abord `is_dead()` . Quand un coup tue, `Health` émet `died` avant que la `Hurtbox` émette `hurt` : sans ce test, on passerait de `Dead` à `Hurt` .
- `signal died` (de `Player`) et `Health.died` sont deux signaux différents : le premier signifie « l'animation de mort est finie, décidez de la suite ». C'est la section (étape 09) qui décidera : réapparition ou game over.

LES ÉTATS

5.5 · Mettre à jour `Idle` , `Run` , `Crouch` et `Air`



Les quatre états existants apprennent à attaquer. Dans `idle.gd` et `run.gd` , ajoute ce bloc **juste après le bloc du saut** :

res://player/states/idle.gd et run.gd (ajout après le saut)

```
if p.try_attack() == &"melee":
    transition_to(&"Melee")
    return
```

`crouch.gd` et `air.gd` changent davantage : remplace-les entièrement par leur version de référence.

res://player/states/crouch.gd

```
extends State

## Accroupi : esquive les tirs hauts et tire au ras du sol.
## Bas + saut sur une plateforme traversable = descendre (hypothèse de confort).

@onready var p: Player = owner
@onready var body_shape: CollisionShape2D = owner.get_node("Body")
@onready var hurt_shape: CollisionShape2D = owner.get_node("Hurtbox/CollisionShape2D")
var _hurt_pos := Vector2.ZERO

func enter(_previous: StringName, _data: Dictionary) -> void:
    p.velocity.x = 0.0
    # On abaisse la zone vulnérable : c'est ce qui rend l'esquive réelle.
    _hurt_pos = hurt_shape.position
    hurt_shape.position.y += 10.0
    hurt_shape.scale.y = 0.6

func exit() -> void:
    hurt_shape.position = _hurt_pos
    hurt_shape.scale.y = 1.0

func physics_update(delta: float) -> void:
    p.apply_gravity(delta)
    p.velocity.x = 0.0
    if not p.is_on_floor():
        transition_to(&"Air")
        return
    if p.wants_jump():
        p.consume_jump()
        p.drop_through_platform()
```

```

        transition_to(&"Air")
    return
if not Input.is_action_pressed("move_down"):
    transition_to(&"Idle")
    return
var dir := p.read_move_x()
if dir != 0.0:
    p.set_facing(int(signf(dir)))
p.try_attack(true)
p.play_anim(&"crouch")

```

res://player/states/air.gd

```

extends State
## Saut et chute. Contrôle aérien total, saut à hauteur variable,
## coup de pied sauté si un ennemi est à portée, sinon tir.

@export var kick_duration := 0.25
@onready var p: Player = owner
var _kick_left := 0.0

func enter(_previous: StringName, _data: Dictionary) -> void:
    _kick_left = 0.0

func exit() -> void:
    p.melee_off()

func physics_update(delta: float) -> void:
    # 1. Saut variable : relâcher le bouton pendant la montée coupe l'élan.
    if Input.is_action_just_released("jump") and p.velocity.y < 0.0:
        p.velocity.y *= p.jump_cut
        p.apply_gravity(delta)

    # 2. Saut retardé (coyote) : on vient de quitter le bord.
    if p.wants_jump() and p.can_jump():
        p.jump()

    # 3. Contrôle aérien complet, typique des run & gun.
    p.run(p.read_move_x())

    # 4. Accroche sous un rebord : maintenir « haut » près du sommet du saut.
    if Input.is_action_pressed("move_up") and p.velocity.y > -60.0:
        var ledge_y := p.find_ledge()
        if not is_nan(ledge_y):
            transition_to(&"Hang", {"ledge_y": ledge_y})
            return

    # 5. Attaque.
    if _kick_left > 0.0:
        _kick_left -= delta
        if _kick_left <= 0.0:
            p.melee_off()
    elif p.try_attack() == &"melee":
        _kick_left = kick_duration
        p.melee_on()

```

```
# 6. Atterrissage.
if p.is_on_floor() and p.velocity.y >= 0.0:
    transition_to(&"Run" if p.read_move_x() != 0.0 else &"Idle")
    return

if _kick_left > 0.0:
    p.play_anim(&"kick")
else:
    p.play_anim(&"jump" if p.velocity.y < 0.0 else &"fall")
```

POURQUOI ABAISSER LA HURTBBOX ACCROUPI

S'accroupir n'a de sens que si les tirs hauts passent au-dessus. Le sprite qui se baisse n'est qu'un dessin : ce qui compte pour le jeu, c'est la **zone vulnérable**. `enter()` la réduit, `exit()` la remet. Ce couple entrée/sortie garantit qu'on ne reste jamais avec une hurtbox rétrécie, quelle que soit la façon de quitter l'état (même par un coup reçu).

5.6 · L'état Melee



res://player/states/melee.gd

```
extends State
## Coup au contact. La Hitbox n'est active que sur les images « actives » du coup.

@export var duration := 0.24
@export var active_from := 0.05
@export var active_to := 0.16
@export var combo_window := 0.35 ## délai pour enchaîner poing puis pied

@onready var p: Player = owner
var _t := 0.0
var _active := false
var _combo := 0
var _last_end_ms := -100000

func enter(_previous: StringName, _data: Dictionary) -> void:
    _t = 0.0
    _active = false
    p.velocity.x = 0.0
    var since_last := (Time.get_ticks_msec() - _last_end_ms) / 1000.0
    _combo = (_combo + 1) % 2 if since_last < combo_window else 0
    p.play_anim(&"punch" if _combo == 0 else &"kick")

func exit() -> void:
    p.melee_off()
    _last_end_ms = Time.get_ticks_msec()

func physics_update(delta: float) -> void:
    p.apply_gravity(delta)
    p.velocity.x = 0.0
    _t += delta
    var should_be_active := _t >= active_from and _t < active_to
```

```

if should_be_active != _active:
    _active = should_be_active
if _active:
    p.melee_on()
else:
    p.melee_off()
if _t >= duration:
    transition_to("&Idle")

```

CONCEPT : PRÉPARATION, IMAGES ACTIVES, RÉCUPÉRATION

Un coup se découpe en trois temps : la **préparation** (0 → 0,05 s : le bras recule, aucun dégât), les images **actives** (0,05 → 0,16 s : la Hitbox existe) et la **recupération** (0,16 → 0,24 s : Batman est vulnérable et ne peut rien faire). Régler ces trois durées, c'est régler le « poids » d'une attaque. Tous les jeux de combat sont construits sur ce découpage.

5.7 · L'état Hang et le rebord



Suspendu sous le balcon. La sonde des mains (HandProbe) touche la zone de rebord (couche 8) : on passe dans l'état Hang.

```
res://player/states/hang.gd
```

```
extends State
```

```
## Suspendu au rebord. Saut = grimper ; bas + saut = se laisser tomber.
```

```
## On peut tirer devant, en diagonale basse et vers le bas.
```

```
@export var hang_offset := 44.0 ## distance mains → pieds, À CALIBRER selon le sprite
```

```
@onready var p: Player = owner
```

```
var _ledge_y := 0.0
```

```
func enter(_previous: StringName, data: Dictionary) -> void:
```

```
    _ledge_y = data.get("ledge_y", p.global_position.y)
```

```
    p.velocity = Vector2.ZERO
```

```
    p.global_position.y = _ledge_y + hang_offset
```

```
func physics_update(_delta: float) -> void:
```

```
    p.velocity = Vector2.ZERO
```

```
    if p.wants_jump():
```

```
        p.consume_jump()
```

```
        if Input.is_action_pressed("move_down"):
```

```
            transition_to(&"Air") # on lâche
```

```
        else:
```

```
            # Grimper : on pose les pieds juste au-dessus du rebord.
```

```
            p.global_position.y = _ledge_y - 1.0
```

```
            transition_to(&"Air") # Air détecte le sol au tick suivant et passe en Idle
```

```
    return
```

```
    var dir := p.read_move_x()
```

```
    if dir != 0.0:
```

```
        p.set_facing(int(signf(dir)))
```

```
    p.try_attack(false, false) # pas de tir vers le haut à travers le balcon
```

```
    p.play_anim(&"hang")
```

À FAIRE : LE REBORD DANS LA SALLE DE TEST

1. Dans `test_room.tscn`, ajoute une `Area2D` nommée `Ledge` : couche 8 seule, masque vide.
2. Sa **position** doit être exactement la surface du balcon : (200, 108). `find_ledge()` renvoie la position de la zone, pas celle de sa forme.
3. Enfant `CollisionShape2D` : rectangle 200×4, en position (0, 2), donc entre y = 108 et y = 112.

INTERPRÉTATION

La vidéo montre Batman suspendu, pas la manière d'y arriver. Comme le balcon se traverse par en dessous, on exige que le joueur **maintienne « haut »** près du sommet du saut ; sinon il traverse et retombe. C'est un choix de game design, exposé à la critique : note-le dans ton journal et change-le si tu trouves mieux.

res://player/states/hurt.gd

```
extends State
## Recul après un coup : pas de contrôle pendant hurt_time.

@onready var p: Player = owner
var _t := 0.0

func enter(_previous: StringName, _data: Dictionary) -> void:
    _t = 0.0
    p.melee_off()
    p.play_anim(&"hurt")

func physics_update(delta: float) -> void:
    _t += delta
    p.apply_gravity(delta)
    p.velocity.x = move_toward(p.velocity.x, 0.0, 600.0 * delta)
    if _t < p.hurt_time:
        return
    transition_to(&"Idle" if p.is_on_floor() else &"Air")
```

res://player/states/grabbed.gd

```
extends State
## Empoigné par un costaud (tome 05). Marteler les boutons pour se dégager.

@export var escape_presses := 6
@onready var p: Player = owner
var _presses := 0
var _grabber: Node2D

func enter(_previous: StringName, data: Dictionary) -> void:
    _grabber = data.get(&"grabber")
    _presses = 0
    p.velocity = Vector2.ZERO
    p.melee_off()
    p.play_anim(&"grabbed")

func physics_update(_delta: float) -> void:
    p.velocity = Vector2.ZERO
    for action in ["attack", "jump", "move_left", "move_right"]:
        if Input.is_action_just_pressed(action):
            _presses += 1
    if _presses >= escape_presses or not is_instance_valid(_grabber):
        if is_instance_valid(_grabber) and _grabber.has_method(&"release"):
            _grabber.release()
        p.velocity = Vector2(-p.facing * 120.0, -150.0)
        p.start_invincibility(0.8)
        transition_to(&"Air")
```

res://player/states/dead.gd

extends State

Animation de mort, puis signal « died » : la section décide (réapparition ou game over).

@export var duration := 2.0

@onready var p: Player = owner

var _t := 0.0

var _reported := false

func enter(_previous: StringName, _data: Dictionary) -> void:

 _t = 0.0

 _reported = false

 p.melee_off()

 p.hurtbox.invincible = true

 p.velocity = Vector2(-p.facing * 60.0, -200.0)

 p.play_anim(&"dead")

func physics_update(delta: float) -> void:

 p.apply_gravity(delta)

if p.is_on_floor():

 p.velocity.x = 0.0

 _t += delta

if _t >= duration **and not** _reported:

 _reported = true

 p.died.emit()

res://player/states/victory.gd

extends State

Pose de victoire après un boss : cape fermée, immobile (vidéo 6:20, 10:41, 14:33).

@onready var p: Player = owner

func enter(_previous: StringName, _data: Dictionary) -> void:

 p.melee_off()

 p.hurtbox.invincible = true

 p.velocity.x = 0.0

 p.play_anim(&"victory")

func physics_update(delta: float) -> void:

 p.apply_gravity(delta)

 p.velocity.x = 0.0

POURQUOI DES COMPTEURS PLUTÔT QUE DES **AWAIT** ?

On aurait pu écrire `await get_tree().create_timer(0.35).timeout` puis `transition_to(&"Idle")` dans `Hurt`. Mais si Batman reçoit un second coup pendant l'attente, le premier `await` se réveille quand même plus tard et le renvoie en `Idle` au mauvais moment. Un compteur (`_t`) vit dans l'état : quand on sort de l'état, il n'existe plus de minuterie fantôme. Règle du parcours : **des compteurs pour la logique de jeu, `await` pour les séquences scriptées.**

TESTER SANS ENNEMIS

5.9 · Une cible, un piège, une réapparition



MÉTHODE : FABRIQUER SES OUTILS DE TEST

Les ennemis n'arrivent qu'à l'étape 07. Plutôt que d'attendre pour tester le combat, on fabrique des objets de test minimalistes : une cible qui encaisse, un piège qui blesse, et des raccourcis clavier pour forcer un état. Ces outils te serviront encore longtemps.

La cible : crée `res://sandbox/dummy.tscn`.

```
Dummy (Node2D, dummy.gd)
├─ Health                max_health = 5
├─ Hurtbox               couche 6 · masque vide · health = Health
├─ CollisionShape2D      rectangle 16×32, position (0, -16)
└─ ColorRect             16×32 · position (-8, -32) · rouge
```

`res://sandbox/dummy.gd`

```
extends Node2D
## Cible d'entraînement : encaisse, clignote, se répare.

@onready var health: Health = $Health
@onready var body: ColorRect = $ColorRect

func _ready() -> void:
    health.damaged.connect(_on_damaged)
    health.died.connect(_on_died)

func _on_damaged(amount: int) -> void:
    print("Cible : -%d (reste %d)" % [amount, health.current])
    body.color = Color.WHITE
    await get_tree().create_timer(0.06).timeout
    body.color = Color(0.8, 0.2, 0.2)

func _on_died() -> void:
    print("Cible détruite : réparation dans 1 s")
    await get_tree().create_timer(1.0).timeout
    health.reset()
```

Ici, `await` est acceptable : c'est un objet de test, et une séquence purement visuelle.

Le piège : une `Hitbox` qui blesse Batman à chaque nouveau contact. Crée `res://sandbox/spikes.gd`, puis dans la salle de test un nœud `Hitbox` nommé `Spikes` avec ce script (couche vide, masque 5, damage 2), une forme de 32×6 et un `ColorRect` rouge, posé sur le sol vers x = 500.

```
res://sandbox/spikes.gd
```

```
extends Hitbox
## Piège de test : blesse à chaque nouveau contact.

func _ready() -> void:
    super._ready() # garde le comportement de Hitbox (monitorable = false, connexion...)
    # Quand Batman sort du piège, il pourra être retouché en y revenant.
    area_exited.connect(func(a: Area2D) -> void: _already_hit.erase(a))
```

La salle de test reçoit enfin un script : réapparition après la mort, et raccourcis pour les états difficiles à provoquer.

```
res://level/test_room.gd
```

```
extends Node2D
## Salle de test : réapparition après la mort, et raccourcis de débogage.

@onready var player: Player = $Player

func _ready() -> void:
    player.died.connect(player.respawn)

func _unhandled_input(event: InputEvent) -> void:
    if event.is_action_pressed("start"):
        # Simule une empoignade : la salle joue le rôle du costaud.
        player.fsm.force(&"Grabbed", {"grabber": self})
    elif event.is InputEventKey and event.pressed and not event.echo:
        match event.keycode:
            KEY_V:
                player.celebrate()
            KEY_I:
                player.fsm.force(&"Idle")
```

À FAIRE

1. Attache `test_room.gd` à `TestRoom`.
2. Place deux instances de `dummy.tscn` : une au sol en (260, 200), une sur le balcon en (240, 108).
3. `F6`, avec les formes de collision visibles.

TESTE

- **Mêlée** : collé à la cible du sol, **J** donne un coup de poing (« Melee »), deux appuis rapides enchaînent poing et pied. La Sortie affiche les dégâts.
- **Loin de la cible**, **J** ne fait rien : il n'y a pas encore d'arme.
- **Coup de pied sauté** : saute près de la cible et appuie sur **J**.
- **Accroche** : sous le balcon, saute en maintenant haut : « Hang ». Saut : Batman grimpe sur le balcon. Bas + saut : il se laisse tomber.
- **Accroupi** : la forme de la Hurtbox (couche 5) rétrécit vers le bas.
- **Piège** : marche dessus. Recul (« Hurt »), clignotement de 1,5 s pendant lequel le piège n'a plus d'effet. Quatre contacts = 8 points de dégâts : « Dead », puis réapparition en haut de l'écran.
- **Raccourcis** : **Entrée** → « Grabbed » ; martèle six fois **J** ou **K** pour te libérer. **V** → « Victory », **I** → retour à « Idle ».

SI ÇA NE MARCHE PAS

- **Le coup ne touche jamais** : la **MeleeSense** et la **MeleeHitbox** doivent avoir le masque 6, la Hurtbox de la cible la couche 6.
- **Le coup touche du mauvais côté** : les formes de **MeleeHitbox** et **MeleeSense** doivent être à $x = +16$ (devant, quand Batman regarde à droite).
- **Impossible de s'accrocher** : vérifie que la **HandProbe** a le masque 8 et le **Ledge** la couche 8, puis que tu maintiens bien haut *pendant* la descente du saut.
- **Après avoir grimpé, Batman retombe à travers le balcon** : la position du **Ledge** n'est pas à la surface du balcon ($y = 108$).
- **Le piège ne blesse qu'une fois** : **super._ready()** manque dans **spikes.gd**, ou le masque n'est pas 5.

5.10 · Les sprites (facultatif)



Tu peux continuer avec le rectangle bleu jusqu'à la fin du parcours. Si tu veux voir Batman maintenant :

À FAIRE

1. Copie `batman-and-robin/spritesheets/Sega Genesis - The Adventures of Batman and Robin - Playable Characters - Batman.png` dans `res://art/player/batman.png`.
2. Sur le nœud `Sprite` : **Sprite Frames** → **Nouveau SpriteFrames**, puis clique dessus : le panneau **SpriteFrames** s'ouvre en bas.
3. Renomme l'animation `default` en `idle`. Bouton **Ajouter des images depuis une feuille de sprites** (icône de grille) : choisis la planche, règle le découpage quand une rangée est régulière, et sélectionne les images.
4. Si la rangée n'est pas régulière (c'est souvent le cas sur ces planches) : crée une ressource `AtlasTexture` par image, avec **Atlas** = la planche et **Region** = le rectangle de l'image (l'éditeur de région permet de le tracer à la souris), puis glisse ces `AtlasTexture` dans l'animation.
5. Crée les animations avec ces noms exacts : `idle`, `run`, `jump`, `fall`, `crouch`, `punch`, `kick`, `hang`, `hurt`, `grabbed`, `dead`, `victory`. Facultatif : les variantes de tir, comme `run_shoot_fwd` ou `idle_shoot_diag_up`.
6. Décoche **Loop** pour `punch`, `kick`, `dead` et `victory`. Vitesse : 10 à 12 images par seconde.
7. Enregistre ce `SpriteFrames` en fichier (`res://art/player/batman_frames.tres`, via la flèche du champ → Enregistrer), et assigne-le au champ `frames` de `batman.tres`.
8. Règle l'**offset** du `Sprite` pour que les pieds touchent l'origine, puis cache le `Placeholder` (icône d'œil).

POURQUOI PASSER PAR `BATMAN.TRES`

`player.gd` applique `hero.frames` au `Sprite` dans `_ready()`. C'est ce qui permettra à l'écran de sélection (étape 10) de transformer le même `player.tscn` en Batman ou en Robin, sans dupliquer la scène.

POINT DE CONTRÔLE

TU AS FINI L'ÉTAPE QUAND

- ☐ Batman s'accroupit, descend du balcon avec bas + saut, s'accroche sous le balcon en maintenant haut, puis grimpe.
- ☐ Devant une cible, l'attaque donne un coup de poing (puis un pied en enchaînant) ; en l'air, un coup de pied.
- ☐ Un coup reçu fait reculer Batman et le rend invincible 1,5 s en clignotant.
- ☐ À 0 point de vie : animation de mort, puis réapparition depuis le haut de l'écran.
- ☐ Les raccourcis `Entrée`, `V` et `I` provoquent Grabbed, Victory et Idle.

```
git add -A && git commit -m "Étape 05 : combat, accroche, dégâts, mort et victoire"
```

CE QUE TU RETIENS

? Pourquoi le tir n'est-il pas un état ?

Parce que Batman peut tirer pendant qu'il fait autre chose (courir, sauter, être accroupi ou suspendu). Un état représente une situation exclusive ; une action combinable avec plusieurs états est une capacité, appelée par les états qui l'autorisent.

? Que se passerait-il si `Crouch.exit()` ne remettait pas la Hurtbox en place ?

En quittant l'accroupi (en se relevant, en tombant, ou touché), Batman garderait une zone vulnérable réduite et abaissée : il deviendrait invulnérable aux tirs hauts pour toujours. Tout ce que `enter()` modifie, `exit()` doit le restaurer.

? Pourquoi le coup de pied sauté n'est-il pas un état ?

Parce qu'en l'air, Batman garde le contrôle de son déplacement et continue de subir la gravité : seule la hitbox s'active un instant. Un compteur (`_kick_left`) dans l'état `Air` suffit.

? Le clignotement d'invincibilité est-il de la logique ou de l'affichage ?

Les deux, séparés : la **logique** est `hurtbox.invincible = true` pendant la durée ; l'**affichage** est `sprite.visible` qui alterne. On pourrait remplacer le clignotement par une transparence sans toucher à la règle.

POUR ALLER PLUS LOIN

- Change `active_from` à 0,12 dans `Melee` : le coup devient lent et « lourd ». Mets-le à 0 : il devient instantané. Lequel est le plus agréable ?
- Ajoute une sortie de `Hang` vers la gauche ou la droite : Batman avance suspendu le long du balcon (attention aux trous).
- Lis [la fin du tome 03](#).

ARMES, JAUGE ET BONUS

Trois armes qui ne diffèrent que par leurs chiffres, une réserve de projectiles qui imite la limite de sprites de la console, une jauge qui se remplit quand on ne tire pas, et des bonus qui changent de couleur. Au passage, une boîte à outils d'effets que tout le jeu utilisera.

⌚ ~3 h

Prérequis : étape 05

Guide : [tome 04](#)

CE QUE TU VAS APPRENDRE

GODOT 4

- `Resource` comme fiche de données, `preload`
- Fonctions `static` et classes utilitaires
- `PackedScene.instantiate()`, `add_child`, `call_deferred`
- `process_mode`, `VisibleOnScreenNotifier2D`
- `CanvasLayer`, `ProgressBar`

DÉVELOPPEMENT DE JEU

- La conception pilotée par les données
- La réserve d'objets (*pooling*)
- Le patron Observateur au service du découplage
- L'objet à état cyclique
- Interroger le monde par groupes

LES RÈGLES

6.1 · Ce que fait le jeu original



Le même bonus, à 1 s d'intervalle



Le même bonus à une seconde d'intervalle. Sa couleur, donc l'arme qu'il donne, change en permanence.

Couleur du bonus	Arme	Caractère
Rouge / orange	Batarang	Tir dispersé : plus de projectiles en éventail quand le niveau monte
Vert	Bolas	Pas de dispersion, mais les plus gros dégâts
Bleu	Shuriken	L'arme la plus faible

- Ramasser la couleur déjà en main fait monter l'arme d'un niveau, jusqu'à 7.
- La barre sous le score se remplit seule en ~1,1 s quand on ne tire pas. Le tir suivant est un **tir puissant**, et la vide.
- Le crâne qui roule au sol déclenche un flash blanc et détruit tous les ennemis à l'écran.

DANS LA VIDÉO

La jauge qui se remplit puis se vide : ▶ 1:46 — ▶ 1:48 . Le bonus qui change de couleur : ▶ 12:30 . Le crâne : ▶ 12:23 .

LES ARMES EN DONNÉES

6.2 · La fiche WeaponData



CONCEPT : LA CONCEPTION PILOTÉE PAR LES DONNÉES

Les trois armes ont le même comportement (tirer N projectiles à telle vitesse, avec tels dégâts) ; seuls les **chiffres** changent. On écrit donc le comportement une seule fois, et chaque arme devient un fichier de données éditable dans l'Inspecteur. Un game designer peut équilibrer le jeu sans toucher au code, et ajouter une quatrième arme ne demande aucune ligne de code.

```
res://weapons/weapon_data.gd
```

```
class_name WeaponData
extends Resource

## Description d'une arme. Les tableaux « par niveau » ont 7 cases (niveaux 1 à 7).

@export var id: StringName = &"batarang"
@export var display_name := "Batarang"
@export var color := Color(1.0, 0.42, 0.24)    ## couleur du bonus et de la jauge
@export var projectile_texture: Texture2D
@export var spin_speed := 14.0                ## rad/s, rotation purement visuelle

@export_group("Tir normal")
@export var speed := 300.0                    ## px/s
@export var fire_interval := 0.16             ## s entre deux tirs, bouton maintenu
@export var count_by_level := PackedInt32Array([1, 2, 3, 3, 4, 4, 5])
@export var damage_by_level := PackedInt32Array([1, 1, 1, 2, 2, 2, 3])
@export var spread_deg := 12.0                ## écart entre deux projectiles de l'éventail
@export var projectile_scale := 1.0

@export_group("Tir puissant")
@export var power_count := 5
@export var power_spread_deg := 15.0
@export var power_damage := 6
@export var power_scale := 2.0
@export var power_speed := 360.0

func count_at(level: int) -> int:
    return count_by_level[clampi(level, 1, count_by_level.size()) - 1]

func damage_at(level: int) -> int:
    return damage_by_level[clampi(level, 1, damage_by_level.size()) - 1]
```

À FAIRE

- Dans `res://weapons/data/`, crée trois ressources `WeaponData` (clic droit → Nouveau → Ressource) : `batarang.tres`, `bolas.tres`, `shuriken.tres`.
- Règle-les avec ces valeurs (les champs non cités gardent leur valeur par défaut) :

Champ	batarang.tres	bolas.tres	shuriken.tres
id / display_name	batarang / Batarang	bolas / Bolas	shuriken / Shuriken
color	#ff6b3d	#43d18f	#5aa8ff
speed	300	260	360
fire_interval	0.16	0.22	0.10
count_by_level	1, 2, 3, 3, 4, 4, 5	1, 1, 1, 1, 1, 1, 1	1, 1, 2, 2, 2, 3, 3
damage_by_level	1, 1, 1, 2, 2, 2, 3	3, 3, 4, 4, 5, 5, 6	1, 1, 1, 1, 1, 2, 2
spread_deg	12	0	6
projectile_scale	1.0	1.0	0.8
power_count / power_damage	5 / 6	1 / 16 (power_scale 2.5)	8 / 3

À CALIBRER

L'effet exact de chaque niveau n'est documenté nulle part : ces chiffres sont des hypothèses raisonnables. C'est justement l'intérêt des données : tu les ajusteras en jouant, sans toucher au code. Hypothèse retenue : changer de couleur ramène l'arme au niveau 1.

6.3 · Le registre des armes



res://weapons/weapons.gd

```
class_name Weapons
## Registre des armes : identifiant -> données.

const DATA := {
    &"batarang": preload("res://weapons/data/batarang.tres"),
    &"bolas": preload("res://weapons/data/bolas.tres"),
    &"shuriken": preload("res://weapons/data/shuriken.tres"),
}
## Ordre du cycle de couleurs des bonus, observé : rouge/orange → bleu → vert.
const PICKUP_CYCLE: Array[StringName] = [&"batarang", &"shuriken", &"bolas"]

static func get_data(id: StringName) -> WeaponData:
    return DATA.get(id, DATA[&"batarang"])
```

PRELOAD ET RESSOURCES PARTAGÉES

`preload()` charge le fichier au moment où le script est compilé ; le chemin doit donc exister, d'où l'ordre : les `.tres` d'abord. Une ressource chargée est **partagée** : tous ceux qui chargent `batarang.tres` reçoivent le même objet. On ne modifie donc jamais une fiche pendant le jeu (sinon toutes les armes changent) ; on lit seulement ses valeurs.

LA BOÎTE À OUTILS D'EFFETS

6.4 · Le piège de la position mondiale



Avant d'écrire les effets, une expérience de deux minutes, pour comprendre un bug qui frappe tous les débutants.

EXPÉRIENCE

1. Nouvelle scène 2D `res://sandbox/spawn_test.tscn`. Ajoute un enfant `Node2D` nommé `Arena`, en position (100, 0).
2. Script sur la racine :

```
res://sandbox/spawn_test.gd
```

```
extends Node2D
```

```
func _ready() -> void:
    var a := ColorRect.new()
    a.size = Vector2(8, 8)
    a.color = Color.RED
    var marker := Node2D.new()
    marker.add_child(a)
    marker.global_position = Vector2(160, 112) # AVANT add_child : piège !
    $Arena.add_child(marker)
    print("Voulu : (160, 112) - obtenu : ", marker.global_position)
```

TESTE

La Sortie affiche « obtenu : (260, 112) » et le carré est décalé de 100 px. Un nœud sans parent n'a pas de « monde » : `global_position` y est rangé tel quel dans `position` (locale), puis devient relatif au parent quand on l'ajoute. Avec un parent à l'origine, le bug est invisible ; avec un parent placé ailleurs (l'arène d'un boss), l'objet surgit à des centaines de pixels.

6.5 · `Fx` : faire apparaître, exploser, flasher



La classe `Fx` règle ce problème une fois pour toutes avec `spawn()`, que tout le jeu utilisera pour faire apparaître bombes, explosions, bonus et ennemis.

res://fx/fx.gd

```
class_name Fx
## Effets visuels partagés.

const EXPLOSION := preload("res://fx/explosion.tscn")

## Ajoute « node » à la scène, à la position MONDE « at ». Parent par défaut : la scène courante.
## deferred = true pendant un signal de collision (pas d'ajout physique à ce moment-là).
static func spawn(node: Node2D, at: Vector2, parent: Node = null, deferred := false) -> void:
    if parent == null:
        parent = (Engine.get_main_loop() as SceneTree).current_scene
        # Avant add_child(), le nœud n'a pas de parent : global_position serait pris pour une
        # position LOCALE. On convertit donc nous-mêmes la position monde dans le repère du parent.
    var parent_2d := parent as Node2D
    node.position = parent_2d.to_local(at) if parent_2d else at
    if deferred:
        parent.add_child.call_deferred(node)
    else:
        parent.add_child(node)

static func explosion(at: Vector2, parent: Node = null) -> void:
    spawn(EXPLOSION.instantiate() as Node2D, at, parent)

## Flash plein écran : un rectangle de couleur au-dessus de tout, pendant quelques images.
static func flash_screen(tree: SceneTree, color := Color.WHITE, duration := 2.0 / 60.0) -> void:
    var layer := CanvasLayer.new()
    layer.layer = 90
    var rect := ColorRect.new()
    rect.color = color
    rect.set_anchors_preset(Control.PRESET_FULL_RECT)
    rect.mouse_filter = Control.MOUSE_FILTER_IGNORE
    layer.add_child(rect)
    tree.current_scene.add_child(layer)
    await tree.create_timer(duration, true, false, true).timeout
    layer.queue_free()
```

Elle a besoin d'une scène d'explosion. Crée d'abord son script, puis la scène :

res://fx/explosion.gd (racine AnimatedSprite2D de explosion.tscn)

```
extends AnimatedSprite2D
## Explosion « fire and forget » : joue son animation puis se détruit.

func _ready() -> void:
    # Tant que l'animation n'est pas dessinée (formes provisoires), on disparaît sans erreur.
    if sprite_frames == null or not sprite_frames.has_animation(&"default"):
        queue_free()
        return
    animation_finished.connect(queue_free)
    play(&"default")
```

À FAIRE

1. Nouvelle scène, racine `AnimatedSprite2D` nommée `Explosion`, script `explosion.gd`, enregistrée en `res://fx/explosion.tscn`.
2. Facultatif mais conseillé : dessine une bande de 4 images de 16×16 (des cercles orange de plus en plus grands, puis gris) et crée une animation `default` **non bouclée**, à 15 i/s. Sans animation, l'explosion se détruit aussitôt, sans erreur.
3. Crée ensuite `res://fx/fx.gd`. L'ordre compte : `fx.gd` fait un `preload` de `explosion.tscn`.

Comprendre le code

- `class_name Fx` sans `extends` : une classe utilitaire, jamais instanciée. On appelle ses fonctions `static` directement : `Fx.explosion(position)`.
- `Engine.get_main_loop() as SceneTree` : une fonction statique n'a pas de `get_tree()` (elle n'appartient à aucun nœud). On récupère l'arbre de scène du jeu par le moteur.
- `parent_2d.to_local(at)` : convertit une position du monde dans le repère du parent. C'est la correction du piège de 6.4.
- `add_child.call_deferred(node)` : l'ajout attend la fin de l'image. Indispensable quand on fait apparaître un objet physique pendant un signal de collision (un bonus qui sort d'un ennemi touché, par exemple).
- `create_timer(duration, true, false, true)` : le 4^e argument fait ignorer `Engine.time_scale` à la minuterie. Utile quand le jeu sera figé un instant (étape 13).

TESTE

Dans `spawn_test.gd`, remplace les deux lignes `marker.global_position = ...` et `$Arena.add_child(marker)` par `Fx.spawn(marker, Vector2(160, 112), $Arena)`. Relance : « obtenu : (160, 112) ».

LA RÉSERVE DE PROJECTILES

6.6 · Pool



Pourquoi une réserve d'objets. La console plafonne à 80 sprites : le jeu réserve un nombre fixe d'emplacements, exactement comme notre `Pool`.

CONCEPT : LA RÉSERVE D'OBJETS (*POOLING*)

Créer et détruire des nœuds en rafale (`instantiate()` puis `queue_free()`) coûte des allocations et des insertions dans l'arbre. On fabrique donc les projectiles **une fois**, au chargement, puis on les « endort » et on les « réveille ». Bonus : un nombre maximal de tirs simultanés, comme sur la console (quand les 24 emplacements sont pris, le bouton ne fait rien).

res://core/pool.gd

```
class_name Pool
extends Node

## Réserve d'objets recyclables. Chaque objet doit avoir un signal « released ».
## Le Pool est un Node simple : ses enfants Node2D sont donc en coordonnées du monde.

@export var scene: PackedScene
@export var size := 24
var _free: Array[Node] = []

func _ready() -> void:
    for i in size:
        var item := scene.instantiate()
        item.process_mode = Node.PROCESS_MODE_DISABLED
        (item as CanvasItem).hide()
        add_child(item)
        # item est typé Node : on passe par connect("released") pour que l'analyseur l'accepte.
        item.connect("released", _on_released.bind(item))
        _free.append(item)

## Renvoie un objet prêt à l'emploi, ou null si la réserve est vide (comme la limite de sprites).
func acquire() -> Node:
    if _free.is_empty():
        return null
    var item: Node = _free.pop_back()
    item.process_mode = Node.PROCESS_MODE_INHERIT
    (item as CanvasItem).show()
    return item

func _on_released(item: Node) -> void:
    (item as CanvasItem).hide()
    item.set_deferred("process_mode", Node.PROCESS_MODE_DISABLED)
    _free.append(item)
```

ENDORMIR UN NŒUD : PROCESS_MODE

`hide()` rend invisible, mais une `Area2D` cachée détecte toujours les collisions : un batarang « rangé » blesserait encore. `process_mode = DISABLED` coupe tout : plus de `_physics_process`, et (avec le réglage par défaut `disable_mode = REMOVE`) la zone sort de la simulation physique.

BIND()

`_on_released.bind(item)` fabrique un `Callable` qui appellera `_on_released(item)`. Le signal `released` n'a pas de paramètre ; `bind` ajoute celui dont on a besoin : *quel* objet revient dans la réserve.

6.7 · Le projectile



Un projectile est une `Hitbox` qui se déplace : il hérite de tout le système de dégâts de l'étape 02.

```

class_name Projectile
extends Hitbox
## Projectile recyclable (Pool). fire() le lance, release() le rend à la réserve.

signal released

@export var lifetime := 2.0
var velocity := Vector2.ZERO
var spin := 0.0
var pierce := false
var survives_offscreen := false
var _life := 0.0
var _active := false

@onready var sprite: Sprite2D = $Sprite2D
@onready var notifier: VisibleOnScreenNotifier2D = $VisibleOnScreenNotifier2D

func _ready() -> void:
    super._ready()
    landed.connect(_on_landed)
    notifier.screen_exited.connect(_on_screen_exited)

func fire(from: Vector2, direction: Vector2, move_speed: float, hit_damage: int,
    texture: Texture2D = null, size_scale := 1.0, is_power := false, spin_speed := 0.0) -> void:
    global_position = from
    velocity = direction.normalized() * move_speed
    damage = hit_damage
    facing = 1 if direction.x >= 0.0 else -1
    if texture:
        sprite.texture = texture
    scale = Vector2.ONE * size_scale
    spin = spin_speed * facing
    pierce = is_power
    single_hit = not pierce
    survives_offscreen = is_power
    _life = 0.0
    _active = true
    reset_hits()
    monitoring = true

func _physics_process(delta: float) -> void:
    if not _active:
        return
    global_position += velocity * delta
    sprite.rotation += spin * delta
    _life += delta
    if _life >= lifetime:
        release()

func release() -> void:
    if not _active:
        return
    # évite un double retour dans la réserve (impact + sortie d'écran)

```

```

_active = false
set_deferred("monitoring", false)
released.emit()

```

```

func _on_landed(_hurtbox: Hurtbox) -> void:
    if not pierce:
        release()

```

```

func _on_screen_exited() -> void:
    if not survives_offscreen:
        release()

```

À FAIRE

1. Nouvelle scène, racine `Area2D` nommée `Projectile`, script `projectile.gd`. Couche vide, masque 6. Enregistre en `res://weapons/projectile.tscn`.
2. Enfants : `Sprite2D` (texture provisoire : `res://icon.svg` avec **Scale** = (0,06 ; 0,06), soit 8 px), `CollisionShape2D` (cercle de rayon 5), `VisibleOnScreenNotifier2D` (laisse son rectangle par défaut).

Projectile (Area2D, projectile.gd)	couche aucune · masque 6 (hurtbox_ennemis)
├ Sprite2D	icon.svg × 0,06 en attendant un vrai sprite
├ CollisionShape2D	cercle de rayon 5
└ VisibleOnScreenNotifier2D	

Comprendre le code

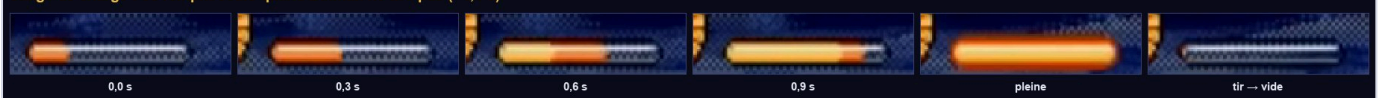
- `extends Hitbox` : un projectile est une hitbox. `super._ready()` exécute d'abord le `_ready()` de `Hitbox` (qui connecte `area_entered`), puis on ajoute nos connexions.
- `fire()` remet **tout** l'état à zéro : un objet recyclé garde les valeurs de sa vie précédente. Oublier une ligne ici (par exemple `reset_hits()`), c'est un bug qui n'apparaît qu'au 25^e tir.
- `_active` protège contre un double retour : un projectile qui touche un ennemi au bord de l'écran recevrait `landed` et `screen_exited`, et entrerait deux fois dans la réserve.
- Le tir puissant (`pierce`) traverse les cibles et survit hors de l'écran.

LE SYSTÈME D'ARMES

6.8 · WeaponSystem : tir, niveaux et jauge



Jauge de charge : se remplit seule quand Batman ne tire pas (~1,1 s)



La jauge de charge. Elle se remplit seule en ~1,1 s quand Batman ne tire pas, puis se vide d'un coup au tir suivant.

```
res://weapons/weapon_system.gd
```

```

class_name WeaponSystem
extends Node

```

```

## Arme équipée, niveau, cadence, jauge de charge. Ajoute ce nœud à player.tscn.

signal charge_changed(ratio: float)
signal equipped(data: WeaponData, level: int)

@export var projectile_scene: PackedScene      ## res://weapons/projectile.tscn
@export var pool_size := 24
@export var charge_time := 1.1                ## mesuré : jauge pleine en ~1,1 s
@export var charge_delay := 0.2               ## s sans tirer avant que la jauge remonte

var data: WeaponData
var level := 1
var charge := 0.0
var _cooldown := 0.0
var _idle := 0.0
var _pool: Pool

func _ready() -> void:
    # La réserve vit dans la scène courante, pas sous Batman :
    # un batarang lancé ne doit pas suivre ses déplacements.
    _pool = Pool.new()
    _pool.scene = projectile_scene
    _pool.size = pool_size
    get_tree().current_scene.add_child.call_deferred(_pool)
    Game.weapon_changed.connect(_on_game_weapon_changed)
    _on_game_weapon_changed(Game.weapon_id, Game.weapon_level)

func _physics_process(delta: float) -> void:
    _cooldown = maxf(_cooldown - delta, 0.0)
    _idle += delta
    if _idle >= charge_delay and charge < 1.0:
        charge = minf(charge + delta / charge_time, 1.0)
        charge_changed.emit(charge)

## Appelé par Player.try_attack() tant que le bouton est maintenu.
func try_fire(from: Vector2, direction: Vector2) -> bool:
    if _cooldown > 0.0 or data == null:
        return false
    if charge >= 1.0:
        _fire_pattern(from, direction, data.power_count, data.power_spread_deg,
            data.power_speed, data.power_damage, data.power_scale, true)
    else:
        _fire_pattern(from, direction, data.count_at(level), data.spread_deg,
            data.speed, data.damage_at(level), data.projectile_scale, false)
    charge = 0.0
    charge_changed.emit(charge)
    _idle = 0.0
    _cooldown = data.fire_interval
    return true

func fill_charge() -> void:
    charge = 1.0
    charge_changed.emit(charge)

```

```

func _fire_pattern(from: Vector2, direction: Vector2, count: int, spread_deg: float,
    move_speed: float, hit_damage: int, size_scale: float, is_power: bool) -> void:
    for i in count:
        var p := _pool.acquire() as Projectile
        if p == null:
            return # plus d'emplacement libre : on ne tire pas, comme sur la console
        var offset_deg := (i - (count - 1) / 2.0) * spread_deg
        var dir := direction.rotated(deg_to_rad(offset_deg))
        p.fire(from, dir, move_speed, hit_damage, data.projectile_texture,
            size_scale, is_power, data.spin_speed)
        p.sprite.modulate = data.color if is_power else Color.WHITE

func _on_game_weapon_changed(id: StringName, new_level: int) -> void:
    data = Weapons.get_data(id)
    level = new_level
    equipped.emit(data, level)

```

Comprendre le code

- **La jauge** : `_idle` compte le temps depuis le dernier tir. Après 0,2 s de repos, `charge` monte de `delta / 1,1` par tick, donc de 0 à 1 en 1,1 s. Tout tir la remet à 0.
- **L'éventail** : pour 3 projectiles écartés de 12°, `(i - 1) × 12` donne -12°, 0°, +12°. La formule `(i - (count - 1) / 2.0)` centre l'éventail quel que soit le nombre.
- **L'arme ne sait pas d'où vient le changement d'arme** : elle écoute `Game.weapon_changed`. Le bonus parle à `Game`, `Game` prévient tout le monde. Chacun peut être supprimé ou remplacé sans casser les autres.
- `add_child.call_deferred(_pool)` : pendant le `_ready()` d'un nœud, la scène courante est encore en construction ; on reporte l'ajout.

À FAIRE

1. Dans `player.tscn`, ajoute un nœud `WeaponSystem` (nommé exactement ainsi) avec **Projectile**
`Scene = projectile.tscn`. `player.gd` le trouve tout seul grâce à `get_node_or_null("WeaponSystem")`.
2. Dans `test_room.tscn`, ajoute un `CanvasLayer` nommé `DebugUI` avec deux enfants : un `ProgressBar` nommé `Charge` (position (8, 8), taille 60×6, **Show Percentage** décoché) et un `Label` nommé `Weapon` (position (8, 16), taille de police 8).
3. Complète `test_room.gd` :

res://level/test_room.gd

extends Node2D

Salle de test : réapparition après la mort, affichage de débogage, raccourcis.

@onready var player: Player = \$Player

@onready var charge_bar: ProgressBar = \$DebugUI/Charge

@onready var weapon_label: Label = \$DebugUI/Weapon

func _ready() -> void:

player.died.connect(player.respawn)

var ws := player.get_node_or_null("WeaponSystem") **as** WeaponSystem

if ws:

ws.charge_changed.connect(**func**(r: float) -> void: charge_bar.value = r * 100.0)

Game.weapon_changed.connect(**func**(id: StringName, lvl: int) -> void: weapon_label.text = "%s %d"
% [id, lvl])

weapon_label.text = "%s %d" % [Game.weapon_id, Game.weapon_level]

func _unhandled_input(event: InputEvent) -> void:

if event.is_action_pressed("start"):

Simule une empoignade : la salle joue le rôle du costaud.

player.fsm.force(&"Grabbed", {"grabber": self})

elif event **is** InputEventKey **and** event.pressed **and not** event.echo:

match event.keycode:

KEY_V:

player.celebrate()

KEY_I:

player.fsm.force(&"Idle")

KEY_1, KEY_2, KEY_3:

Raccourci de test : choisir l'arme, ou monter de niveau si c'est déjà la même.

var id: StringName = [&"batarang", &"bolas", &"shuriken"][event.keycode - KEY_1]

Game.set_weapon(id, Game.weapon_level + 1 **if** id == Game.weapon_id **else** 1)

TESTE

- Loin de la cible, maintenir **J** tire en rafale, devant soi. Haut + **J** : vers le haut. Diagonales en sautant. Accroupi : au ras du sol.
- Sans tirer pendant ~1,3 s, la barre se remplit ; le tir suivant est gros, coloré, et traverse la cible.
- Les touches **1**, **2**, **3** changent l'arme ; appuyer plusieurs fois sur la même fait monter le niveau (le batarang s'élargit en éventail).
- Ouvre **Débogueur → Moniteurs → Object → Nodes** : en tirant sans arrêt, le nombre de nœuds ne bouge pas. C'est la réserve qui travaille.
- Avec le shuriken niveau 7 et une cadence élevée, le tir s'interrompt parfois : les 24 emplacements sont pris.

SI ÇA NE MARCHE PAS

- « **Invalid call. Nonexistent function 'instantiate' in base 'Nil' »** : `Projectile Scene` n'est pas renseignée sur le `WeaponSystem`.
- **Les tirs restent collés à Batman** : la réserve a été ajoutée sous Batman au lieu de la scène courante.
- **Les tirs ne partent que vers la droite** : vérifie `compute_aim()` et que `facing` change bien (l'étiquette de débogage affiche la flèche).

LES BONUS

6.9 · La base `Pickup`



res://pickups/pickup.gd

```
class_name Pickup
extends CharacterBody2D
## Objet qui jaillit, retombe, clignote avant de disparaître, et se ramasse au contact.

@export var gravity := 600.0
@export var lifetime := 8.0 ## À CALIBRER
@onready var area: Area2D = $Area2D
@onready var sprite: Sprite2D = $Sprite2D

func _ready() -> void:
    area.body_entered.connect(_on_body_entered)
    velocity.y = -140.0 # petit saut à l'apparition

func _physics_process(delta: float) -> void:
    velocity.y = minf(velocity.y + gravity * delta, 300.0)
    move_and_slide()
    if is_on_floor():
        velocity.x = 0.0
    lifetime -= delta
    if lifetime < 2.0:
        sprite.visible = int(lifetime * 15.0) % 2 == 0
    if lifetime <= 0.0:
        queue_free()

func _on_body_entered(body: Node2D) -> void:
    if body is Player:
        collect(body)
        queue_free()

## À surcharger.
func collect(_player: Player) -> void:
    pass
```

CONCEPT : LA MÉTHODE À SURCHARGER

`Pickup` gère tout ce qui est commun (tomber, clignoter, détecter Batman) et laisse un trou : `collect()`. Chaque bonus concret hérite de `Pickup` et ne remplit que ce trou. C'est le patron *méthode modèle (template method)*.

6.10 · Le bonus d'arme à couleur cyclique



res://pickups/weapon_pickup.gd

```
class_name WeaponPickup
extends Pickup
## Disque chauve-souris dont la couleur (donc l'arme) change environ chaque seconde.

@export var cycle_period := 1.0    ## mesuré : ~1 changement par seconde
var _index := 0
var _t := 0.0

func _ready() -> void:
    super._ready()
    _index = randi() % Weapons.PICKUP_CYCLE.size()
    _refresh()

func _physics_process(delta: float) -> void:
    super._physics_process(delta)
    _t += delta
    if _t >= cycle_period:
        _t -= cycle_period
        _index = (_index + 1) % Weapons.PICKUP_CYCLE.size()
        _refresh()

func current_id() -> StringName:
    return Weapons.PICKUP_CYCLE[_index]

func _refresh() -> void:
    sprite.modulate = Weapons.get_data(current_id()).color

func collect(_player: Player) -> void:
    var id := current_id()
    if id == Game.weapon_id:
        Game.set_weapon(id, Game.weapon_level + 1)    # même couleur : niveau +1 (max 7)
    else:
        Game.set_weapon(id, 1)                        # hypothèse : nouvelle arme au niveau 1
    Game.add_score(100)
```

À FAIRE

1. Nouvelle scène, racine `CharacterBody2D` nommée `WeaponPickup`, script `weapon_pickup.gd`.
Couche vide, masque 1 et 2.
2. Enfants : `Sprite2D` (`icon.svg`, scale 0,1), `CollisionShape2D` (rectangle 10×10, position (0, -5)),
`Area2D` (couche 7, masque 3) avec son propre `CollisionShape2D` (rectangle 14×14, position (0, -7)).
3. Enregistre en `res://pickups/weapon_pickup.tscn`, puis places-en deux dans la salle de test.

```
WeaponPickup (CharacterBody2D)  couche aucune · masque 1+2 : il tombe et se pose
├─ Sprite2D
├─ CollisionShape2D              petit rectangle
└─ Area2D                      couche 7 « bonus » · masque 3 « corps_joueur »
    └─ CollisionShape2D
```

TESTE

Les bonus font un petit saut, se posent et changent de couleur chaque seconde. Ramasse-en un au bon moment : l'étiquette de débogage affiche l'arme et son niveau. Au bout de 6 s, le bonus clignote, puis disparaît à 8 s.

CONCEPT : L'OBJET À ÉTAT CYCLIQUE

C'est un mini-jeu de timing : toucher le bonus au bon moment pour avoir l'arme voulue. Une seule variable (`_index`) et un compteur suffisent. La couleur n'est qu'une **conséquence** de l'état, recalculée par `_refresh()` : on ne stocke jamais deux fois la même information.

6.11 · Le crâne et le bonus de charge



Le crâne smart bomb. Batman touche le crâne qui roule ; 2 images de blanc ; tous les ennemis à l'écran explosent.

res://pickups/skull_bomb.gd

```
class_name SkullBomb
extends Pickup
## Crâne qui roule vers le joueur. Contact : flash blanc + destruction des ennemis à l'écran.

@export var roll_speed := 40.0
@export var boss_damage := 10 ## hypothèse : les boss encaissent un coup fixe

func _physics_process(delta: float) -> void:
    var player := get_tree().get_first_node_in_group("player") as Node2D
    if player and is_on_floor():
        velocity.x = signf(player.global_position.x - global_position.x) * roll_speed
        sprite.rotation += velocity.x / 6.0 * delta # il roule
    super._physics_process(delta)

func collect(player: Player) -> void:
    Fx.flash_screen(get_tree(), Color.WHITE, 2.0 / 60.0)
    var view := player.camera.view_rect() if player.camera else get_viewport().get_visible_rect()
    for enemy in get_tree().get_nodes_in_group("enemies"):
        if view.has_point(enemy.global_position) and enemy.has_method("kill"):
            enemy.kill()
    for boss in get_tree().get_nodes_in_group("bosses"):
        # Seul un boss en plein combat encaisse : celui de la section 1 est déjà dans la scène
        # dès le départ. get() renvoie null si la propriété n'existe pas (pas de classe Boss avant
        le tome 07).
        if not boss.get("active"):
            continue
        var h: Health = boss.get_node_or_null("Health")
        if h:
            h.damage(boss_damage)
```

res://pickups/charge_pickup.gd

```
class_name ChargePickup
extends Pickup
## Remplit instantanément la jauge (TASVideos).

func collect(player: Player) -> void:
    if player.weapons:
        player.weapons.fill_charge()
```

À FAIRE

1. Crée `skull_bomb.tscn` et `charge_pickup.tscn` comme `weapon_pickup.tscn` (même arbre), avec leur script. Astuce : $\equiv$ **Scène** $\rightarrow$ **Enregistrer la scène sous** à partir de `weapon_pickup.tscn`, puis change le script de la racine.
2. Donne-leur une couleur reconnaissable (`modulate` du `Sprite2D`) : blanc cassé pour le crâne, jaune pour la charge.
3. Places-en un de chaque dans la salle de test.

TESTE

Le crâne roule vers Batman ; le toucher blanchit l'écran pendant 2 images (les cibles ne sont pas des ennemis : elles survivent, on testera la destruction à l'étape 07). Le bonus de charge remplit la barre d'un coup.

LES GROUPES : INTERROGER LE MONDE SANS LE CONNAÎTRE

Le crâne ne sait pas quels ennemis existent. Il demande au `SceneTree` « tous les nœuds du groupe `enemies` », et chaque ennemi s'ajoutera lui-même au groupe à sa création. Pour les boss, il vérifie aussi qu'ils sont `active` : un boss qui attend au bout de la section ne doit pas perdre de vie avant le combat. `boss.get("active")` lit la propriété par son nom, sans avoir besoin de la classe `Boss` (elle n'arrive qu'à l'étape 09).

POINT DE CONTRÔLE

TU AS FINI L'ÉTAPE QUAND

- ☐ Maintenir l'attaque tire en rafale, dans 8 directions, à la cadence de l'arme.
- ☐ Sans tirer pendant ~1,3 s, la jauge se remplit ; le tir suivant est gros, coloré et traverse les cibles.
- ☐ Un bonus posé au sol change de couleur chaque seconde ; le ramasser change l'arme ou augmente son niveau.
- ☐ Plus de 24 projectiles à l'écran est impossible.
- ☐ Le nombre de nœuds reste stable pendant les rafales.

```
git git add -A && git commit -m "Étape 06 : armes en données, réserve, jauge, bonus"
```

CE QUE TU RETIENS

? Pourquoi les armes sont-elles des fichiers `.tres` plutôt que trois scripts ?

Parce que leur comportement est identique et que seuls les chiffres diffèrent. Un seul code pour le comportement, des données pour les variantes : on équilibre sans programmer, et on ajoute une arme sans écrire de code.

? Que se passerait-il si `Pool` se contentait de `hide()` ?

Les projectiles rangés resteraient actifs physiquement : invisibles, ils continueraient de détecter et de blesser les ennemis qui passent à l'endroit où ils ont été rangés. `process_mode = DISABLED` les retire de la simulation.

? Pourquoi le bonus appelle-t-il `Game.set_weapon()` au lieu de modifier directement le `WeaponSystem` de Batman ?

Pour le découplage : le bonus n'a pas à savoir où est le système d'armes. `Game` émet `weapon_changed`, et tous ceux qui sont concernés (le système d'armes, le HUD de l'étape 10) réagissent. L'arme survit aussi aux changements de scène, puisqu'elle est rangée dans `Game`.

? Quel bug évite `Fx.spawn()` ?

Un nœud qui reçoit `global_position` avant d'avoir un parent stocke cette valeur comme position locale ; une fois ajouté sous un parent qui n'est pas à l'origine, il apparaît décalé de la position de ce parent. `spawn()` convertit la position monde dans le repère du parent avant l'ajout.

POUR ALLER PLUS LOIN

- Crée une quatrième arme « boomerang » (couleur jaune) uniquement avec un nouveau `.tres` et une ligne dans `Weapons.DATA`.
- Dessine de vrais sprites pour le batarang (8×8, 2 images) et assigne-les à `projectile_texture`.
- Lis [la section « réserve de projectiles » du tome 04](#).

ENNEMIS ET ARÈNES À VAGUES

Le cœur du rythme de Batman & Robin : la caméra se fige, des vagues d'ennemis arrivent jusqu'à un quota, « GO! » s'affiche, on avance. On écrit une classe de base commune, six ennemis qui n'en écrivent que le comportement, et un « directeur » qui orchestre les vagues.

⌚ ~5 h

Prérequis : étape 06

Guide : [tome 05](#)

CE QUE TU VAS APPRENDRE

GODOT 4

- Hériter d'une classe à soi (`extends Enemy`)
- `enum` + `match`
- `Path2D` , `PathFollow2D`
- `RayCast2D` , `tree_exiting`
- Tableaux de ressources dans l'Inspecteur

DÉVELOPPEMENT DE JEU

- Héritage et composition
- L'IA par états simples
- Le « directeur » de rencontres
- Tension et détente
- Les tables de butin
- Les trajectoires dessinées

LE RYTHME

7.1 · Observer une arène



Le cycle d'une arène. La caméra est figée pendant la vague ; une fois l'écran nettoyé, la bulle « GO! » apparaît ; la caméra repart.

DANS LA VIDÉO

Regarde [▶ 1:52](#) – [▶ 2:27](#) : la caméra s'arrête, les clowns tombent du haut, sautent du balcon ou arrivent par les côtés. Jamais plus de trois ou quatre à la fois. Quand l'écran est vide, « GO! » clignote à droite et la caméra repart. Même chose à [▶ 3:18](#) , [▶ 7:49](#) , [▶ 11:41](#) .

CONCEPT : TENSION ET DÉTENTE

Arène (tension : l'écran est figé, il faut survivre) → couloir (détente : on avance, on ramasse des bonus) → arène... Ce rythme est le squelette du level design de presque tous les jeux d'action. Le joueur respire entre deux combats, et chaque nouvelle arène est un petit événement.

LA BASE COMMUNE

7.2 · Atelier : des formes de substitution pour les ennemis



Les ennemis utilisent un `AnimatedSprite2D` nommé `Sprite`, dont le code change la teinte (flash quand on les touche, couleur des ballons). Un rectangle enfant ne suivrait pas ces teintes : il faut que le sprite affiche lui-même quelque chose.

À FAIRE

1. Dans un logiciel de dessin, crée une image **blanche** de 16×16 pixels et enregistre-la en `res://art/placeholder/white.png`.
2. Crée une ressource `SpriteFrames` (clic droit → Nouveau → Ressource → SpriteFrames) : `res://art/placeholder/placeholder_frames.tres`. Ouvre-la, et ajoute `white.png` comme unique image de l'animation `default`.
3. Pour chaque ennemi, le `Sprite` recevra ce `SpriteFrames`, une **Scale** qui donne sa taille (16×32 = scale (1, 2)), un **Offset** y = -8 pour que le bas touche l'origine, et une couleur via **Modulate**.

MODULATE ET SELF_MODULATE

Deux teintes se multiplient : `modulate` s'applique au nœud **et à ses enfants**, `self_modulate` au nœud seul. Le code des ennemis utilise `self_modulate` pour le flash et la couleur des ballons ; on réserve donc `modulate` à la couleur de base de la forme de substitution.

7.3 · La classe `Enemy`



CONCEPT : HÉRITAGE ET COMPOSITION

Tous les ennemis ont une vie, une hurtbox, un score, un butin et une mort spectaculaire. On met ce socle dans une classe `Enemy`, qui s'appuie sur les composants de l'étape 02 (composition). Chaque ennemi concret hérite d' `Enemy` (héritage) et n'écrit qu'une fonction : `think()`, son comportement. On combine les deux approches, chacune là où elle est la plus simple.

```
res://enemies/enemy.gd
```

```
class_name Enemy
extends CharacterBody2D

## Base de tous les ennemis : vie, flash de dégâts, score, butin, mort.
## Les classes filles écrivent think(delta) et règlent velocity.

signal killed(enemy: Enemy)

@export var score_value := 100
@export var gravity := 900.0
```

```

@export var uses_gravity := true
@export var drops: Array[PackedScene] = []
@export_range(0.0, 1.0) var drop_chance := 0.0
@export var knockback_scale := 0.4

var facing := -1
var player: Player
var _flash_left := 0.0
var _drop_left := 0.0
var _dead := false

@onready var sprite: AnimatedSprite2D = $Sprite
@onready var health: Health = $Health
@onready var hurtbox: Hurtbox = $Hurtbox
## Facultative : tous les ennemis n'en ont pas.
@onready var contact_hitbox: Hitbox = get_node_or_null("ContactHitbox")

func _ready() -> void:
    add_to_group("enemies")
    player = get_tree().get_first_node_in_group("player") as Player
    hurtbox.hurt.connect(_on_hurt)
    health.died.connect(kill)

func _physics_process(delta: float) -> void:
    if uses_gravity and not is_on_floor():
        velocity.y = minf(velocity.y + gravity * delta, 400.0)
    think(delta)
    move_and_slide()
    # Toucher le corps repousse Batman À L'OPPOSÉ de l'ennemi, quel que soit son regard.
    if contact_hitbox and player:
        contact_hitbox.facing = 1 if player.global_position.x >= global_position.x else -1
    if _flash_left > 0.0:
        _flash_left -= delta
        if _flash_left <= 0.0:
            sprite.self_modulate = Color.WHITE
    if _drop_left > 0.0:
        _drop_left -= delta
        if _drop_left <= 0.0:
            set_collision_mask_value(2, true)

## Comportement propre à chaque ennemi.
func think(_delta: float) -> void:
    pass

## Comme Player.play_anim : ne fait rien si l'animation n'existe pas encore.
func play_anim(anim: StringName) -> void:
    if sprite.sprite_frames and sprite.sprite_frames.has_animation(anim) and sprite.animation != anim:
        sprite.play(anim)

func face_player() -> void:
    if player:
        set_facing(int(signf(player.global_position.x - global_position.x)))

```

```

func set_facing(dir: int) -> void:
    if dir != 0:
        facing = dir
        sprite.flip_h = facing < 0  # sprites dessinés regardant vers la droite

func dx_to_player() -> float:
    return player.global_position.x - global_position.x if player else INF

func dy_to_player() -> float:
    return player.global_position.y - global_position.y if player else INF

func drop_down() -> void:
    ## Saute du balcon : ignore les plateformes traversables un instant.
    ## Un compteur plutôt qu'un await : rien ne reste en suspens si l'ennemi meurt entre-temps.
    set_collision_mask_value(2, false)
    _drop_left = 0.3

func kill() -> void:
    if _dead:
        return
    _dead = true
    Game.add_score(score_value)
    # Explosion et butin vont dans la scène courante, pas sous notre parent : un membre de
    # Formation a pour parent un PathFollow2D, qui emporterait l'explosion sur sa trajectoire.
    Fx.explosion(global_position + Vector2(0, -16))
    if not drops.is_empty() and randf() < drop_chance:
        var item: Node2D = drops.pick_random().instantiate()
        Fx.spawn(item, global_position + Vector2(0, -12), null, true)  # différé : pas d'ajout
                               physique pendant une collision
    killed.emit(self)
    queue_free()

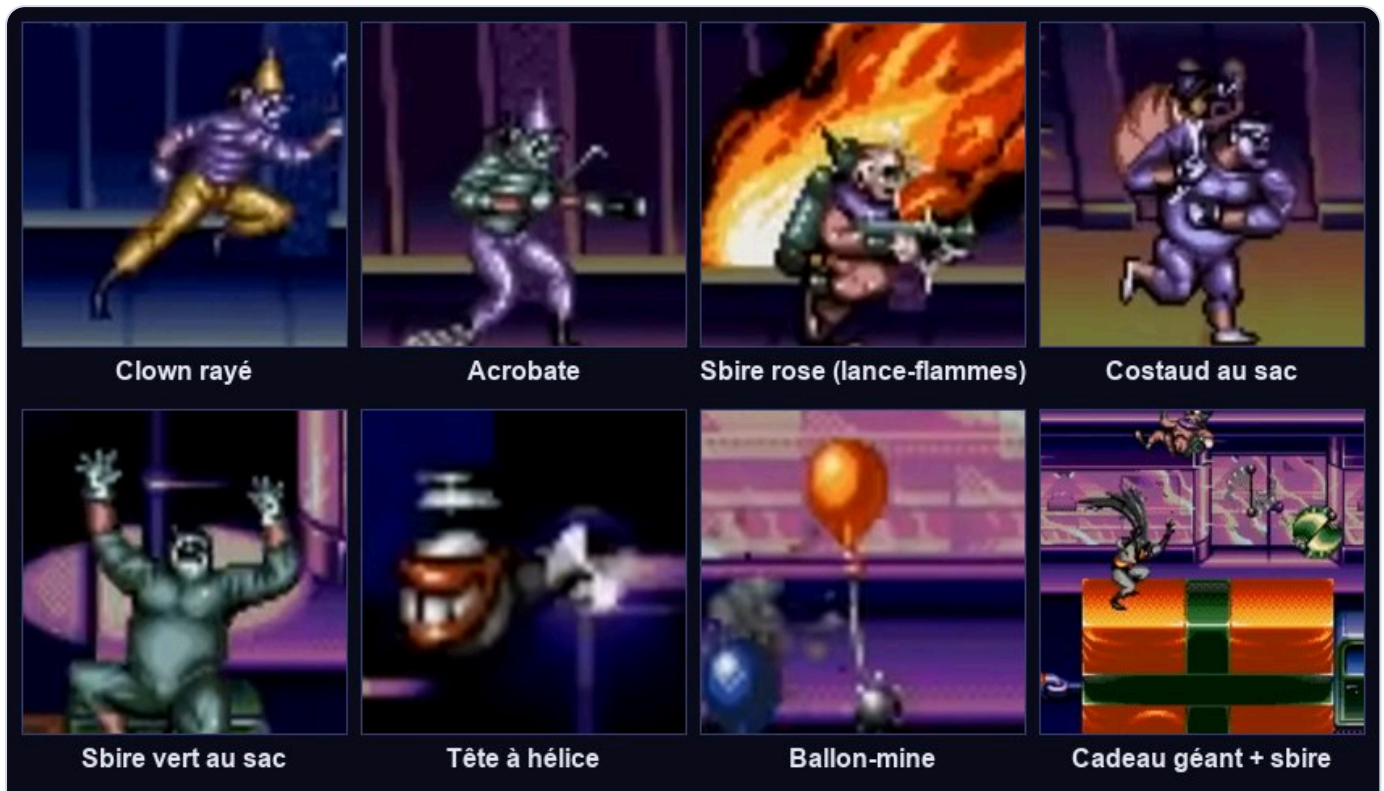
func _on_hurt(hitbox: Hitbox) -> void:
    sprite.self_modulate = Color(4, 4, 4)  # flash blanc rapide ; version palette au tome 06
    _flash_left = 0.06
    velocity.x = hitbox.get_knockback().x * knockback_scale

```

Comprendre le code

- `_physics_process` est écrit une fois pour tous : gravité, puis `think()` (le comportement propre), puis `move_and_slide()`, puis les minuteriers. Les classes filles ne réécrivent pas `_physics_process`.
- `health.died.connect(kill)` : la mort passe toujours par `kill()`, que l'ennemi meure de ses blessures ou par le crâne. `_dead` empêche de mourir deux fois (deux balles dans la même image).
- `drops` + `drop_chance` : une **table de butin** minimale. On règle dans l'Inspecteur ce que lâche un ennemi et avec quelle probabilité.
- `Color(4, 4, 4)` : une teinte supérieure à 1 éclaircit, d'où le flash blanc.
- `contact_hitbox.facing` : le recul dû au contact pousse Batman à l'opposé de l'ennemi, de quelque côté qu'il arrive.

LE BESTIAIRE



Le bestiaire du niveau 1, extrait de la vidéo. Chaque ennemi a son script, qui hérite d' `Enemy`.

7.4 · Le clown



Il arrive en tombant ou en marchant, s'approche, prépare son coup, frappe avec son aiguillon, recule et recommence. Sur le balcon, il saute quand Batman passe dessous.

res://enemies/clown.gd

```
class_name Clown
extends Enemy

enum S { ENTER, APPROACH, WINDUP, STRIKE, RECOVER }

@export var walk_speed := 55.0
@export var attack_range := 28.0
@export var windup_time := 0.25
@export var strike_time := 0.15
@export var recover_time := 0.45

var _s := S.ENTER
var _t := 0.0
@onready var poke: Hitbox = $PokeHitbox

func _ready() -> void:
    super._ready()
    poke.monitoring = false
```

```

func think(delta: float) -> void:
    _t += delta
    match _s:
        S.ENTER:
            velocity.x = 0.0
            if is_on_floor():
                _go(S.APPROACH)
        S.APPROACH:
            face_player()
            velocity.x = facing * walk_speed
            # Sur le balcon, Batman juste en dessous : on saute.
            if is_on_floor() and dy_to_player() > 40.0 and absf(dx_to_player()) < 48.0:
                drop_down()
            elif absf(dx_to_player()) < attack_range and absf(dy_to_player()) < 24.0:
                _go(S.WINDUP)
        S.WINDUP:
            velocity.x = 0.0
            if _t >= windup_time:
                poke.scale.x = facing
                poke.facing = facing
                poke.reset_hits()
                poke.monitoring = true
                _go(S.STRIKE)
        S.STRIKE:
            if _t >= strike_time:
                poke.monitoring = false
                _go(S.RECOVER)
        S.RECOVER:
            velocity.x = -facing * 20.0 # petit recul
            if _t >= recover_time:
                _go(S.APPROACH)
    play_anim(&"walk" if absf(velocity.x) > 1.0 else &"idle")

func _go(next: S) -> void:
    _s = next
    _t = 0.0

```

DEUX STYLES DE MACHINE À ÉTATS

Pour Batman, une machine **par nœuds** : dix états, beaucoup de logique. Pour un clown de cinq états de quelques lignes, un `enum` et un `match` suffisent et se lisent d'un coup d'œil. La phase `WINDUP` (préparation) est essentielle : c'est elle qui laisse au joueur le temps de voir venir le coup. C'est de la **télégraphie**.

À FAIRE

1. Nouvelle scène, racine `CharacterBody2D` nommée `Clown`, script `clown.gd`. Couche 4, masque 1 et 2. Enregistre en `res://enemies/clown.tscn`.
2. Construis cet arbre (le `Sprite` avec `placeholder_frames.tres`, scale (1, 2), offset y = -8, modulate violet) :

```
Clown (CharacterBody2D, clown.gd)  couche 4 « corps_ennemi » · masque 1+2
├─ Sprite (AnimatedSprite2D)      forme de substitution violette 16×32
├─ CollisionShape2D                rectangle 14×32, position (0, -16)
├─ Health                         max_health = 2 (À CALIBRER)
├─ Hurtbox                        couche 6 · masque vide · health = Health
│   └─ CollisionShape2D            rectangle 14×30, position (0, -16)
├─ ContactHitbox (Hitbox)         couche vide · masque 5 · damage 1 : toucher le corps fait mal
│   └─ CollisionShape2D            rectangle 12×28, position (0, -16)
└─ PokeHitbox (Hitbox)            couche vide · masque 5 · damage 1 · désactivée par défaut
    └─ CollisionShape2D            rectangle 16×8, position (14, -20)
```

3. Sur la racine, règle **Drops** : ajoute un élément, `weapon_pickup.tscn` ; **Drop Chance** = 0,3.
4. Places-en deux dans la salle de test : un au sol, un sur le balcon.

TESTE

- Les clowns marchent vers Batman, s'arrêtent, frappent, reculent.
- Passe sous le clown du balcon : il saute dans la rue.
- Deux tirs de batarang le tuent ; parfois il lâche un bonus.
- Le crâne de l'étape 06 détruit maintenant tous les clowns à l'écran.

SI ÇA NE MARCHE PAS

- « **Invalid get index 'hurt' (on base: 'null instance')** » : un nœud n'a pas le nom attendu par `enemy.gd` (`Sprite`, `Health`, `Hurtbox`).
- **Le clown traverse le sol** : son masque doit contenir la couche 1.
- **Le clown ne blesse pas Batman** : ses hitbox doivent regarder la couche 5, et la Hurtbox de Batman être sur la couche 5.

7.5 · Les vagues : `WaveData`



CONCEPT : LE « DIRECTEUR » DE RENCONTRES

Plutôt que de placer 300 ennemis à la main, on décrit des **vagues** : quels ennemis, combien en même temps au maximum, combien à abattre. Un directeur lit cette description et fait apparaître les ennemis au bon rythme. Le level design devient un réglage de données, et seuls les ennemis réellement à l'écran existent en mémoire. Beaucoup de jeux modernes vont plus loin, avec un directeur qui adapte la pression au joueur.

res://level/wave_data.gd

```
class_name WaveData
extends Resource
## Une vague : on fait apparaître des ennemis jusqu'à en avoir abattu « kill_quota ».

@export var enemies: Array[PackedScene] = []    ## pris dans l'ordre, en boucle
@export var kill_quota := 6
@export var max_alive := 3                      ## jamais plus de N à l'écran
@export var spawn_interval := 0.8
## Origines possibles : "top" (tombe du ciel), "left", "right", "balcony", "below".
@export var origins: PackedStringArray = ["top", "right", "left"]
@export var formation: NodePath                 ## facultatif : Formation à lancer avec la vague
```

7.6 · L'arène : WaveArena



res://level/wave_arena.gd

```
class_name WaveArena
extends Area2D
## Zone déclencheuse (masque 3). Quand Batman y entre : la caméra se fige,
## les vagues s'enchaînent, puis « GO! » et la caméra repart.

signal started
signal cleared

@export var waves: Array[WaveData] = []
@export var ground_y := 208.0
@export var balcony_y := 112.0
@export var go_sign_scene: PackedScene    ## res://level/go_sign.tscn

var _player: Player
var _index := -1
var _killed := 0
var _alive := 0
var _cooldown := 0.0
var _running := false
var _done := false

func _ready() -> void:
    body_entered.connect(_on_body_entered)

func _on_body_entered(body: Node2D) -> void:
    if body is Player and _player == null and not _done:
        _player = body
        # Différé : la première vague peut lancer une Formation, donc ajouter des corps
        # physiques. C'est interdit pendant un signal de collision comme body_entered.
        _begin.call_deferred()

func _begin() -> void:
    _running = true
    _player.camera.lock_on(global_position.x)
    started.emit()
    _next_wave()
```

```

func _next_wave() -> void:
    _index += 1
    if _index >= waves.size():
        _finish()
        return
    _killed = 0
    _cooldown = 0.6
    var w := waves[_index]
    if not w.formation.is_empty():
        var f := get_node(w.formation) as Formation
        f.start()

func _physics_process(delta: float) -> void:
    if not _running:
        return
    var w := waves[_index]
    if _killed >= w.kill_quota:
        if _alive == 0:
            _next_wave() # écran nettoyé : vague suivante
            return
    _cooldown -= delta
    if _cooldown <= 0.0 and _alive < w.max_alive and _killed + _alive < w.kill_quota:
        _spawn(w)
        _cooldown = w.spawn_interval

func _spawn(w: WaveData) -> void:
    if w.enemies.is_empty():
        return
    var scene := w.enemies[( _killed + _alive ) % w.enemies.size()]
    var e := scene.instantiate() as Enemy
    e.killed.connect(func(_e: Enemy) -> void: _killed += 1)
    e.tree_exiting.connect(func() -> void: _alive -= 1) # tué OU sorti de l'écran
    # Position fixée AVANT l'entrée dans l'arbre : le _ready() du ballon-mine la lit.
    Fx.spawn(e, _origin_point(w.origins[randi() % w.origins.size()]))
    _alive += 1

func _origin_point(origin: String) -> Vector2:
    var r := _player.camera.view_rect()
    match origin:
        "top":
            return Vector2(randf_range(r.position.x + 40, r.end.x - 40), r.position.y - 20)
        "left":
            return Vector2(r.position.x - 20, ground_y)
        "balcony":
            return Vector2(randf_range(r.position.x + 40, r.end.x - 40), balcony_y - 1)
        "below": # ballons-mines du convoi (tome 09)
            return Vector2(randf_range(r.position.x + 60, r.end.x + 20), r.end.y + 20)
        _:
            return Vector2(r.end.x + 20, ground_y)

func _finish() -> void:
    _running = false

```

```

_done = true
_player.camera.unlock()
if go_sign_scene:
    get_tree().current_scene.add_child(go_sign_scene.instantiate())
cleared.emit()

```

CE SCRIPT UTILISE **FORMATION**

`wave_arena.gd` mentionne le type `Formation`, créé en 7.11. En attendant, l'éditeur affiche une erreur « Could not find type "Formation" ». Deux options : créer dès maintenant `res://enemies/formation.gd` avec le code de 7.11, ou commenter provisoirement les deux lignes qui utilisent `Formation`. Je te conseille la première.

Comprendre le code

- **Deux compteurs** : `_killed` ne compte que les ennemis abattus (le quota) ; `_alive` compte ceux qui sont présents, quelle que soit la façon dont ils disparaissent. Sans `_alive`, un ennemi qui s'échappe de l'écran bloquerait l'arène pour toujours.
- `tree_exiting` : un signal émis par *tout* nœud qui quitte l'arbre. Une seule connexion couvre la mort, la sortie d'écran et le changement de scène.
- `_killed + _alive < w.kill_quota` : on n'ajoute pas d'ennemi si ceux déjà présents suffisent à finir le quota.
- `_begin.call_deferred()` : `body_entered` est un signal de collision. Si la première vague lance une formation, elle ajoute des corps physiques ; Godot l'interdit pendant ce signal (erreur « Can't change this state while flushing queries »). On reporte donc le démarrage à la fin de l'image.

7.7 · La bulle « GO! » et une première arène



res://level/go_sign.gd (racine CanvasLayer de go_sign.tscn)

```

extends CanvasLayer
## Bulle « GO! » au bord droit, qui clignote puis disparaît (vidéo 2:25, 7:49).
## Enfant : TextureRect « Bubble » ancré au centre-droit de l'écran.

@export var duration := 2.5
@onready var bubble: TextureRect = $Bubble
var _t := 0.0

func _process(delta: float) -> void:
    _t += delta
    bubble.visible = int(_t * 4.0) % 2 == 0 # clignote 2 fois par seconde
    if _t >= duration:
        queue_free()

```

À FAIRE

1. Nouvelle scène, racine `CanvasLayer` nommée `GoSign`, script `go_sign.gd`, enregistrée en `res://level/go_sign.tscn`.
2. Enfant `TextureRect` nommé `Bubble` : dans la barre d'outils, **Ancres** → **Centre droit**, taille 40×20. Si tu n'as pas encore d'image de bulle, ajoute-lui un enfant `Label` « GO! » (taille de police 12, jaune) : il clignotera avec son parent.
3. Dans la salle de test, retire les clowns posés à la main. Ajoute une `WaveArena` en (600, 112), couche vide, masque 3, avec une forme rectangle 8×224 : une ligne verticale que Batman franchira.
4. Dans l'Inspecteur de l'arène : **Go Sign Scene** = `go_sign.tscn`. **Waves** : ajoute deux éléments, chacun **Nouveau WaveData**.
 - Vague 1 : `enemies` = [clown.tscn], `kill_quota` = 4, `max_alive` = 2, `origins` = ["top", "right"].
 - Vague 2 : `enemies` = [clown.tscn], `kill_quota` = 6, `max_alive` = 3, `origins` par défaut.
5. Assure-toi que le sol de la salle s'étend bien au-delà de x = 800.

TESTE

Cours vers la droite. En passant x = 600, la caméra se recentre sur l'arène et se fige. Des clowns tombent du haut et arrivent par la droite, jamais plus de deux à la fois. Après 4 morts et un écran vide, la deuxième vague démarre (trois à la fois). À la fin, « GO! » clignote à droite et la caméra repart.

MÉTHODE : RÉGLER LE RYTHME AVEC LES DONNÉES

Joue l'arène trois fois en ne changeant que `max_alive` et `spawn_interval`. Avec 1 et 1,5 s, c'est une promenade ; avec 4 et 0,3 s, c'est l'enfer. Le même code donne des expériences très différentes : c'est le travail du level designer.

7.8 · Le costaud qui empoigne



```
res://enemies/thug.gd
```

```
class_name Thug
extends Enemy

## Charge, attrape Batman (état Grabbed du tome 03), le serre jusqu'à ce qu'il se dégage.
## Nœud enfant supplémentaire : GrabArea (Area2D, masque 3 « corps_joueur »).

enum S { APPROACH, CHARGE, HOLD, STUNNED }

@export var walk_speed := 40.0
@export var charge_speed := 150.0
@export var charge_trigger := 90.0
@export var squeeze_interval := 0.7
@export var squeeze_damage := 1

var _s := S.APPROACH
var _t := 0.0
@onready var grab_area: Area2D = $GrabArea

func think(delta: float) -> void:
```

```

_t += delta
match _s:
    S.APPROACH:
        face_player()
        velocity.x = facing * walk_speed
        if absf(dx_to_player()) < charge_trigger and absf(dy_to_player()) < 20.0:
            _go(S.CHARGE)
    S.CHARGE:
        velocity.x = facing * charge_speed
        if _try_grab():
            _go(S.HOLD)
        elif _t > 0.8:
            _go(S.STUNNED) # charge ratée : il s'essouffle
    S.HOLD:
        velocity.x = 0.0
        if not _holding():
            _go(S.STUNNED)
        elif _t >= squeeze_interval:
            _t = 0.0
            player.health.damage(squeeze_damage)
    S.STUNNED:
        velocity.x = 0.0
        if _t > 0.9:
            _go(S.APPROACH)

func _try_grab() -> bool:
    if player == null or player.hurtbox.invincible:
        return false
    if player in grab_area.get_overlapping_bodies():
        player.fsm.force(&"Grabbed", {"grabber": self})
        return true
    return false

func _holding() -> bool:
    return player != null and player.fsm.current_name() == &"Grabbed"

## Appelé par l'état Grabbed du joueur quand il se dégage.
func release() -> void:
    _go(S.STUNNED)

func _go(next: S) -> void:
    _s = next
    _t = 0.0

```

À FAIRE

1. Crée `res://enemies/thug.tscn` comme le clown, mais : pas de `PokeHitbox`, une forme plus large (20×34), Health 4, une couleur marron, et un enfant `GrabArea` (`Area2D`, couche vide, masque 3) avec un rectangle 26×30 en (0, -16).
2. Ajoute `thug.tscn` dans la liste `enemies` de la vague 2 : [clown, clown, thug].

TESTE

Le costaud s'approche, charge, et empoigne Batman (« Grabbed »). Il le serre toutes les 0,7 s. Martèle les boutons : Batman se libère avec un petit saut et quelques instants d'invincibilité ; le costaud reste étourdi. C'est la première fois que deux machines à états (celle de Batman, celle du costaud) se parlent :

`fsm.force()` dans un sens, `release()` dans l'autre.

7.9 · Le tireur et les balles ennemies



Les tirs ennemis réutilisent le projectile de l'étape 06 : seul le masque change.

res://enemies/gunner.gd

```
class_name Gunner
extends Enemy
## Se poste, vise, tire une rafale, attend. Variante : le sbire rose au lance-flammes
## (portée courte, grosse Hitbox au lieu de balles).

@export var burst := 3
@export var burst_gap := 0.15
@export var rest_time := 1.4
@export var bullet_speed := 140.0
@export var aim_diagonal := true ## vise en diagonale si Batman est plus haut ou plus bas

var _t := 0.0
var _shots_left := 0
var _pool: Pool

func _ready() -> void:
    super._ready()
    _pool = get_tree().get_first_node_in_group("enemy_bullets") as Pool
    _t = rest_time * randf()

func think(delta: float) -> void:
    velocity.x = 0.0
    face_player()
    _t -= delta
    if _t > 0.0:
        return
    if _shots_left == 0:
        _shots_left = burst
    _shoot()
    _shots_left -= 1
    _t = burst_gap if _shots_left > 0 else rest_time

func _shoot() -> void:
    if _pool == null or player == null:
        return
    var b := _pool.acquire() as Projectile
    if b == null:
        return
    var dir := Vector2(facing, 0.0)
    if aim_diagonal and absf(dy_to_player()) > 40.0:
```

```
dir.y = signf(dy_to_player())  
b.fire(global_position + Vector2(12 * facing, -22), dir, bullet_speed, 1)
```

À FAIRE

1. Ouvre `projectile.tscn` → ≡ **Scène** → **Enregistrer la scène sous** → `res://enemies/enemy_bullet.tscn`. Dans la copie : masque 5 seul (au lieu de 6), et une teinte orange sur le `Sprite2D`.
2. Dans la salle de test, ajoute un nœud `Pool` nommé `EnemyBullets`, avec **Scene** = `enemy_bullet.tscn`, et mets-le dans le groupe `enemy_bullets`. Chaque section en aura un.
3. Crée `res://enemies/gunner.tscn` sur le modèle du clown (sans `PokeHitbox`), couleur verte, Health 2.
4. Place un tireur sur le balcon de la salle de test.

TESTE

Le tireur se tourne vers Batman et tire des rafales de 3 balles. Depuis la rue, il tire en diagonale vers le bas. Accroupi, Batman esquive les balles horizontales (c'est la Hurtbox abaissée de l'étape 05 qui agit).

7.10 · La bombe



Une seule bombe pour les têtes à hélice, Harley et le Joker : elle tombe, rebondit un nombre réglable de fois, puis explose.

res://enemies/bomb.gd

```
class_name Bomb
extends Hitbox

@export var fall_gravity := 500.0  ## « gravity » est déjà une propriété native d'Area2D
@export var bounces := 1
@export var bounce_damping := 0.55
@export var fuse := 3.0
var velocity := Vector2.ZERO
@onready var ray: RayCast2D = $RayCast2D

func _ready() -> void:
    super._ready()
    single_hit = true
    landed.connect(func(_h: Hurtbox) -> void: explode())

func _physics_process(delta: float) -> void:
    velocity.y += fall_gravity * delta
    global_position += velocity * delta
    ray.force_raycast_update()
    if velocity.y > 0.0 and ray.is_colliding():
        global_position.y = ray.get_collision_point().y - 4.0
        if bounces > 0:
            bounces -= 1
            velocity.y = -velocity.y * bounce_damping
        else:
            explode()
    fuse -= delta
    if fuse <= 0.0:
        explode()

func explode() -> void:
    if is_queued_for_deletion():
        return
    Fx.explosion(global_position)
    queue_free()
```

À FAIRE

1. Nouvelle scène, racine `Area2D` nommée `Bomb`, script `bomb.gd`, couche vide, masque 5. Enregistre en `res://enemies/bomb.tscn`.
2. Enfants : `Sprite2D` (`icon.svg`, scale 0,06, teinte noire), `CollisionShape2D` (cercle de rayon 4), `RayCast2D` (**Target Position** = (0, 6), masque 1 et 2).

```
Bomb (Area2D, bomb.gd)    masque 5 · damage 1 · single_hit
├─ Sprite2D
├─ CollisionShape2D
└─ RayCast2D              cible (0, 6) · masque 1+2 : détecte le sol
```

RAYCAST2D

Un rayon qui part du nœud et s'arrête au premier obstacle de son masque. La bombe n'est pas un corps physique (c'est une zone) : elle ne « tombe » pas toute seule sur le sol. Le rayon, pointé vers le bas, lui dit où est le sol. `force_raycast_update()` recalcule le rayon tout de suite, après le déplacement de ce tick.

7.11 · Les formations de têtes à hélice



Formation : chaque tête suit la même Path2D avec un retard



Deux formations de têtes à hélice. Une colonne qui descend puis serpente, puis une ligne qui traverse le haut de l'écran. Toutes deux se dessinent avec une `Path2D`.

CONCEPT : LA TRAJECTOIRE DESSINÉE PLUTÔT QUE PROGRAMMÉE

Une `Path2D` se dessine à la souris dans l'éditeur. Chaque membre de la formation suit la même courbe avec un retard : on obtient une « chenille » sans aucune équation. Le level designer règle tout à l'œil.

res://enemies/formation.gd

```
class_name Formation
extends Path2D

## Fait défiler N ennemis le long de la courbe, chacun avec un retard.
## Dessine la courbe dans l'éditeur. Appelle start() (par exemple depuis une arène).

signal finished ## tous les membres sont morts ou sortis

@export var enemy_scene: PackedScene
@export var count := 6
@export var delay := 0.35 ## s entre deux membres
@export var duration := 6.0 ## s pour parcourir toute la courbe
@export var autostart := false

var _followers: Array[PathFollow2D] = []
var _t := 0.0
var _running := false

func _ready() -> void:
    if autostart:
        start()
```

```

func start() -> void:
    for i in count:
        var f := PathFollow2D.new()
        f.rotates = false
        f.loop = false
        add_child(f)
        var e: Node2D = enemy_scene.instantiate()
        f.add_child(e)
        f.visible = false
        _followers.append(f)
    _t = 0.0
    _running = true

func _physics_process(delta: float) -> void:
    if not _running:
        return
    _t += delta
    var alive := 0
    for i in _followers.size():
        var f := _followers[i]
        if f.get_child_count() == 0:
            continue # l'ennemi a été détruit
        alive += 1
        var local_t := _t - i * delay
        f.visible = local_t >= 0.0
        f.progress_ratio = clampf(local_t / duration, 0.0, 1.0)
        if local_t >= duration:
            f.get_child(0).queue_free() # sorti de l'écran : il s'échappe
    if alive == 0:
        _running = false
        finished.emit()

```

res://enemies/propeller_head.gd

```
class_name PropellerHead
extends Enemy
## Tête de clown volante. Le mouvement vient de la Formation ; elle se contente de bombarder.

@export var bomb_scene: PackedScene    ## res://enemies/bomb.tscn
@export var bomb_interval := Vector2(1.5, 3.5)
var _next := 0.0

func _ready() -> void:
    super._ready()
    uses_gravity = false
    _next = randf_range(bomb_interval.x, bomb_interval.y)

func think(delta: float) -> void:
    velocity = Vector2.ZERO
    if not is_visible_in_tree():
        return
    _next -= delta
    if _next <= 0.0 and bomb_scene:
        _next = randf_range(bomb_interval.x, bomb_interval.y)
        Fx.spawn(bomb_scene.instantiate() as Node2D, global_position + Vector2(0, 8))
    play_anim(&"fly")
```

À FAIRE

1. Crée `res://enemies/propeller_head.tscn` : racine `CharacterBody2D`, couche 4, **masque vide** (elle vole et ne bute sur rien), forme de 14×14, Health 1, Hurtbox et ContactHitbox, couleur jaune. **Bomb Scene** = `bomb.tscn`.
2. Dans la salle de test, ajoute un enfant de l'arène : `Path2D` nommé `Caterpillar`, script `formation.gd`. Sélectionne-le : la barre d'outils propose des outils de courbe. Avec l'outil « Ajouter un point », clique pour dessiner une courbe qui descend du haut de l'écran puis serpente vers la gauche. Les coordonnées des points sont relatives au `Path2D`.
3. **Enemy Scene** = `propeller_head.tscn`, **Count** = 8, **Delay** = 0,3.
4. Ajoute une troisième vague à l'arène : `kill_quota` = 0, `enemies` vide, et **Formation** = le chemin vers `Caterpillar` (bouton « Assigner »).

QUOTA 0 ET FORMATION

Avec `kill_quota = 0`, la vague se termine dès que `_alive` vaut 0. Les membres d'une formation ne sont pas comptés dans `_alive` : l'arène passe donc aussitôt à la suite pendant que la chenille vole. Si tu veux attendre la fin de la formation, connecte son signal `finished` à une méthode de l'arène qui appelle `_next_wave()`. C'est un bon exercice.

TESTE

À la troisième vague, une file de têtes jaunes suit ta courbe et lâche des bombes qui rebondissent puis explosent. Abattues, elles explosent **sur place** (sans suivre la trajectoire : c'est `Fx.explosion()` qui les place dans la scène courante).

7.12 · Le ballon-mine



res://enemies/balloon_mine.gd

```
class_name BalloonMine
extends Enemy
## Monte en oscillant. Explode si on le touche (ContactHitbox) ou si on le tire (Health 1).

@export var rise_speed := 22.0
@export var sway := 10.0
@export var sway_rate := 2.2
@export var colors: Array[Color] = [Color("e04b3a"), Color("43b05c"), Color("3f6fe0"),
    Color("f08a2c")]
var _t := 0.0
var _base_x := 0.0

func _ready() -> void:
    super._ready()
    uses_gravity = false
    _base_x = global_position.x
    _t = randf() * TAU
    sprite.self_modulate = colors.pick_random()
    $ContactHitbox.landed.connect(func(_h: Hurtbox) -> void: kill())

func think(delta: float) -> void:
    _t += delta
    var target_x := _base_x + sin(_t * sway_rate) * sway
    # Asservissement simple : la vitesse est proportionnelle à l'écart à la cible.
    velocity = Vector2((target_x - global_position.x) * 10.0, -rise_speed)
    if global_position.y < -300.0:
        queue_free() # parti dans le ciel
```

À FAIRE

1. Crée `res://enemies/balloon_mine.tscn` : couche 4, masque vide, Health 1, Hurtbox, ContactHitbox (masque 5), forme ronde de rayon 8. Garde la **Modulate** du `Sprite` blanche : la couleur aléatoire passe par `self_modulate`.
2. Teste-le en ajoutant une vague `[balloon_mine.tscn]` avec `origins = ["below"]`, `max_alive` 6, `kill_quota` 8.

CONCEPT : L'ASSERVISSEMENT

On ne téléporte pas le ballon sur sa courbe sinusoïdale : on lui donne une vitesse proportionnelle à l'écart entre où il est et où il devrait être. Il suit sa cible en douceur et reste un vrai corps physique. C'est le principe du régulateur proportionnel, utilisé partout (caméras, tourelles, IA).

REMARQUE

`kill()` donne des points même quand le ballon explose contre Batman. Pour l'éviter, ajoute à `kill()` un paramètre `award := true`.

7.13 · Les objets cassables



À gauche, une poubelle éclate et lâche un bonus ; à droite, un costaud empoigne Batman.

Un seul script pour tout ce qui se casse : poubelles (bonus), petits cadeaux (bonus), paquet géant (un sbire en jaillit).

res://level/breakable.gd

```
class_name Breakable
extends Node2D
## Objet destructible : Health + Hurtbox (couche 6). Lâche des bonus et/ou un ennemi.

@export var drops: Array[PackedScene] = []
@export var spawn_enemy: PackedScene
@export var score_value := 50
@export var broken_texture: Texture2D ## ex. vitrine brisée qui reste affichée

@onready var health: Health = $Health

func _ready() -> void:
    health.died.connect(_break)

func _break() -> void:
    Game.add_score(score_value)
    Fx.explosion(global_position)
    for scene in drops:
        _spawn(scene, Vector2(randf_range(-6, 6), -10))
    if spawn_enemy:
        _spawn(spawn_enemy, Vector2(0, -16))
    if broken_texture and has_node("Sprite2D"):
        $Sprite2D.texture = broken_texture
        $Hurtbox.set_deferred("monitorable", false)
    else:
        queue_free()

func _spawn(scene: PackedScene, offset: Vector2) -> void:
    # Différé : _break() est appelé pendant un signal de collision.
    Fx.spawn(scene.instantiate() as Node2D, global_position + offset, null, true)
```

À FAIRE

1. Crée `res://level/trash_can.tscn` : racine `Node2D` (script `breakable.gd`), enfants `Sprite2D` (forme grise 12×16), `Health` (max 1), `Hurtbox` (couche 6, health = Health) avec sa forme.
2. **Drops** = [weapon_pickup.tscn]. Places-en deux dans la salle de test, avant l'arène.

TESTE

Un tir sur la poubelle : elle explose et un bonus d'arme jaillit, puis se pose et change de couleur.

POINT DE CONTRÔLE

TU AS FINI L'ÉTAPE QUAND

- ☐ En entrant dans l'arène, la caméra se fige ; des clowns tombent du haut et arrivent par la droite, jamais plus que `max_alive` à la fois.
- ☐ Après le quota, la vague suivante démarre une fois l'écran vide, puis « GO! » clignote et la caméra repart.
- ☐ Un costaud attrape Batman ; marteler les boutons le libère.
- ☐ Le tireur tire des rafales ; les têtes à hélice suivent ta courbe et bombardent.
- ☐ Une poubelle détruite lâche un bonus d'arme ; le crâne détruit tout ce qui est à l'écran.

```
git git add -A && git commit -m "Étape 07 : ennemis, arènes à vagues, formations, cassables"
```

CE QUE TU RETIENS

? Pourquoi `Enemy` appelle-t-il `think()` au lieu de laisser chaque ennemi écrire son `_physics_process` ?

Pour que le socle commun (gravité, déplacement, flash, minuteries) soit écrit une fois et appliqué dans le bon ordre. Les classes filles ne fournissent que ce qui les distingue. Si on corrige la gravité, tous les ennemis en profitent.

? Que se passerait-il sans le compteur `_alive` ?

L'arène compterait seulement les ennemis abattus. Un ennemi qui sort de l'écran sans mourir (une tête à hélice en fin de trajectoire) ne serait jamais « fini », et la vague suivante ne démarrerait jamais.

? À quoi sert la phase `WINDUP` du clown ?

À télégraphier l'attaque : un court délai visible avant le coup laisse au joueur le temps de réagir (reculer, sauter, frapper le premier). Sans elle, les coups paraissent injustes.

? Pourquoi l'arène démarre-t-elle en différé ?

Parce que `body_entered` est émis pendant le traitement des collisions, et qu'ajouter des corps physiques à ce moment-là est interdit. Si la première vague lance une formation, ses ennemis seraient ajoutés pendant ce signal. `call_deferred` reporte le démarrage à la fin de l'image.

POUR ALLER PLUS LOIN

- Connecte `Formation.finished` pour que l'arène attende la fin de la chenille.
- Crée un clown « rapide » : une scène héritée de `clown.tscn` (clic droit → **Nouvelle scène héritée**) avec `walk_speed` à 90 et une autre couleur.
- Crée le sbire au lance-flammes : un `Gunner` sans balles, avec une grande `Hitbox` activée pendant 0,6 s devant lui.

EFFETS MEGA DRIVE

On donne au jeu son grain de 1995 : des fondus en 8 paliers, une façade dont chaque ligne défile à sa propre vitesse, un feu qui ondule, des flashes de palette. Tu vas écrire tes premiers shaders, et comprendre comment la console faisait « l'impossible ».

~4 h

Prérequis : étape 07

Guide : [tome 06](#)

CE QUE TU VAS APPRENDRE

GODOT 4

- Shaders `canvas_item` : `fragment()`, `vertex()`, uniformes, `varying`
- `ShaderMaterial` et paramètres depuis le code
- Autoload d'une **scène**
- `Curve`, `Image`, `Parallax2D`

DÉVELOPPEMENT DE JEU

- Séparer le calcul et l'affichage
- Les effets raster (par ligne)
- Couleurs indexées et palettes
- Séparer le visuel de la collision
- La parallaxe et la pseudo-3D par l'échelle

PREMIER CONTACT AVEC LES SHADERS

CONCEPT : QU'EST-CE QU'UN SHADER ?

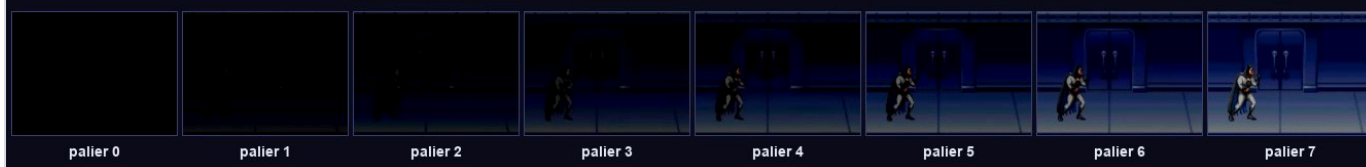
Un shader est un petit programme exécuté par la carte graphique **pour chaque pixel** d'un objet (la fonction `fragment()`), des millions de fois par seconde et en parallèle. Il reçoit la position du pixel (`UV`, de 0 à 1 sur l'objet) et doit écrire sa couleur (`COLOR`). Les paramètres réglables depuis l'extérieur sont des `uniform`. Le langage de Godot ressemble au GLSL.

FONDUS ET FLASHS

8.1 - `PaletteFx` : le post-traitement 9 bits



Fondu d'entrée mesuré (1:46) : 8 paliers × 3 images = 0,4 s · le bleu monte en premier



Un fondu Mega Drive, mesuré. 8 paliers de 3 images chacun (0,4 s). Chaque canal de couleur descend indépendamment, d'où la teinte qui « tourne ».

SUR MEGA DRIVE

Chaque couleur de la console tient sur 9 bits : 3 bits par canal, donc **8 niveaux** de rouge, de vert et de bleu. Un fondu au noir retire un niveau à chaque canal toutes les quelques images : 8 paliers visibles. Un fondu Godot lisse (`modulate` animé) n'a pas ce grain. On le recrée avec un post-traitement plein écran.

```
res://fx/palette_fx.gdshader
```

```
shader_type canvas_item;
// Post-traitement plein écran façon CRAM : couleurs 9 bits, fondus et flashes par paliers.

uniform sampler2D screen : hint_screen_texture, filter_nearest;
uniform int fade = 0;           // 0..7 : paliers retirés à chaque canal (vers le noir)
uniform int flash = 0;          // 0..7 : paliers ajoutés à chaque canal (vers le blanc)
uniform bool quantize = true;   // forcer les 8 niveaux par canal de la Mega Drive

void fragment() {
    vec3 c = texture(screen, SCREEN_UV).rgb;
    vec3 lv = quantize ? floor(c * 7.0 + 0.5) : c * 7.0; // niveaux 0..7
    lv = max(lv - float(fade), 0.0);                     // chaque canal descend séparément
    lv = min(lv + float(flash), 7.0);
    COLOR = vec4(lv / 7.0, 1.0);
}
```

```
res://fx/palette_fx.gd (Autoload « PaletteFx », scène palette_fx.tscn)
```

```
extends CanvasLayer
## palette_fx.tscn : CanvasLayer > ColorRect « Rect » (ancrage plein écran, matériau palette_fx).
## Ajoute la SCÈNE en autoload sous le nom PaletteFx.

@onready var rect: ColorRect = $Rect
var _mat: ShaderMaterial
var _fade := 0 ## palier courant : un fondu repart toujours de là où on en est

func _ready() -> void:
    layer = 100
    process_mode = Node.PROCESS_MODE_ALWAYS
    rect.mouse_filter = Control.MOUSE_FILTER_IGNORE
    _mat = rect.material as ShaderMaterial

func set_fade(steps: int) -> void:
    _fade = clampi(steps, 0, 7)
    _mat.set_shader_parameter("fade", _fade)

func set_flash(steps: int) -> void:
    _mat.set_shader_parameter("flash", clampi(steps, 0, 7))

## Fondu au noir en 8 paliers. Mesuré dans la vidéo (1:46) : 3 images par palier, soit 0,4 s.
func fade_out(frames_per_step := 3) -> void:
    for s in range(_fade, 8):
        set_fade(s)
        await _frames(frames_per_step)
```

```
func fade_in(frames_per_step := 3) -> void:
    for s in range(_fade, -1, -1):
        set_fade(s)
        await _frames(frames_per_step)
```

Flash blanc : le crâne (2 images), chaque flash de la banque (3 images).

```
func flash_white(frames := 2, steps := 7) -> void:
    set_flash(steps)
    await _frames(frames)
    set_flash(0)
```

```
func _frames(n: int) -> void:
    for i in n:
        await get_tree().physics_frame
```

À FAIRE

1. Crée le shader : clic droit sur `res://fx/` → **Nouveau** → **Ressource** → **Shader** (ou **Nouveau shader**), mode `canvas_item`, nom `palette_fx.gdshader`, et tape le code.
2. Nouvelle scène, racine `CanvasLayer` nommée `PaletteFx`, script `palette_fx.gd`. Enfant `ColorRect` nommé `Rect` : barre d'outils **Ancres** → **Rect complet** ; Inspecteur **Material** → **Nouveau ShaderMaterial**, puis **Shader** = `palette_fx.gdshader`. Enregistre en `res://fx/palette_fx.tscn`.
3. **≡** **Projet** → **Paramètres du projet** → **Globaux** → **Autoload** : ajoute la **scène** `res://fx/palette_fx.tscn`, nom `PaletteFx`, juste après `Game`.
4. Dans `test_room.gd`, ajoute ces deux cas dans le `match event.keycode` :

`res://level/test_room.gd` (ajout dans le match)

```
KEY_F:
    await PaletteFx.fade_out()
    await PaletteFx.fade_in()
KEY_G:
    PaletteFx.flash_white()
```

TESTE

F : l'écran s'assombrit en 8 paliers bien visibles (0,4 s), avec une teinte qui change en route, puis revient. **G** : flash blanc de 2 images. Les couleurs de tout le jeu sont maintenant limitées à 8 niveaux par canal : les dégradés deviennent des aplats, comme sur la console.

Comprendre le code

- `hint_screen_texture` : le shader lit ce qui est déjà dessiné à l'écran sous lui. Comme le `CanvasLayer` est sur la couche 100, au-dessus de tout, il voit l'image entière et la retouche.
- `floor(c * 7.0 + 0.5)` : ramène chaque canal (0 à 1) à un niveau entier de 0 à 7. Le fondu retire `fade` niveaux à chaque canal : un canal à 2 atteint 0 avant un canal à 7, d'où la teinte qui tourne.
- Un autoload peut être une **scène**, avec ses nœuds enfants : pratique pour un effet qui a besoin d'un `ColorRect` et d'un matériau.
- `process_mode = ALWAYS` : les fondus marcheront même quand le jeu sera en pause.

- `await get_tree().physics_frame` : attend un tick. Compter des ticks plutôt que des secondes donne des paliers d'exactement 3 images, comme sur la console.

8.2 · Brancher le crâne sur `PaletteFx`



Le flash du crâne (étape 06) utilisait un rectangle blanc. Remplace le corps de `Fx.flash_screen()` pour passer par la palette :

```
res://fx/fx.gd (nouvelle version de flash_screen)
```

```
## Flash plein écran. Délègue à PaletteFx : un flash de palette, comme sur la console.  
## (Les paramètres tree et color sont gardés pour ne pas modifier les appels existants.)  
static func flash_screen(_tree: SceneTree, _color := Color.WHITE, duration := 2.0 / 60.0) -> void:  
    PaletteFx.flash_white(roundi(duration * 60.0))
```

MÉTHODE : CHANGER L'IMPLÉMENTATION, GARDER L'INTERFACE

Tous ceux qui appellent `Fx.flash_screen()` continuent de fonctionner sans modification : seul l'intérieur de la fonction a changé. C'est l'intérêt de passer par une fonction plutôt que de dupliquer le code du flash partout.

LE PLAN RASTER

8.3 · Le principe



SUR MEGA DRIVE

L'écran est dessiné ligne par ligne. Entre deux lignes, le processeur vidéo lit une **table H-Scroll** qui contient un décalage horizontal par ligne. À chaque image, le programme calcule une table de 224 valeurs, et le matériel l'applique gratuitement. Décaler chaque ligne d'une quantité différente donne du relief à une image plate : c'est le « secret #1 » de Jon Burton.



La façade pseudo-3D, mesurée. Chaque bande défile à sa propre vitesse ; le trottoir, où marchent les personnages, vaut 1.

CONCEPT : SÉPARER LE CALCUL ET L'AFFICHAGE

On reproduit exactement la division du travail de la console. Un script GDScript joue le rôle du processeur : à chaque image, il remplit deux tableaux de 224 valeurs (`h_scroll` : décalage de chaque ligne ; `v_line` : quelle ligne de l'image source afficher). Un shader joue le rôle du processeur vidéo : pour chaque pixel, il lit la valeur de sa ligne. Un seul shader suffira pour la façade, l'intro en perspective ou d'autres effets : seules les tables changeront.

```
res://fx/raster_plane.gdshader
```

```
shader_type canvas_item;
// Émulation d'un plan du VDP piloté par une table par ligne.
// Ce shader s'applique à un ColorRect de 320x224 placé dans un CanvasLayer :
// UV.y correspond donc exactement à une ligne d'écran.

uniform sampler2D plane : repeat_enable, filter_nearest;
uniform float h_scroll[224]; // décalage horizontal de chaque ligne, en pixels
uniform float v_line[224];   // ligne de la texture source affichée sur chaque ligne d'écran
uniform vec2 screen_size = vec2(320.0, 224.0);

void fragment() {
    vec2 px = floor(UV * screen_size); // pixel d'écran (x, numéro de ligne)
    int line = clamp(int(px.y), 0, 223);
    vec2 tex_size = vec2(textureSize(plane, 0));
    vec2 src = vec2(px.x + h_scroll[line], v_line[line]) + 0.5; // centre du texel
    COLOR = texture(plane, src / tex_size); // repeat_enable : la texture boucle
}
```

```

class_name RasterPlane
extends ColorRect
## « 68000 » d'un plan raster : remplit les tables à chaque image, le shader les applique.
## À placer dans un CanvasLayer (layer négatif pour être derrière le monde).

const LINES := 224

@export var plane_texture: Texture2D
var h_scroll := PackedFloat32Array()
var v_line := PackedFloat32Array()
var _mat: ShaderMaterial

func _ready() -> void:
    h_scroll.resize(LINES)
    v_line.resize(LINES)
    for y in LINES:
        v_line[y] = y
    position = Vector2.ZERO
    size = Vector2(320, LINES)
    mouse_filter = Control.MOUSE_FILTER_IGNORE
    _mat = (material as ShaderMaterial).duplicate() # chaque plan a ses propres tables
    material = _mat
    _mat.set_shader_parameter("plane", plane_texture)

func _process(_delta: float) -> void:
    var cam := get_viewport().get_camera_2d()
    if cam:
        compute(cam.get_screen_center_position())
    _mat.set_shader_parameter("h_scroll", h_scroll)
    _mat.set_shader_parameter("v_line", v_line)

## À surcharger : remplir h_scroll et v_line pour l'image courante.
func compute(_camera_center: Vector2) -> void:
    pass

```

Comprendre le code

- `uniform float h_scroll[224]` : un tableau de paramètres. Le script lui envoie un `PackedFloat32Array` de la même taille à chaque image (448 nombres en tout : rien pour une carte graphique).
- `floor(UV * screen_size)` : `UV` va de 0 à 1 sur le rectangle ; multiplié par 320×224, il donne le pixel d'écran. La partie entière de `y` est le numéro de ligne.
- `repeat_enable` : quand on lit au-delà du bord de la texture, on retombe au début. La façade boucle à l'infini sans calcul.
- `duplicate()` : un matériau est une ressource partagée. Sans copie, deux plans raster partageraient les mêmes tables.
- Le placement dans un `CanvasLayer` rend le rectangle fixe à l'écran : il ne défile pas avec la caméra. C'est la table qui simule le défilement.

8.4 · Une texture de test générée par script



Pour régler une façade, une image de test vaut mieux que le vrai décor : des bandes de couleur montrent la vitesse de chaque ligne, et des lignes repères indiquent le balcon et le trottoir. Plutôt que de la dessiner, on la fait générer par un script d'éditeur.

```
res://tools/make_debug_facade.gd
```

```
@tool
extends EditorScript

## Génère une façade de test 640x224 : bandes verticales de 32 px (6 teintes x 2 nuances),
## ligne blanche au niveau du balcon (y = 108), ligne jaune au niveau du trottoir (y = 200).
## 14 couleurs en tout : l'image reste convertible en couleurs indexées (8.7).

const PATH := "res://art/s1/facade_debug.png"

func _run() -> void:
    var img := Image.create_empty(640, 224, false, Image.FORMAT_RGBA8)
    for x in 640:
        var band := int(x / 32.0)
        var hue := Color.from_hsv((band % 6) / 6.0, 0.5, 0.6)
        for y in 224:
            var k := 0.7 if int(y / 16.0) % 2 == 0 else 1.0 # rangées alternées
            img.set_pixel(x, y, Color(hue.r * k, hue.g * k, hue.b * k))
    for x in 640:
        img.set_pixel(x, 108, Color.WHITE)
        img.set_pixel(x, 200, Color.YELLOW)
    DirAccess.make_dir_recursive_absolute(PATH.get_base_dir())
    img.save_png(PATH)
    EditorInterface.get_resource_filesystem().scan() # l'éditeur importe le nouveau fichier
    print("Façade de test écrite : ", PATH)
```

À FAIRE

1. Crée ce script dans `res://tools/`, puis **Fichier → Exécuter**.
2. `res://art/s1/facade_debug.png` apparaît dans le Système de fichiers. Double-clique dessus pour la voir.

8.5 · La façade pseudo-3D



res://fx/facade_raster.gd

```
class_name FacadeRaster
extends RasterPlane
## Secret #1 : chaque ligne de la façade défile à sa propre vitesse.
## Les lignes où l'on marche (trottoir, dessus du balcon) DOIVENT valoir 1,0,
## sinon les sprites glisseraient sur le décor.

@export var depth: Curve          ## x : ligne d'écran (0 = haut, 1 = bas) ; y : facteur de vitesse
@export var world_origin := Vector2(0, 0) ## position monde du coin haut-gauche de la texture

func compute(camera_center: Vector2) -> void:
    var view_left := camera_center.x - 160.0
    var view_top := camera_center.y - 112.0
    for y in LINES:
        var k := depth.sample(float(y) / (LINES - 1)) if depth else 1.0
        # floor : le VDP ne connaît que des pixels entiers. Chaque ligne avance à son rythme.
        h_scroll[y] = floor((view_left - world_origin.x) * k)
        v_line[y] = y + floor(view_top - world_origin.y)
```

À FAIRE

1. Dans `test_room.tscn`, ajoute un `CanvasLayer` nommé `Background`, **Layer** = -10 (derrière le monde).
2. Enfant `ColorRect` nommé `Facade`, script `facade_raster.gd`. **Material** → **Nouveau ShaderMaterial**, shader `raster_plane.gdshader`. **Plane Texture** = `facade_debug.png`.
3. **Depth** → **Nouveau Curve**, puis clique sur la courbe pour l'éditer. Dans l'Inspecteur de la courbe : **Max Value** = 1,2 ; **Point Count** = 9 ; renseigne la **Position** de chaque point, et règle **Left Mode** et **Right Mode** à `Linear` :

Point	0	1	2	3	4	5	6	7	8
x (ligne / 223)	0,14	0,44	0,47	0,52	0,72	0,73	0,78	0,89	1,0
y (facteur)	1,06	1,06	1,0	1,0	0,95	0,91	0,91	1,0	1,0

4. Les `ColorRect` gris et bruns du sol et du balcon cachent la façade : rends-les semi-transparents (couleur avec alpha 0,4), pour voir la façade et vérifier l'alignement.
5. `F6` et cours.

TESTE

En courant, le haut de la façade (lignes 32–98) défile un peu plus vite que le sol, la vitrine (lignes 163–174) un peu moins vite. La ligne blanche (balcon) et la ligne jaune (trottoir) restent parfaitement calées sur les rectangles de collision : Batman marche dessus sans glisser. L'image plate a pris du relief.

MESURE CONTRE RÈGLE DE JEU

La vidéo mesure 1,06 pour toute la bande 32–114, qui contient le balcon. Mais Batman **marche** sur le balcon (ligne 108 $\approx$ 0,48) : avec 1,06, il glisserait sur le décor. La courbe force donc 1,0 sur la tranche du balcon (points 2 et 3). Quand une mesure contredit une règle de jeu, c'est la règle de jeu qui gagne : elle se voit à chaque pas.

CONCEPT : SÉPARER LE VISUEL DE LA COLLISION

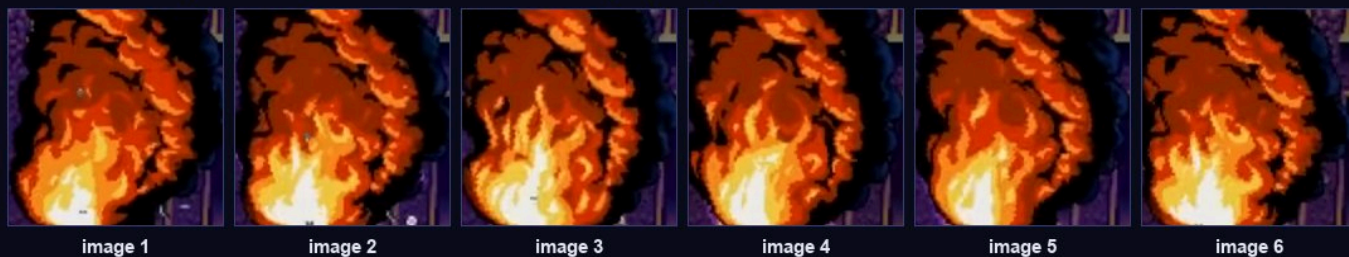
La façade est une image affichée par un shader ; elle ne sait rien de la physique. Le trottoir et le balcon sur lesquels Batman marche sont des `StaticBody2D` invisibles alignés avec elle. Le joueur ne marche jamais « sur l'image ». Presque tous les jeux 2D fonctionnent ainsi.

LE FEU ONDULANT

8.6 · Le shader du feu



Feu de la banque : 6 images d'animation + décalage par ligne



Les 6 images d'animation du feu de la banque. Burton : « une animation de quatre ou cinq images... je compte : six. » Puis un décalage horizontal par ligne : peu en bas, beaucoup en haut.

D'abord, une bande de 6 flammes de test, générée elle aussi :

```
res://tools/make_debug_fire.gd
```

```
@tool
extends EditorScript

## Génère une bande de 6 flammes de 16x32 (image de 96x32) pour tester le shader du feu.

const PATH := "res://art/s1/fire_debug.png"

func _run() -> void:
    var img := Image.create_empty(96, 32, false, Image.FORMAT_RGBA8)
    for f in 6:
        for y in 32:
            var t := y / 31.0
            var half := 1.0 + t * 6.0 + sin(f * 1.05 + y * 0.45) * 1.5
            var col := Color(1.0, 0.25 + 0.6 * t, 0.1 * t)
            # 0 en haut, 1 en bas
            # demi-largeur de la flamme
            # rouge en haut, jaune en bas
            for x in 16:
                if absf(x - 7.5) <= half:
                    img.set_pixel(f * 16 + x, y, col)
    DirAccess.make_dir_recursive_absolute(PATH.get_base_dir())
    img.save_png(PATH)
    EditorInterface.get_resource_filesystem().scan()
    print("Bande de flammes écrite : ", PATH)
```

```
res://fx/billowing_fire.gdshader
```

```
shader_type canvas_item;
// Pour un Sprite2D avec hframes = 6 (les 6 images de flamme côte à côte).
// Chaque LIGNE entière du sprite est décalée : peu en bas, beaucoup en haut.

uniform float frames = 6.0;           // = hframes du Sprite2D
uniform float amplitude = 5.0;         // décalage maximal en pixels (au sommet)
uniform float wavelength = 22.0;       // nombre de lignes pour une période de l'onde
uniform float speed = 5.0;
uniform float power = 1.6;             // forme de la montée d'amplitude du bas vers le haut
uniform float time_offset = 0.0;       // pour désynchroniser plusieurs foyers

// Dans fragment(), COLOR vaut déjà « texture x modulate ». On garde donc la seule
// modulation (modulate, self_modulate) dans une varying calculée dans vertex().
varying vec4 tint;

void vertex() {
    tint = COLOR;
}

void fragment() {
    vec2 tex_px = 1.0 / TEXTURE_PIXEL_SIZE;
    float line = floor(UV.y * tex_px.y);           // ligne entière, comme le VDP
    float height01 = 1.0 - line / tex_px.y;         // 0 en bas, 1 en haut
    float wave = sin((TIME + time_offset) * speed + line * 6.2831853 / wavelength);
    float shift_px = floor(wave * amplitude * pow(height01, power) + 0.5);

    // On reste dans l'image courante de la planche (sinon on « mord » sur la voisine).
    float frame_w = 1.0 / frames;
    float frame_start = floor(UV.x / frame_w) * frame_w;
    float u = UV.x + shift_px * TEXTURE_PIXEL_SIZE.x;
    if (u < frame_start || u >= frame_start + frame_w) {
        COLOR = vec4(0.0);
    } else {
        COLOR = texture(TEXTURE, vec2(u, UV.y)) * tint;
    }
}
```

```
res://fx/fire.gd (sur le Sprite2D, hframes = 6)
```

```
extends Sprite2D
## Fait tourner les 6 images de flamme ; le shader ajoute l'ondulation par ligne.

@export var fps := 12.0
var _t := 0.0

func _ready() -> void:
    _t = randf() * 10.0
    (material as ShaderMaterial).set_shader_parameter("time_offset", randf() * 10.0)

func _process(delta: float) -> void:
    _t += delta
    frame = int(_t * fps) % hframes
```

À FAIRE

1. Exécute `make_debug_fire.gd`.
2. Dans la salle de test, ajoute un `Sprite2D` nommé `Fire` en (300, 150) : **Texture** = `fire_debug.png`, **Animation** → **Hframes** = 6, script `fire.gd`. **Material** → **Nouveau ShaderMaterial**, shader `billowing_fire.gdshader`, et coche **Resource** → **Local To Scene** sur le matériau.
3. Duplique le foyer (`Ctrl D`) deux fois et décale les copies.

TESTE

Les flammes changent d'image 12 fois par seconde et ondulent : presque immobiles en bas, très mobiles en haut. Les trois foyers ne sont pas synchronisés. Pendant que le jeu tourne, sélectionne un foyer dans l'arbre Distant et fais varier `amplitude` et `power` dans le matériau.

PIÈGE GODOT 4 : `COLOR` DANS `FRAGMENT()`

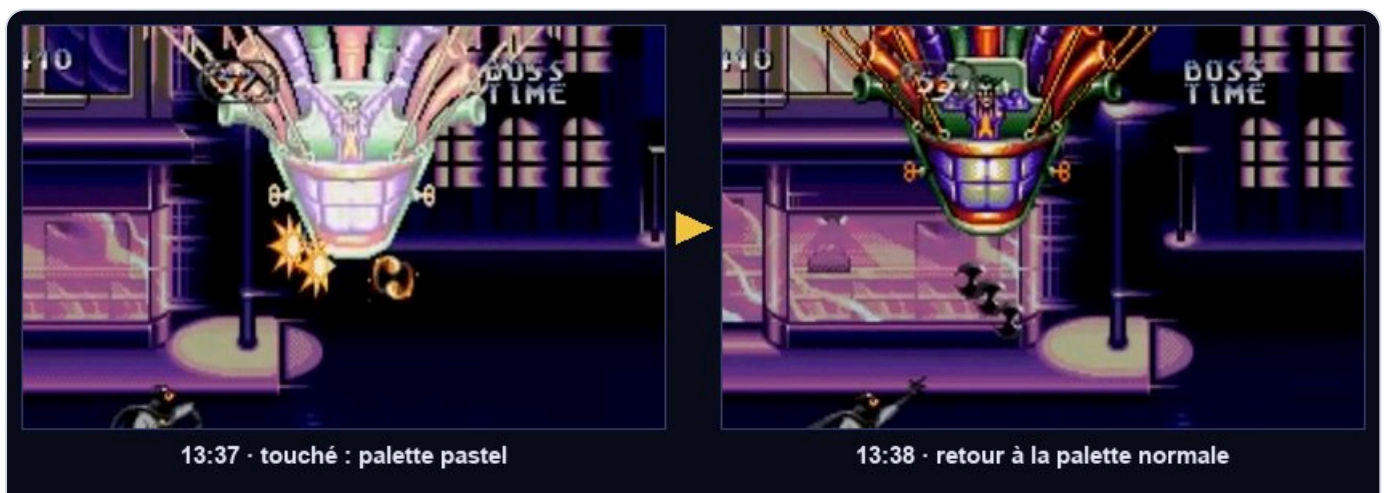
En entrée de `fragment()`, `COLOR` vaut **déjà la couleur de la texture multipliée par la teinte**. Écrire `texture(TEXTURE, uv) * COLOR` multiplierait la texture par elle-même (et ici par sa version non déformée). On recopie donc `COLOR` dans une `varying` depuis `vertex()`, où il ne contient que la teinte. Retiens ce motif : tu le retrouveras dans les deux shaders suivants.

LOCAL TO SCENE

Sans cette case, les trois foyers partageraient le même matériau, donc le même `time_offset` : ils onduleraient à l'unisson. « Local To Scene » donne à chaque instance sa propre copie de la ressource.

LE FLASH DE DÉGÂTS

8.7 - HitFlash



Le flash de dégâts du Joker. Toute la montgolfière bascule vers une palette pastel pendant quelques images.

res://fx/hit_flash.gdshader

```
shader_type canvas_item;
// Teinte un sprite vers une couleur claire, comme une palette « touchée » de la CRAM.

uniform float flash : hint_range(0.0, 1.0) = 0.0;
uniform vec4 flash_color : source_color = vec4(1.0, 0.95, 0.92, 1.0);

varying vec4 tint; // modulate seul (voir billowing_fire.gdshader)

void vertex() {
    tint = COLOR;
}

void fragment() {
    vec4 c = texture(TEXTURE, UV) * tint;
    c.rgb = mix(c.rgb, flash_color.rgb, flash);
    COLOR = c;
}
```

res://fx/hit_flash.gd

```
class_name HitFlash
extends Node
## Branche une Hurtbox sur le shader hit_flash d'un sprite.
## Joker : amount ≈ 0,55 pour le rendu pastel (vidéo 13:38). Ennemis : 1,0.

@export var target: CanvasItem ## le sprite (matériau hit_flash, « Local To Scene » coché)
@export var hurtbox: Hurtbox
@export var amount := 1.0
@export var duration := 0.12
var _t := 0.0
var _mat: ShaderMaterial

func _ready() -> void:
    _mat = target.material as ShaderMaterial
    hurtbox.hurt.connect(func(_h: Hitbox) -> void: _t = duration)

func _process(delta: float) -> void:
    if _t <= 0.0:
        return
    _t -= delta
    # Clignote une image sur deux pendant la durée.
    var on := _t > 0.0 and int(_t * 30.0) % 2 == 0
    _mat.set_shader_parameter("flash", amount if on else 0.0)
```

À FAIRE : TESTER SUR LA CIBLE D'ENTRAÎNEMENT

1. Ouvre `sandbox/dummy.tscn`. Sur le `ColorRect` : **Material** → **Nouveau ShaderMaterial**, shader `hit_flash.gdshader`, **Local To Scene** coché.
2. Ajoute un nœud `HitFlash` : **Target** = `ColorRect`, **Hurtbox** = `Hurtbox`, **Amount** = 0,55.

TESTE

Chaque coup sur la cible la fait clignoter vers le pastel pendant 0,12 s. Un `ColorRect` n'a pas de texture : Godot lui fournit une texture blanche, et `tint` contient sa couleur. Le même nœud servira aux boss.

COMPOSITION, ENCORE

`HitFlash` ne sait pas à qui il appartient : il relie une `Hurtbox` à un sprite. On l'ajoute à n'importe quel objet touchable, sans modifier son script. C'est un composant de plus dans ta boîte.

COULEURS INDEXÉES (FACULTATIF)

8.8 · Changer l'ambiance sans redessiner



Bascule de palette : mêmes tuiles, autres couleurs



2:42 · palette « nuit »



2:57 · même rue, palette « incendie »

La même rue avant et après l'explosion. Les formes sont identiques, seules les couleurs changent : un changement de palette, pas un nouveau décor.

CONCEPT : SÉPARER LA FORME ET LA COULEUR

Une image indexée stocke pour chaque pixel un **numéro de couleur**, pas une couleur. Une palette, à part, dit quelle couleur correspond à chaque numéro. On change l'ambiance d'un décor entier (nuit → incendie) en changeant 16 couleurs, sans redessiner un pixel. L'étape 09 propose une version simple de la bascule de la banque (échange de texture) ; cette section est la version fidèle, à faire si tu es curieux.

```
@tool
```

```
extends EditorScript
```

```
## Convertit une image (16 couleurs max) en carte d'index + bande de palette 16x1.
```

```
## Fichier > Exécuter. Résultat : *_idx.png et *_pal.png à côté de la source.
```

```
const SOURCE := "res://art/s1/facade_debug.png"
```

```
func _run() -> void:
```

```
    var img := Image.load_from_file(SOURCE)
```

```
    img.convert(Image.FORMAT_RGBA8)
```

```
    var palette: Array[Color] = [Color(0, 0, 0, 0)] # index 0 = transparent, comme la CRAM
```

```
    var idx := Image.create_empty(img.get_width(), img.get_height(), false, Image.FORMAT_RGBA8)
```

```
    for y in img.get_height():
```

```
        for x in img.get_width():
```

```
            var c := img.get_pixel(x, y)
```

```
            var i := 0
```

```
            if c.a >= 0.5:
```

```
                c.a = 1.0
```

```
                i = palette.find(c)
```

```
                if i == -1:
```

```
                    if palette.size() >= 16:
```

```
                        push_error("Plus de 16 couleurs : découpe l'image ou réduis sa palette.")
```

```
                        return
```

```
                    palette.append(c)
```

```
                    i = palette.size() - 1
```

```
            var v := i / 255.0
```

```
            idx.set_pixel(x, y, Color(v, v, v, 1.0))
```

```
    var pal := Image.create_empty(16, 1, false, Image.FORMAT_RGBA8)
```

```
    for i in palette.size():
```

```
        pal.set_pixel(i, 0, palette[i])
```

```
    idx.save_png(SOURCE.get_basename() + "_idx.png")
```

```
    pal.save_png(SOURCE.get_basename() + "_pal.png")
```

```
    print("%d couleurs. Fichiers écrits." % palette.size())
```

```
res://fx/indexed.gdshader
```

```
shader_type canvas_item;
// TEXTURE = carte d'index (valeur rouge = index / 255). palette = 16 colonnes x N lignes.

uniform sampler2D palette : filter_nearest;
uniform float palette_count = 2.0; // nombre de lignes dans la texture de palette
uniform float from_row = 0.0; // palette de départ (0 = nuit)
uniform float to_row = 1.0; // palette d'arrivée (1 = incendie)
uniform int blend_step = 0; // 0..7 : transition par paliers, comme la CRAM

varying vec4 tint; // modulate seul : COLOR, dans fragment(), contient déjà la carte d'index

void vertex() {
    tint = COLOR;
}

void fragment() {
    float idx = floor(texture(TEXTURE, UV).r * 255.0 + 0.5);
    if (idx < 0.5) {
        COLOR = vec4(0.0); // index 0 : transparent
    } else {
        float u = (idx + 0.5) / 16.0;
        vec4 a = texture(palette, vec2(u, (from_row + 0.5) / palette_count));
        vec4 b = texture(palette, vec2(u, (to_row + 0.5) / palette_count));
        vec3 mixed = mix(a.rgb, b.rgb, float(blend_step) / 7.0);
        COLOR = vec4(floor(mixed * 7.0 + 0.5) / 7.0, 1.0) * tint; // reste en 9 bits
    }
}
```

À FAIRE

1. Exécute `indexer.gd` : la Sortie annonce « 15 couleurs » (14 + le transparent). Deux fichiers apparaissent : `facade_debug_idx.png` (gris très sombre, c'est normal) et `facade_debug_pal.png` (16×1).
2. Sélectionne `facade_debug_idx.png` → onglet **Import : Compress** → **Mode** = `LossLess`, **Mipmaps** décoché → **Réimporter**. Une compression mélangerait les index.
3. Dans ton logiciel de dessin, ouvre `facade_debug_pal.png`, agrandis le canevas à 16×2, recopie la ligne du haut en bas, puis repeins la ligne du bas en tons violets et ocre (ambiance incendie). Enregistre sous `facade_debug_pal2.png` et importe-le avec les mêmes réglages.
4. Dans `sandbox/`, une scène avec un `Sprite2D` : texture `facade_debug_idx.png`, matériau `indexed.gdshader`, **palette** = `facade_debug_pal2.png`.

TESTE

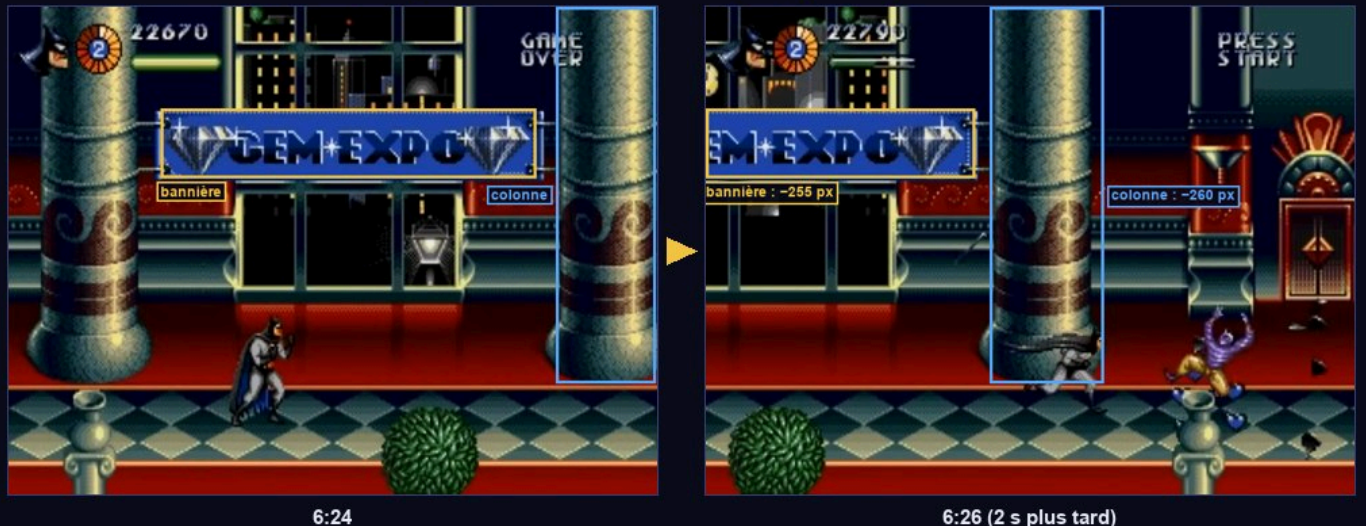
Dans l'éditeur (pas besoin de lancer), fais glisser `blend_step` de 0 à 7 dans le matériau : la façade passe de la palette « nuit » à la palette « incendie » en 8 paliers, sans qu'un seul pixel de forme change.

PARALLAXE ET ÉCHELLE

8.9 · Deux outils pour la suite



Mesure : colonnes et mur se déplacent autant → même plan, pas de parallaxe



Contre-exemple mesuré. Dans le Gem Expo, colonnes, bannière et sol se déplacent exactement d'autant : pas de parallaxe ici. Mesurer avant de coder évite d'ajouter un effet qui n'existe pas.

PARALLAX2D

Pour une couche qui défile d'un bloc (ciel, skyline, rue du convoi), pas besoin de raster : le nœud

Parallax2D (Godot 4.3+) suffit. **Scroll Scale** < 1 = derrière le plan de jeu (bouge moins), > 1 = devant.

Repeat Size fait boucler l'image. **Autoscroll** la fait défiler toute seule. Tu l'utiliseras aux étapes 11 et 12.

SUR MEGA DRIVE

La console ne sait pas agrandir un sprite. Pour simuler la profondeur (la montgolfière du Joker qui approche, les sphères du grappin), le jeu stocke quelques tailles **pré-calculées**. Godot sait agrandir en continu, mais un agrandissement lisse trahirait le remake. On arrondit donc l'échelle à des paliers. Ajoute cette fonction à **fx.gd** :

res://fx/fx.gd (ajout)

```
## Arrondit une échelle au palier le plus proche parmi « sizes », comme des sprites pré-calculés.
static func snap_scale(value: float, sizes: PackedFloat32Array) -> float:
    var best := sizes[0]
    for s in sizes:
        if absf(s - value) < absf(best - value):
            best = s
    return best
```

POINT DE CONTRÔLE

TU AS FINI L'ÉTAPE QUAND

- ☐ La façade de test s'affiche derrière la salle ; en courant, le haut et la vitrine défilent à des vitesses différentes, et les lignes repères restent calées sur les sols de collision.
- ☐ Les foyers de 6 images ondulent : immobiles en bas, très mobiles en haut, désynchronisés.
- ☐ `PaletteFx.fade_out()` assombrit l'écran en 8 paliers visibles ; `flash_white()` blanchit 2 images ; le crâne passe par la palette.
- ☐ La cible clignote en pastel quand on la touche.
- ☐ (Facultatif) Un décor indexé passe de la palette « nuit » à la palette « incendie » en 8 paliers.

```
git git add -A && git commit -m "Étape 08 : effets Mega Drive (palette, raster, feu, flash)"
```

CE QUE TU RETIENS

? Pourquoi le plan raster est-il dans un `CanvasLayer` et non dans le monde ?

Parce qu'il représente l'écran, pas un objet du monde : il doit rester fixe devant la caméra. C'est la table `h_scroll`, calculée à partir de la position de la caméra, qui simule le défilement, ligne par ligne.

? Pourquoi les lignes où l'on marche doivent-elles valoir exactement 1 ?

Parce que les personnages et le sol de collision défilent à la vitesse de la caméra (facteur 1). Si le décor sous leurs pieds défilait plus vite ou plus lentement, ils sembleraient glisser.

? Que vaut `COLOR` au début de `fragment()` dans un shader `canvas_item` de Godot 4 ?

La couleur de la texture multipliée par la teinte (`modulate`). Pour récupérer la teinte seule, on la copie dans une `varying` depuis `vertex()`.

? Que gagne-t-on à stocker une image en couleurs indexées ?

On sépare la forme (les index) de la couleur (la palette). Changer quelques couleurs de la palette recolorise instantanément tout ce qui l'utilise : fondus, flashes, changements d'ambiance, variantes d'ennemis.

POUR ALLER PLUS LOIN

- Écris un `RasterPlane` qui ondule toute l'image comme une vague de chaleur : `h_scroll[y] = sin(TIME + y * 0,1) * 3`. Un nouveau script, le même shader.
- Écris un `RasterPlane` qui écrase l'image verticalement en jouant sur `v_line` (le « secret #5 » de Burton, les immeubles en 3D).
- Lis le tome 06 en entier : il explique chaque effet côté console.

SECTION 1 : RUE, BANQUE ET PREMIER BOSS

Toutes les briques sont prêtes : on assemble enfin un vrai morceau du niveau. La rue à balcon et ses quatre arènes, l'explosion scriptée de la banque qui fait basculer l'ambiance, puis la machine d'Harley Quinn en trois phases.

~6 h

Prérequis : étape 08

Guide : [tome 07](#)

CE QUE TU VAS APPRENDRE

GODOT 4

- `change_scene_to_file()`, `@export_file`
- `call_group()`, `Tween`
- Scènes héritées
- `CPUParticles2D`, `Marker2D`
- La scène principale du projet

DÉVELOPPEMENT DE JEU

- L'assemblage de niveau
- L'événement scripté et le changement caché sous un flash
- La grammaire d'un boss : phases, télégraphie, point faible
- Le tir balistique
- Le playtest et l'accessibilité

LE SQUELETTE D'UNE SECTION

9.1 · La classe `Section` et `SceneFlow`



Les trois sections partagent la même mécanique : gérer la mort du joueur, finir sur un boss, passer à la suite. On la factorise.

res://sections/section.gd

```
class_name Section
extends Node2D
## Base commune des 3 sections du niveau 1.

@export_file("*.tscn") var next_scene: String
@export var player: Player
@export var victory_delay := 3.0    ## durée de la pose de victoire (vidéo : ~3 s)

func _ready() -> void:
    Game.current_section = scene_file_path    # point de reprise du « continue »
    player.died.connect(_on_player_died)

func _on_player_died() -> void:
    if Game.lose_life():
        player.respawn()
    else:
        SceneFlow.game_over()

## Appelé par l'arène du boss quand il est vaincu.
func boss_defeated(points: int) -> void:
    Game.add_score(points)
    player.celebrate()
    await get_tree().create_timer(victory_delay).timeout
    SceneFlow.goto(next_scene)    # SceneFlow s'occupe du fondu au noir puis du fondu d'entrée
```

SceneFlow sera complet à l'étape 10. Cette version minimale suffit pour l'instant :

res://autoload/scene_flow.gd (version minimale)

```
extends Node
## Change de scène entre deux fondus Mega Drive.

func goto(path: String) -> void:
    if path.is_empty():
        push_warning("SceneFlow.goto : aucune scène indiquée (next_scene vide ?)")
        return
    await PaletteFx.fade_out()
    get_tree().change_scene_to_file(path)
    await get_tree().process_frame    # le changement de scène a lieu en fin d'image
    await get_tree().process_frame
    await PaletteFx.fade_in()

func game_over() -> void:
    goto(Game.current_section)    # remplacé au tome 10 par l'écran de continue
```

À FAIRE

1. Crée les deux scripts.
2. Ajoute `res://autoload/scene_flow.gd` en autoload sous le nom **SceneFlow**, après **PaletteFx**.

UNE SEULE AUTORITÉ POUR LES TRANSITIONS

Les fondus sont gérés à **un seul endroit** : `SceneFlow`. Si chaque section faisait aussi les siens, on obtiendrait des doubles fondus ou des écrans qui « reclignent ». Règle générale : un effet global appartient à un seul propriétaire.

`@EXPORT_FILE` ET `SCENE_FILE_PATH`

`@export_file("*.tscn")` affiche un sélecteur de fichier dans l'Inspecteur, filtré sur les scènes.

`scene_file_path` est le chemin du fichier `.tscn` dont la racine a été chargée : c'est ce qu'on mémorise pour reprendre la section après un « continue ».

LA RUE

9.2 · Monter la scène `s1_streets.tscn`



Début de la section. ▶ 1:50



Le balcon praticable. ▶ 3:30



Après l'explosion de la banque. ▶ 2:52

À FAIRE

1. Nouvelle scène 2D, racine `S1Streets`, script `section.gd`. Enregistre en `res://sections/s1_streets.tscn`. Laisse la racine en (0, 0) : tout le niveau se repère par rapport à elle.
2. Construis l'arbre ci-dessous. Les éléments marqués $\diamond$ viennent plus loin dans cette étape.

```
S1Streets (Node2D, section.gd)      next_scene = (provisoirement) s1_streets.tscn · player = Player
├─ Background (CanvasLayer, layer = -10)
│   └─ Facade (ColorRect)          facade_raster.gd + raster_plane.gdshader (comme à l'étape 08)
├─ World (Node2D)
│   ├── Ground (StaticBody2D)      couche 1 · rectangle 3000×16 · en (1500, 208)
│   ├── Balcony (StaticBody2D)     couche 2 · plusieurs CollisionShape2D one-way de 8 px de haut, y = 112
│   ├── Ledges (Node2D)           une Area2D couche 8 sous chaque tronçon de balcon, y = 108
│   ├── Props (Node2D)            poubelles, crânes
│   ├── Fires (Node2D, caché)      $\diamond$  foyers fire.gd devant les fenêtres de la banque
│   ├── ArenaA ... ArenaD (WaveArena)  $\diamond$ 
│   ├── BankEvent (Area2D)         $\diamond$ 
│   └─ BossArena (Area2D)          $\diamond$  → HarleyMachine
├─ EnemyBullets (Pool)            groupe « enemy_bullets » · scene = enemy_bullet.tscn
├─ Player (player.tscn)           en (40, 200)
├─ Camera (Camera2D, camera_rig.gd) target = Player
│   └─ Debris (CPUParticles2D)     $\diamond$  papiers et billets, émis après l'explosion
```

3. **Le balcon à trous** : sous `Balcony`, un `CollisionShape2D` par tronçon, par exemple de x = 200 à 900, de 1 000 à 1 800 et de 1 900 à 2 600. Chaque forme : rectangle de (longueur)×8, One Way Collision coché, position y = 112. Ajoute des `ColorRect` semi-transparents pour les voir.
4. **Les rebords** : pour chaque tronçon, une `Area2D` (couche 8) positionnée en y = 108, avec une forme de (longueur)×4 en (0, 2).
5. Pour `Facade`, réutilise la façade de test et la courbe de l'étape 08. Astuce : copie le nœud `Background` de la salle de test (`Ctrl C`) et colle-le ici (`Ctrl V`).
6. Sélectionne la racine et renseigne **Player** et **Next Scene** (provisoirement la section elle-même : elle bouclera sur elle-même tant que la section 2 n'existe pas).

MÉTHODE : LE « GREYBOX »

On construit le niveau en formes grises (ou semi-transparentes) avant tout décor définitif. On peut ainsi régler les distances, la hauteur du balcon, la place des arènes et le rythme, et tout déplacer en quelques secondes. Le décor est la dernière couche, posée quand le niveau est amusant.

TESTE

F6 : Batman parcourt une rue de 3 000 px avec un balcon à trous, sur lequel il peut monter en s'accrochant. La façade défile derrière. Pour mourir, provisoirement, place un piège `spikes` : Batman perd une vie et réapparaît ; à la troisième mort, la section recommence (version minimale du game over).

9.3 · Placer les arènes



À ~117 px/s, chaque seconde de vidéo représente environ 117 px de rue. Ce découpage est donc approximatif : ajuste-le en jouant.

Repère vidéo	X approx.	Contenu
▶ 1:46	0-320	Départ, poubelles, premier bonus d'arme
▶ 1:52 — ▶ 2:25	~480	Arène A : clowns du balcon et de la rue (2 vagues) → GO!
▶ 2:27	~700-1000	Couloir : tireurs, crâne ; la façade de la banque apparaît
▶ 2:44	~1100	Événement de la banque
▶ 2:51 — ▶ 3:18	~1300	Arène B : costauds, sbires armés → GO!
▶ 3:20 — ▶ 3:37	~1650	Arène C : mélange, tireurs sur le balcon → GO!
▶ 4:06 — ▶ 4:42	~2200	Arène D : tireurs, clowns → GO!
▶ 4:44	~2500-2700	Rue vide, puis BossArena

À FAIRE

- Place quatre **WaveArena** aux X du tableau (forme 8×224, masque 3, **Go Sign Scene** renseignée).
- Remplis leurs vagues. Une proposition de départ :
 - A** : [clown] quota 4, max 2, origines top + balcony ; puis [clown] quota 5, max 3.
 - B** : [clown, thug] quota 5, max 3 ; puis [thug, gunner] quota 4, max 2.
 - C** : [gunner] quota 3, origine balcony ; puis [clown, gunner, thug] quota 6, max 3.
 - D** : [clown, gunner] quota 8, max 4 ; puis [thug] quota 3, max 2.
- Dans **Props**, place quelques **trash_can.tscn** (avec un **WeaponPickup**) dans les couloirs, et un **skull_bomb.tscn** vers x = 800.
- Dans le couloir entre A et la banque, pose deux **gunner.tscn** à la main sur le balcon.

CONCEPT : LE RYTHME TENSION / DÉTENTE

Arène (tension, écran figé) → couloir (détente, on avance) → arène... Le joueur respire entre deux combats, et un nouvel ennemi ou un bonus dans un couloir se remarque d'autant mieux. Quand tu places tes arènes, garde cette alternance, et fais monter la difficulté d'une arène à l'autre.

TESTE

Joue la rue entière. Note dans ton journal chaque moment où tu t'ennuies (trop long, trop facile) et chaque moment où tu meurs sans comprendre pourquoi. Ce sont tes deux listes de réglages.

L'EXPLOSION DE LA BANQUE

9.4 · Observer l'événement



La banque : ~24 flashes de 3 images ; la palette change pendant l'un d'eux



La séquence, image par image. Avant ; un flash de 3 images ; des explosions entre deux flashes ; le flash pendant lequel la palette bascule ; la rue en feu.

DANS LA VIDÉO

De [2:43.9](#) à [2:50.4](#) : des explosions en chaîne à l'étage, et environ 24 flashes blancs de 3 images chacun, à intervalles irréguliers (un toutes les 0,27 s en moyenne). La palette bascule vers le violet et l'ocre à [2:49.6](#), pendant un flash. Ensuite, des flammes sortent des fenêtres et des papiers tombent en continu.

CONCEPT : CACHER UN CHANGEMENT SOUS UN FLASH

Changer tout le décor d'un coup serait brutal. Les flashes servent de **rideau** : c'est pendant l'un d'eux qu'on change la palette, qu'on allume les feux et qu'on lance les débris. Quand le blanc retombe, le monde a changé, et le joueur l'accepte comme la conséquence de l'explosion.

9.5 · Préparer la façade « incendie »



Version simple : une seconde texture de façade, aux couleurs de l'incendie, qu'on échangera pendant un flash. Avec la façade de test, il suffit de relancer le générateur avec d'autres teintes.

À FAIRE

- Dans `make_debug_facade.gd`, change deux lignes : `const PATH := "res://art/s1/facade_debug_fire.png"` et `var hue := Color.from_hsv(0.75 + (band % 6) / 6.0 * 0.2, 0.6, 0.55)` (des violets vers le rose).
- Exécute-le, puis remets les deux lignes d'origine.
- Avec de vrais décors, ce serait la même façade repeinte en violet et ocre ; la version fidèle est la bascule indexée de l'étape 08.

9.6 · `bank_event.gd`



```
res://sections/s1/bank_event.gd
```

```
extends Area2D
## Déclencheur (masque 3) : explosion de la Bank of Gotham City.
```

```

@export var blast_points: Array[Marker2D] = []    ## fenêtres de l'étage
@export var facade: ColorRect                    ## le plan raster
@export var fire_texture: Texture2D              ## la même façade, palette « incendie »
@export var fires: Node2D                       ## foyers fire.gd, cachés au départ
@export var debris: CPUParticles2D
var _done := false

func _ready() -> void:
    body_entered.connect(_on_body_entered)
    fires.hide()

func _on_body_entered(body: Node2D) -> void:
    if _done or not body is Player:
        return
    _done = true
    _run()

@export var duration := 6.5                      ## mesuré : 2:43.9 → 2:50.4
@export var switch_at := 5.7                     ## mesuré : la palette bascule à 2:49.6
## Intervalle aléatoire entre deux flashes. Moyenne 0,225 s + 3 images de flash ≈ 0,27 s :
## environ 24 flashes en 6,5 s, comme mesuré.
@export var flash_gap := Vector2(0.1, 0.35)

func _run() -> void:
    var t := 0.0
    var switched := false
    while t < duration:
        # Un flash blanc de 3 images exactement, comme mesuré dans la vidéo.
        # C'est pendant un flash, quand on ne voit rien, qu'on change le monde.
        PaletteFx.set_flash(7)
        if not switched and t >= switch_at:
            switched = true
            _switch_to_fire()
        await PaletteFx.flash_white(3, 7)
        t += 3.0 / 60.0
        # Entre deux flashes : des explosions à l'étage.
        var m: Marker2D = blast_points.pick_random()
        Fx.explosion(m.global_position + Vector2(randf_range(-16, 16), randf_range(-8, 8)))
        var gap := randf_range(flash_gap.x, flash_gap.y)
        await get_tree().create_timer(gap).timeout
        t += gap
    if not switched:
        _switch_to_fire()

func _switch_to_fire() -> void:
    facade.material.set_shader_parameter("plane", fire_texture)
    fires.show()
    debris.emitting = true

```

Comprendre le code

- Ici, `await` est le bon outil : c'est une **séquence scriptée**, linéaire, qui ne dépend d'aucun état de jeu. Une boucle `while` avec des pauses se lit comme un scénario.

- Les exports déclarés au milieu du fichier sont permis en GDScript, mais on les met d'habitude en haut. Déplace-les si tu préfères : c'est un bon exercice de lecture.
- `facade.material.set_shader_parameter("plane", ...)` : le plan raster a dupliqué son matériau dans `_ready()` ; `facade.material` désigne bien cette copie.

À FAIRE

1. Sous `World`, ajoute `BankEvent` (`Area2D`, masque 3, forme 8×224) en $x \approx 1\,100$, avec ce script.
2. Enfants de `BankEvent` : trois ou quatre `Marker2D` à la hauteur des fenêtres de l'étage ($y \approx 60$). Glisse-les dans le tableau **Blast Points**.
3. `Fires` : trois instances du foyer de l'étape 08 (`Sprite2D` + `fire.gd`) à la hauteur des fenêtres. Le script les cache au démarrage.
4. `Debris` (enfant de la caméra) : `CPUParticles2D`, **Emitting** décoché, **Amount** 30, **Lifetime** 6, **Emission Shape** = Rectangle avec **Rect Extents** = (160, 2) (ce sont des demi-tailles : la zone fait 320×4), position (0, -120), **Gravity** (0, 30), **Angle** aléatoire, une petite texture (un rectangle blanc de 3×2 fait l'affaire).
5. Renseigne les exports de `BankEvent` : **Facade**, **Fire Texture** (`facade_debug_fire.png`), **Fires**, **Debris**.

TESTE

En passant $x \approx 1\,100$: des explosions à l'étage et des flashes blancs stroboscopiques pendant 6,5 s. Pendant l'un des derniers flashes, la façade change de couleurs, les feux apparaissent et les papiers commencent à tomber. Ils continuent jusqu'au boss.

ACCESSIBILITÉ

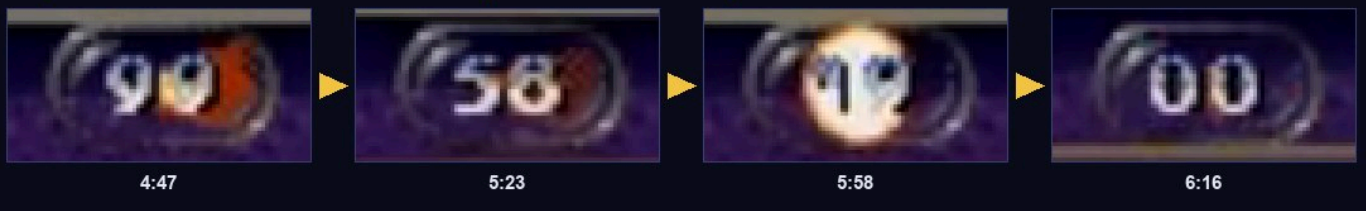
24 flashes blancs en 6 s forment un stroboscope qui peut gêner les personnes photosensibles. C'est fidèle à l'original, mais un jeu moderne propose une option : remplace `flash_white(3, 7)` par `flash_white(3, 3)` (flash atténué) quand l'option est active. Penser à l'accessibilité fait partie du métier.

LE PREMIER BOSS

9.7 · La base commune des boss



La pastille = vitalité du boss : 99 → 00 (pas un chronomètre)



La pastille centrale. Elle descend au rythme des impacts et tombe à 00 à la destruction : c'est la vitalité du boss que renvoie `Boss.vitality()`.

```
res://bosses/boss.gd
```

```
class_name Boss
extends Node2D

## Base des boss : vitalité 99 → 00 (pastille du HUD), démarrage, mort, points.

signal vitality_changed(value: int)
signal defeated

@export var score_value := 5000
@onready var health: Health = $Health
var active := false
var player: Player

func _ready() -> void:
    add_to_group("bosses")
    player = get_tree().get_first_node_in_group("player") as Player
    health.changed.connect(func(_c: int, _m: int) -> void: vitality_changed.emit(vitality()))
    health.died.connect(_on_died)

## Valeur affichée dans la pastille : 99 à pleine vie, 00 à la mort (vidéo 6:16, 10:34, 14:23).
func vitality() -> int:
    return ceil(99.0 * health.current / health.max_health)

func start() -> void:
    active = true
    get_tree().call_group("hud", "show_boss", self) # le HUD (tome 10) s'abonne
    vitality_changed.emit(vitality())

func _on_died() -> void:
    active = false
    await die_sequence()
    get_tree().call_group("hud", "hide_boss")
    defeated.emit()

## À surcharger : explosions, chute du pilote...
func die_sequence() -> void:
    await get_tree().create_timer(1.0).timeout

## Utilitaire : chapelet d'explosions aléatoires dans un rectangle local.
func explode_area(area: Rect2, count: int, interval: float) -> void:
    for i in count:
        var p := Vector2(randf_range(area.position.x, area.end.x), randf_range(area.position.y,
            area.end.y))
        Fx.explosion(to_global(p)) # dans la scène courante, pas sous la BossArena
        await get_tree().create_timer(interval).timeout
```

res://bosses/boss_arena.gd

```
class_name BossArena
extends Area2D
## Déclencheur (masque 3) : fige la caméra, lance le boss, prévient la section à sa mort.

@export var boss: Boss
@export var section: Section
@export var intro_delay := 1.0
var _done := false

func _ready() -> void:
    body_entered.connect(_on_body_entered)

func _on_body_entered(body: Node2D) -> void:
    if _done or not body is Player:
        return
    _done = true
    (body as Player).camera.lock_on(global_position.x)
    boss.defeated.connect(func() -> void: section.boss_defeated(boss.score_value), CONNECT_ONE_SHOT)
    await get_tree().create_timer(intro_delay).timeout
    boss.start()
```

Et la fonction de tir balistique, à ajouter à `fx.gd` :

res://fx/fx.gd (ajout)

```
## Vitesse initiale pour qu'un projectile soumis à « gravity » aille de « from » à « to »
## en « time » secondes. Physique de lycée :  $y(t) = y_0 + v_y * t + g * t^2 / 2$ .
static func ballistic(from: Vector2, to: Vector2, gravity: float, time: float) -> Vector2:
    var d := to - from
    return Vector2(d.x / time, d.y / time - 0.5 * gravity * time)
```

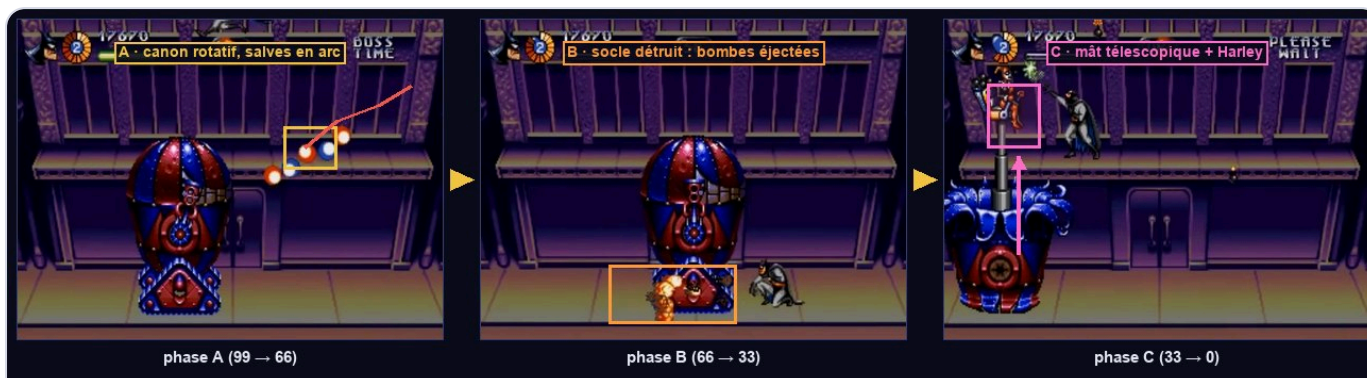
CONCEPT : `CALL_GROUP`, LE MESSAGE À QUI VEUT L'ENTENDRE

Le boss ne sait pas si un HUD existe. Il « crie » `show_boss` à tous les nœuds du groupe `hud`. S'il n'y en a aucun (comme maintenant, avant l'étape 10), rien ne se passe et rien ne plante. C'est un découplage encore plus fort que les signaux.

CONCEPT : VISER AVEC LA PHYSIQUE

Pour qu'une bombe lobée tombe sur Batman, on fixe la **durée du vol** et on résout l'équation du mouvement pour trouver la vitesse de départ. Changer `time` donne des lobes plus tendus ou plus hauts, sans toucher au reste.

9.8 · La machine d'Harley : le plan du combat



Les trois phases de la machine d'Harley. A : canon rotatif et salves en arc · B : socle détruit, bombes éjectées · C : mât télescopique, Harley exposée au niveau du balcon.

Phase	Vitalité	Partie vulnérable	Attaque	Mouvement
A · complète	99 → 66	Capsule	Salves de 4 boules en arc, canon qui balaie de l'horizontale à la verticale	Glisse de gauche à droite
B · capsule	66 → 33	Capsule	Canon qui tourne à 360°, salves plus fréquentes	Plus rapide
C · Harley	33 → 0	Harley seule	Bombes lobées sur Batman	Mât télescopique qui monte au niveau du balcon

CONCEPT : LA GRAMMAIRE D'UN BOSS

- **Des phases** liées à des seuils de vie : le combat évolue, et le joueur sent qu'il progresse.
- **Une transition spectaculaire** (explosion, pièce qui saute), avec une **brève invincibilité** pendant laquelle le joueur souffle.
- **Un point faible qui se déplace** : en phase C, il faut monter sur le balcon pour atteindre Harley. Le décor devient un élément du combat.
- **La télégraphie** : le canon pointe là où il va tirer.

9.9 · La machine d'Harley : le code



```
res://bosses/harley_machine/harley_machine.gd
```

```
class_name HarleyMachine
extends Boss
## Boss de la section 1 (vidéo 4:46-6:21).

enum Phase { A, B, C }

@export var ball_scene: PackedScene          ## bomb.tscn : bounces = 1, fall_gravity = 300
@export var harley_bomb_scene: PackedScene    ## bomb.tscn : bounces = 0
@export var arena_half_width := 110.0
@export var slide_speed := 45.0
@export var base_height := 24.0              ## de combien la capsule descend en phase B
@export var mast_min := 16.0
```

```

@export var mast_max := 80.0          ## jusqu'au niveau du balcon
@export var transition_time := 1.2

var phase := Phase.A
var _t := 0.0
var _dir := 1
var _center_x := 0.0
var _fire_t := 1.5
var _angle := 0.0                    ## angle du canon (radians)
var _transitioning := false

@onready var base: Node2D = $Base
@onready var capsule: Node2D = $Capsule
@onready var cannon: Node2D = $Capsule/Cannon
@onready var muzzle: Marker2D = $Capsule/Cannon/Muzzle
@onready var crown: Node2D = $Crown
@onready var mast: Node2D = $Mast
@onready var pole: Sprite2D = $Mast/Pole
@onready var harley: Node2D = $Mast/Harley
@onready var hurt_capsule: Hurtbox = $Capsule/Hurtbox
@onready var hurt_harley: Hurtbox = $Mast/Harley/Hurtbox

func _ready() -> void:
    super._ready()
    _center_x = global_position.x
    crown.hide()
    mast.hide()
    hurt_harley.monitorable = false
    health.changed.connect(_check_phase)

func _physics_process(delta: float) -> void:
    if not active or _transitioning:
        return
    _t += delta
    match phase:
        Phase.A:
            _slide(delta, slide_speed)
            # Balaye de l'horizontale (0) à la verticale (PI/2), côté joueur.
            _angle = (sin(_t * 0.9) * 0.5 + 0.5) * PI / 2.0
            _aim_towards_player()
            _volley(delta, 1.6, 4)
        Phase.B:
            _slide(delta, slide_speed * 1.4)
            _angle = fposmod(_angle + 1.8 * delta, TAU) # tours complets
            cannon.rotation = -_angle
            _volley(delta, 1.1, 4)
        Phase.C:
            _slide(delta, slide_speed * 0.6)
            _update_mast()
            _harley_attack(delta)

func _slide(delta: float, speed: float) -> void:
    global_position.x += _dir * speed * delta
    var offset := global_position.x - _center_x
    if absf(offset) > arena_half_width:

```

```

global_position.x = _center_x + signf(offset) * arena_half_width
_dir = -_dir

func _aim_towards_player() -> void:
    var side := signf(player.global_position.x - global_position.x) if player else -1.0
    # side = 1 : le canon part vers la droite et monte ; side = -1 : vers la gauche.
    cannon.rotation = -_angle if side > 0.0 else PI + _angle

func _volley(delta: float, interval: float, count: int) -> void:
    _fire_t -= delta
    if _fire_t > 0.0 or ball_scene == null:
        return
    _fire_t = interval
    var dir := Vector2.RIGHT.rotated(cannon.rotation)
    for i in count:
        var b := ball_scene.instantiate() as Bomb
        b.velocity = dir * (110.0 + i * 35.0) # vitesses étagées : les boules forment un arc
        Fx.spawn(b, muzzle.global_position)

func _update_mast() -> void:
    var k := sin(_t * 0.8) * 0.5 + 0.5
    var length := lerpf(mast_min, mast_max, k)
    harley.position.y = -length
    pole.scale.y = length / pole.texture.get_height() if pole.texture else 1.0
    pole.position.y = -length / 2.0

func _harley_attack(delta: float) -> void:
    _fire_t -= delta
    if _fire_t > 0.0 or harley_bomb_scene == null or player == null:
        return
    _fire_t = 1.3
    var b := harley_bomb_scene.instantiate() as Bomb
    b.velocity = Fx.ballistic(harley.global_position, player.global_position + Vector2(0, -16),
        b.fall_gravity, 0.9)
    Fx.spawn(b, harley.global_position)

func _check_phase(current: int, maximum: int) -> void:
    if phase == Phase.A and current <= maximum * 66 / 99:
        _go_phase(Phase.B)
    elif phase == Phase.B and current <= maximum * 33 / 99:
        _go_phase(Phase.C)

func _go_phase(next: Phase) -> void:
    _transitioning = true
    health.invincible = true
    match next:
        Phase.B:
            await explode_area(Rect2(-32, -base_height, 64, base_height), 6, 0.08)
            base.hide()
            _eject_ring(4)
            var tw := create_tween()
            tw.tween_property(capsule, "position:y", capsule.position.y + base_height, 0.4)

```

```

Phase.C:
    await explode_area(Rect2(-36, -96, 72, 48), 8, 0.07)
    capsule.hide()
    hurt_capsule.set_deferred("monitorable", false)
    crown.show()
    mast.show()
    hurt_harley.set_deferred("monitorable", true)
await get_tree().create_timer(transition_time).timeout
phase = next
_fire_t = 1.0
health.invincible = false
_transitioning = false

func _eject_ring(count: int) -> void:
    ## Le socle détruit projette des bombes en couronne (vidéo 5:23).
    for i in count:
        var b := ball_scene.instantiate() as Bomb
        b.velocity = Vector2.UP.rotated(lerpf(-1.0, 1.0, i / float(count - 1))) * 180.0
        Fx.spawn(b, capsule.global_position)

func die_sequence() -> void:
    # Anneau d'explosions plein écran (vidéo 6:16-6:18).
    for i in 3:
        PaletteFx.flash_white(2, 4)
        await explode_area(Rect2(-120, -110, 240, 120), 6, 0.06)
    hide()

```

Comprendre le code

- `_check_phase` est branché sur `health.changed` : c'est la **vie** qui pilote les phases, pas le temps. `maximum * 66 / 99` donne le seuil quelle que soit la vie maximale choisie.
- `_transitioning` + `health.invincible` : pendant la transition, le boss ne bouge plus, ne tire plus et n'encaisse plus. Le joueur voit la pièce exploser et souffle.
- `_go_phase` utilise `await` : c'est une séquence scriptée (explosions, puis pièce qui disparaît, puis pause). Pendant ce temps, `_physics_process` est bloqué par `_transitioning`.
- Changer de partie vulnérable revient à basculer `monitorable` sur deux Hurtbox : la capsule cesse d'être visible pour les tirs, Harley le devient.
- Toutes les bombes passent par `Fx.spawn()` : le boss est l'enfant de la `BossArena`, placée loin de l'origine. Avec l'ancien schéma (`global_position` puis `add_child` sous le parent), les bombes apparaîtraient à des milliers de pixels.

À FAIRE

1. **Deux variantes de bombe** : clic droit sur `bomb.tscn` → **Nouvelle scène héritée**. Dans la copie, **Fall Gravity** = 300 : enregistre en `res://bosses/harley_machine/harley_ball.tscn`. Recommence avec **Bounces** = 0 : `harley_bomb.tscn`.
2. **La scène du boss** : nouvelle scène, racine `Node2D` nommée `HarleyMachine`, script `harley_machine.gd`, enregistrée dans `res://bosses/harley_machine/`. Construis l'arbre ci-dessous ; chaque `Sprite2D` utilise `white.png` avec une échelle et une couleur (socle rose 64×24, capsule rayée rouge 56×48, canon gris 24×6 dont le pivot est à gauche, couronne bleue 60×12, mât gris, Harley 16×20).

```

HarleyMachine (Node2D, harley_machine.gd)    score_value = 5000
├── Health                                     max_health = 99 (À CALIBRER)
├── Base (Node2D)                             socle rose à pointes
│   └── Sprite2D
├── Capsule (Node2D)                         position (0, -24) : posée sur le socle
│   ├── Sprite2D                             matériau hit_flash (Local To Scene)
│   ├── Hurtbox                             couche 6 · health = Health
│   │   └── CollisionShape2D
│   └── Cannon (Node2D)                     pivot du canon, sur le côté du dôme
│       ├── Sprite2D                       décalé de +12 en x : le canon part du pivot
│       └── Muzzle (Marker2D)               bout du canon, en (24, 0)
├── Crown (Sprite2D, caché)                  couronne bleue « en pétales »
├── Mast (Node2D, caché)
│   ├── Pole (Sprite2D)                    texture white.png : sa hauteur est étirée par le code
│   └── Harley (Node2D)
│       ├── Sprite2D                       Harley dans son fauteuil
│       └── Hurtbox                         couche 6 · health = Health
└── HitFlash (HitFlash)                    target = Capsule/Sprite2D · hurtbox = Capsule/Hurtbox

```

3. Exports du boss : **Ball Scene** = `harley_ball.tscn`, **Harley Bomb Scene** = `harley_bomb.tscn`.
4. **L'arène du boss** : sous `World`, une `Area2D` nommée `BossArena`, script `boss_arena.gd`, masque 3, forme 8×224, en (2 700, 112). Instancie `HarleyMachine` comme **enfant** de `BossArena`, en position locale (0, 96) : son origine est au sol (y = 208).
5. Exports de `BossArena` : **Boss** = `HarleyMachine`, **Section** = `S1Streets`.

TESTE

- Pour aller vite, déplace temporairement `Player` à x = 2 550. En entrant dans l'arène, la caméra se fige et la machine démarre une seconde plus tard.
- Phase A : elle glisse, le canon balaie, des salves de 4 boules partent en arc et rebondissent. Les boules naissent bien au bout du canon.
- Accroupi, Batman esquive les boules basses. Les tirs sur la capsule la font clignoter.
- Au tiers de la vie : explosions, le socle disparaît, 4 bombes en couronne, la capsule descend. Phase B : le canon tourne en continu.
- Aux deux tiers : le dôme explose, Harley monte sur son mât jusqu'au niveau du balcon et lobe des bombes là où se trouvait Batman. Monte sur le balcon pour la toucher.
- À 0 : explosions en anneau, la machine disparaît, Batman prend la pose de victoire, et 3 s plus tard la section recommence par un fondu (en attendant la section 2).

SI ÇA NE MARCHE PAS

- **Le boss ne démarre pas** : vérifie le masque 3 de `BossArena` et ses exports.
- **« Invalid get index 'score_value' » à la mort du boss** : l'export **Boss** de l'arène est vide.
- **Le boss perd de la vie avant le combat** : as-tu bien la version de `skull_bomb.gd` qui vérifie `boss.get("active")` ?
- **Harley est intouchable en phase C** : sa Hurtbox doit être sur la couche 6 et désigner le même `Health` que la capsule.

À CALIBRER

Vitalité (99 points, donc 99 batarangs de niveau 1), cadence des salves et vitesses des boules sont des hypothèses. Les seuils 66 et 33 découpent le combat en trois tiers, ce qui correspond aux durées observées dans la vidéo (~35 s, ~38 s, ~18 s). Chronomètre ton combat et compare.

9.11 · Jouer la section de bout en bout



À FAIRE

1. Remets `Player` au départ, en (40, 200).
2. ≡ **Projet** → **Paramètres du projet** → **Application** → **Exécuter** → **Scène principale** = `s1_streets.tscn`. Désormais `F5` lance la section. (L'étape 10 remplacera par l'intro.)
3. Joue la section entière trois fois, sans rien modifier entre deux parties. Note tout dans ton journal.
4. Puis règle : position des arènes, contenu des vagues, vie du boss.

MÉTHODE : LE PLAYTEST

Jouer pour tester n'est pas jouer pour s'amuser : on note ce qu'on ressent au moment où on le ressent, sans corriger en cours de partie. Ensuite seulement, on trie et on règle. Mieux encore : fais jouer quelqu'un d'autre et regarde-le sans rien dire. Tout ce qu'il ne comprend pas est un problème de ton jeu, pas de lui.

POINT DE CONTRÔLE

TU AS FINI L'ÉTAPE QUAND

- ☐ La section apparaît par un fondu en 8 paliers ; les arènes s'enchaînent avec leurs « GO! ».
- ☐ À la banque : explosions et flashes stroboscopiques de 3 images, puis rue en feu, flammes ondulantes, papiers qui tombent.
- ☐ Le boss glisse, tire des arcs de boules, perd son socle à 66, son dôme à 33, puis Harley monte sur son mât.
- ☐ À 00 : explosions, pose de victoire, +5 000, fondu, et la scène suivante se charge.
- ☐ Mourir fait perdre une vie et réapparaître ; sans vie, la section recommence.

```
git add -A && git commit -m "Étape 09 : section 1 complète avec la machine d'Harley"
```

CE QUE TU RETIENS

? Pourquoi la banque utilise-t-elle `await` alors que Batman n'en utilise pas ?

La banque est une séquence linéaire, qui ne peut pas être interrompue par une décision de jeu : `await` la rend lisible comme un scénario. La logique de Batman, elle, peut être interrompue à tout moment (coup reçu, mort) : des compteurs dans les états évitent les minuteries fantômes.

? Qu'est-ce qui déclenche les changements de phase du boss ?

Le signal `health.changed`, comparé à des seuils (66 % et 33 % de la vie). Les phases suivent donc les dégâts infligés par le joueur, quel que soit le temps passé.

? Pourquoi le boss « crie » au HUD par `call_group` au lieu de l'appeler directement ?

Pour ne pas dépendre de son existence. S'il n'y a pas de HUD (scène de test, HUD pas encore écrit), l'appel ne touche personne et rien ne plante. Le boss n'a besoin de connaître ni la place ni le type du HUD.

? Comment calcule-t-on la vitesse de départ d'une bombe qui doit atterrir sur Batman ?

On choisit la durée du vol t . Horizontalement, $v_x = dx / t$. Verticalement, $y(t) = y_0 + v_y \cdot t + g \cdot t^2 / 2$, donc $v_y = dy / t - g \cdot t / 2$.

POUR ALLER PLUS LOIN

- Ajoute l'option d'accessibilité : une variable `Game.reduce_flashes` lue par `bank_event.gd`.
- Ajoute à la machine une `Hitbox` de contact (masque 5) : toucher la machine fait mal.
- Lis le tome 07 du guide et compare ta section à la frise du [tome 01](#).

HUD, ÉCRANS ET FLUX DE JEU

Ce qui transforme une section jouable en un jeu : un HUD fidèle, l'intro, l'écran titre, les options, le choix du héros, les interludes, le « continue », la musique, et le chef d'orchestre qui enchaîne tout. On le fait maintenant, pour que les deux sections suivantes s'insèrent dans un vrai jeu.

~5 h

Prérequis : étape 09

Guide : tome 10

CE QUE TU VAS APPRENDRE

GODOT 4

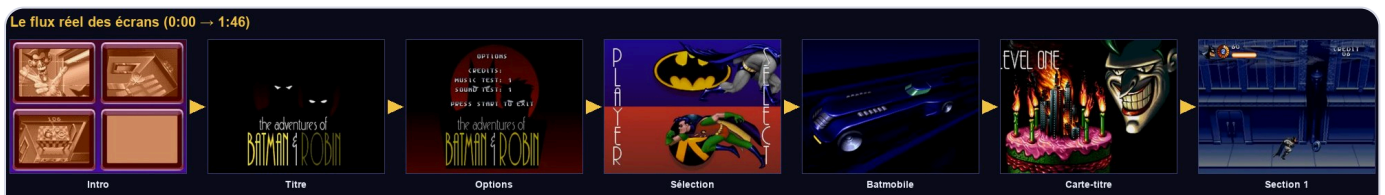
- Les nœuds `Control`, les ancres, le focus
- `Theme` et police par défaut
- Dessiner soi-même : `_draw()`, `queue_redraw()`
- `AnimationPlayer` et pistes d'appel de méthode
- Bus audio, `AudioStreamPlayer`, `ConfigFile`

DÉVELOPPEMENT DE JEU

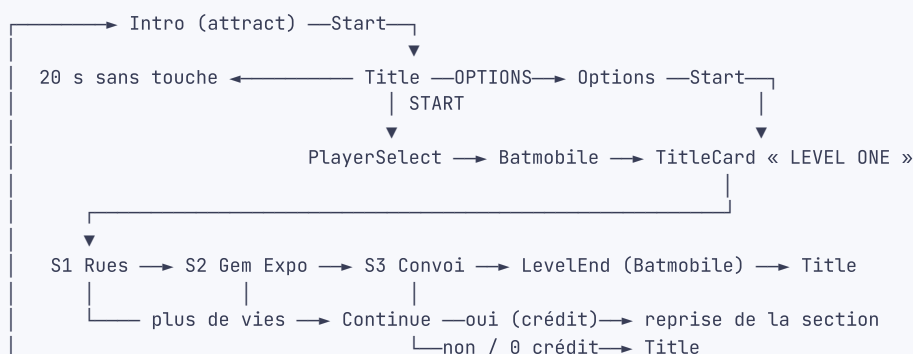
- Le jeu comme machine à états
- Le HUD découplé par signaux
- La cinématique comme frise temporelle
- Réglage persistant contre état de partie
- L'*attract mode* des jeux d'arcade

LE FLUX

10.1 · Le jeu est une machine à états



Les écrans réels, dans l'ordre de la vidéo. Chacun est une scène Godot ; `SceneFlow` les enchaîne avec des fondus.



CONCEPT : LE JEU LUI-MÊME EST UNE MACHINE À ÉTATS

Titre, sélection, partie, continue : chaque écran est un **état du jeu**, chaque scène Godot en est la réalisation, et `SceneFlow` gère les transitions. Ce sont les mêmes idées que pour Batman, à l'échelle de l'application entière. Recopie ce schéma : chaque flèche est un appel à `SceneFlow` que tu vas écrire.

10.2 · `SceneFlow` , version complète



Remplace le contenu de `scene_flow.gd` :

```
res://autoload/scene_flow.gd (Autoload « SceneFlow »)

extends Node
## Chef d'orchestre des écrans. Toutes les transitions passent par ici (fondus Mega Drive).

const INTRO := "res://screens/intro.tscn"
const TITLE := "res://screens/title.tscn"
const CONTINUE := "res://screens/continue.tscn"

var _busy := false

func goto(path: String) -> void:
    if _busy or path.is_empty():
        return
    _busy = true
    get_tree().paused = false
    await PaletteFx.fade_out()
    get_tree().change_scene_to_file(path)
    await get_tree().process_frame # le changement a lieu en fin d'image
    await get_tree().process_frame
    await PaletteFx.fade_in()
    _busy = false

func game_over() -> void:
    goto(CONTINUE if Game.credits > 0 else TITLE)

func to_title() -> void:
    goto(TITLE)
```

- `_busy` : un second appel pendant une transition est ignoré. Sans lui, deux appuis rapides sur Start lanceraient deux fondus entremêlés.
- Les chemins des écrans sont des constantes : on les écrit à un seul endroit.

L'AUDIO

10.3 · Bus audio et autoload

Audio



À FAIRE

1. Ouvre le panneau **Audio** (en bas). **Ajouter un bus** deux fois : renomme-les **Music** et **SFX**. Leur sortie (*Send*) est **Master**.
2. Facultatif : sur **Music**, ajoute un effet **Compressor** avec **Sidechain** = **SFX** : la musique baisse sous les grosses explosions (*ducking*).
3. Crée **res://autoload/audio.gd** et ajoute-le en autoload **Audio**. Ordre final des autoloads : **Game**, **PaletteFx**, **Audio**, **SceneFlow**.

res://autoload/audio.gd (Autoload « Audio »)

```
extends Node
```

```
## Musique persistante entre les scènes, bruitages « fire and forget ».
```

```
var _music := AudioStreamPlayer.new()
```

```
func _ready() -> void:
```

```
    process_mode = Node.PROCESS_MODE_ALWAYS
```

```
    _music.bus = &"Music"
```

```
    add_child(_music)
```

```
func play_music(stream: AudioStream) -> void:
```

```
    if stream == null or (_music.stream == stream and _music.playing):
```

```
        return # même morceau : on ne le relance pas (réapparition, continue...)
```

```
    _music.stream = stream
```

```
    _music.play()
```

```
func stop_music() -> void:
```

```
    _music.stop()
```

```
func sfx(stream: AudioStream, pitch_jitter := 0.04) -> void:
```

```
    if stream == null:
```

```
        return
```

```
    var p := AudioStreamPlayer.new()
```

```
    p.stream = stream
```

```
    p.bus = &"SFX"
```

```
    p.pitch_scale = 1.0 + randf_range(-pitch_jitter, pitch_jitter)
```

```
    add_child(p)
```

```
    p.finished.connect(p.queue_free)
```

```
    p.play()
```

POURQUOI UN AUTOLOAD POUR LA MUSIQUE

Un lecteur de musique placé dans une section serait détruit au changement de scène : la musique se couperait à chaque fondu. Dans un autoload, elle survit et enchaîne. `pitch_jitter` varie légèrement la hauteur de chaque bruitage : dix coups de poing identiques paraissent mécaniques, dix coups légèrement différents paraissent vivants.

La section reçoit sa musique. Version finale de `section.gd` :

```
res://sections/section.gd

class_name Section
extends Node2D
## Base commune des 3 sections du niveau 1.

@export_file("*.tscn") var next_scene: String
@export var player: Player
@export var victory_delay := 3.0    ## durée de la pose de victoire (vidéo : ~3 s)
@export var music: AudioStream

func _ready() -> void:
    Game.current_section = scene_file_path    # point de reprise du « continue »
    player.died.connect(_on_player_died)
    Audio.play_music(music)

func _on_player_died() -> void:
    if Game.lose_life():
        player.respawn()
    else:
        SceneFlow.game_over()

## Appelé par l'arène du boss quand il est vaincu.
func boss_defeated(points: int) -> void:
    Game.add_score(points)
    player.celebrate()
    await get_tree().create_timer(victory_delay).timeout
    SceneFlow.goto(next_scene)    # SceneFlow s'occupe du fondu au noir puis du fondu d'entrée
```

MUSIQUE DE JESPER KYD

La bande-son originale est en synthèse FM : Godot ne lit pas les fichiers VGM. Pour un projet personnel, convertis tes propres enregistrements en OGG. Pour un projet publiable, compose ou utilise une musique libre de droits. Sans musique, `play_music(null)` ne fait rien : tout fonctionne.

LE HUD

10.4 · Un thème pour tout le jeu



LE THEME

Tous les nœuds `Control` (textes, boutons, barres) tirent leur apparence d'un `Theme`. En 320×224, la police par défaut de 16 px est énorme : on règle une fois pour toutes un thème global.

À FAIRE

1. Crée une ressource `Theme` : `res://ui/theme.tres`. Dans l'Inspecteur, **Default Font Size** = 8.
Facultatif : **Default Font** = une police pixel libre (par exemple « Press Start 2P », licence OFL).
2. ≡ **Projet** → **Paramètres du projet** → **GUI** → **Thème** → **Personnalisé (Custom)** = `theme.tres`.

10.5 · Le médaillon et la jauge, dessinés à la main



Le HUD original, décomposé. Chaque zone correspond à un nœud de `hud.tscn`.

CONCEPT : DESSINER SOI-MÊME AVEC `_DRAW()`

L'anneau de vie segmenté n'existe pas comme nœud tout fait. Un `Control` peut se dessiner lui-même : on surcharge `_draw()`, on y appelle `draw_arc()`, `draw_circle()`, `draw_rect()`, et on demande un nouveau dessin avec `queue_redraw()` quand la donnée change. Godot ne redessine pas à chaque image : seulement quand on le demande.

```

class_name LifeMedallion
extends Control

## Médaille : nombre de vies au centre, anneau de segments orange = santé.

@export var segments := 8 ## À CALIBRER (8 à 10 visibles dans la vidéo)
@export var radius := 11.0
@export var thickness := 4.0
@export var on_color := Color("ff8a2a")
@export var off_color := Color("3a1c14")
@export var core_color := Color("2f62d0")

var lives := 2:
    set(value):
        lives = value
        queue_redraw()
var _health := 8
var _max := 8

func set_health(current: int, maximum: int) -> void:
    _health = current
    _max = maxi(maximum, 1)
    queue_redraw()

func _draw() -> void:
    var c := size / 2.0
    var filled := ceili(float(_health) / _max * segments)
    var step := TAU / segments
    for i in segments:
        var a0 := -PI / 2.0 + i * step + 0.09
        var a1 := -PI / 2.0 + (i + 1) * step - 0.09
        draw_arc(c, radius, a0, a1, 6, on_color if i < filled else off_color, thickness)
    draw_circle(c, radius - thickness * 0.5 - 1.0, core_color)
    var font := get_theme_default_font()
    draw_string(font, c + Vector2(-8, 4), str(lives), HORIZONTAL_ALIGNMENT_CENTER, 16, 10)

```

res://ui/charge_bar.gd

```
class_name ChargeBar
extends Control
## Jauge de charge : se remplit seule, prend la couleur de l'arme, clignote quand elle est pleine.

var color := Color("ff6b3d"):
    set(value):
        color = value
        queue_redraw()
var _ratio := 0.0

func set_ratio(ratio: float) -> void:
    _ratio = ratio
    queue_redraw()

func _process(_delta: float) -> void:
    if _ratio >= 1.0:
        queue_redraw() # clignotement

func _draw() -> void:
    draw_rect(Rect2(Vector2.ZERO, size), Color(0, 0, 0, 0.55))
    var w := floorf(size.x * _ratio)
    var c := color
    if _ratio >= 1.0 and (Time.get_ticks_msec() / 80) % 2 == 0:
        c = color.lightened(0.5)
    draw_rect(Rect2(Vector2.ZERO, Vector2(w, size.y)), c)
```

LES PROPRIÉTÉS AVEC SET

`var lives := 2: set(value): ...` déclare un **setter** : chaque fois qu'on écrit `medallion.lives = 3`, la fonction s'exécute. Ici, elle demande un nouveau dessin. L'extérieur n'a pas à savoir qu'il faut redessiner.

10.6 · hud.tscn et hud.gd



res://ui/hud.gd

```
class_name Hud
extends CanvasLayer
## HUD fidèle à l'original. N'est connu de personne : il écoute Game, le joueur et les boss.

@onready var portrait: TextureRect = $Root/Portrait
@onready var medallion: LifeMedallion = $Root/Medallion
@onready var score_label: Label = $Root/Score
@onready var charge: ChargeBar = $Root/Charge
@onready var boss_badge: Control = $Root/BossBadge
@onready var boss_value: Label = $Root/BossBadge/Value
@onready var p2_label: Label = $Root/P2
var _boss: Boss
var _t := 0.0
```

```

func _ready() -> void:
    add_to_group("hud")
    boss_badge.hide()
    Game.score_changed.connect(func(s: int) -> void: score_label.text = str(s))
    Game.lives_changed.connect(func(l: int) -> void: medallion.lives = l)
    score_label.text = str(Game.score)
    medallion.lives = Game.lives
    await get_tree().process_frame # laisse le joueur finir son _ready
    _bind_player(get_tree().get_first_node_in_group("player") as Player)

func _bind_player(player: Player) -> void:
    if player == null:
        return
    if player.hero:
        portrait.texture = player.hero.portrait
    player.health.changed.connect(medallion.set_health)
    medallion.set_health(player.health.current, player.health.max_health)
    var weapons := player.weapons as WeaponSystem
    if weapons:
        weapons.charge_changed.connect(charge.set_ratio)
        weapons.equipped.connect(func(d: WeaponData, _l: int) -> void: charge.color = d.color)
        if weapons.data:
            charge.color = weapons.data.color

## Appelée par Boss.start() via call_group.
func show_boss(boss: Boss) -> void:
    _boss = boss
    boss.vitality_changed.connect(_on_vitality)
    _on_vitality(boss.vitality())
    boss_badge.show()

## Appelée par Boss à sa mort via call_group.
func hide_boss() -> void:
    if _boss and _boss.vitality_changed.is_connected(_on_vitality):
        _boss.vitality_changed.disconnect(_on_vitality)
    _boss = null
    boss_badge.hide()

func _on_vitality(value: int) -> void:
    boss_value.text = "%02d" % value

func _process(delta: float) -> void:
    # Zone du joueur 2 : messages alternés chaque seconde (vidéo).
    _t += delta
    var messages: Array[String]
    if boss_badge.visible:
        messages = ["BOSS TIME", "PLEASE WAIT"]
    else:
        messages = ["PRESS START", "CREDIT %02d" % Game.credits, "GAME OVER"]
    p2_label.text = messages[int(_t) % messages.size()]

```

À FAIRE

1. Nouvelle scène, racine `CanvasLayer` nommée `Hud`, **Layer** = 10, script `hud.gd`. Enregistre en `res://ui/hud.tscn`.
2. Enfant `Control` nommé `Root` : **Ancres** → **Rect complet**, **Mouse** → **Filter** = Ignore.
3. Sous `Root`, place les nœuds aux positions et tailles indiquées (Inspecteur → **Layout** → **Transform**) :

```
Hud (CanvasLayer, hud.gd)          layer = 10
├─ Root (Control, plein écran, mouse_filter = Ignore)
│   ├─ Portrait (TextureRect)      (2, 2)    24×24 : tête du héros
│   ├─ Medallion (Control)         (26, 1)   26×26 : life_medallion.gd
│   ├─ Score (Label)              (56, 0)
│   ├─ Charge (Control)           (56, 12)  52×6 : charge_bar.gd
│   ├─ BossBadge (Control)        (146, 2)  28×12
│   │   └─ ColorRect              28×12, rouge sombre : la pastille
│   │       └─ Value (Label)      « 99 », centré
│   └─ P2 (Label)                 (246, 2)  PRESS START / CREDIT 06 / GAME OVER
```

4. Instancie `hud.tscn` dans `s1_streets.tscn` (n'importe où dans l'arbre : un `CanvasLayer` s'affiche toujours par-dessus le monde).

TESTE

F5 : le HUD affiche le médaillon (anneau orange plein, « 2 » au centre), le score, la jauge colorée qui se remplit quand tu ne tires pas, et « PRESS START / CREDIT 06 / GAME OVER » qui alternent en haut à droite. Prends un coup : l'anneau perd des segments. Ramasse un bonus vert : la jauge devient verte. Au boss : la pastille apparaît avec « 99 », descend sous tes tirs, et la zone de droite affiche « BOSS TIME / PLEASE WAIT ».

CONCEPT : UN HUD QUE PERSONNE NE CONNAÎT

Ni Batman, ni `Game`, ni le boss ne savent que le HUD existe. Le HUD **écoute** : `Game.score_changed`, `player.health.changed`, `weapons.charge_changed`, et répond aux appels de groupe du boss. Supprime le HUD de la section : le jeu tourne exactement pareil. C'est le découplage par signaux poussé jusqu'au bout.

LES ÉCRANS

10.7 · L'intro, une frise temporelle



L'intro, plan par plan. Chaque plan est une piste de l' `AnimationPlayer`.

CONCEPT : UNE CINÉMATIQUE EST UNE ANIMATION

Un `AnimationPlayer` peut animer n'importe quelle propriété (position, visibilité, couleur) et **appeler des méthodes** à un instant précis. On construit toute l'intro sur une frise temporelle, à la souris, sans aucun code de minuterie. Le découpage complet de la vraie intro est dans le [tome 10](#) ; ici, on fait une version courte avec des textes.

res://screens/intro.gd

extends Control

Cinématique d'intro (une animation « intro » dans l'AnimationPlayer). Start = passer.

@onready var anim: AnimationPlayer = \$AnimationPlayer

func _ready() -> void:

anim.play(&"intro")

anim.animation_finished.connect(**func**(_n: StringName) -> void: SceneFlow.to_title())

func _unhandled_input(event: InputEvent) -> void:

if event.is_action_pressed("start") **or** event.is_action_pressed("attack"):
SceneFlow.to_title()

Appelée par une piste « Appel de méthode » de l'animation.

func lightning() -> void:

PaletteFx.flash_white(2, 5)

À FAIRE

1. Nouvelle scène, racine **Control** nommée **Intro** (Rect complet), script **intro.gd**, enregistrée en **res://screens/intro.tscn**.
2. Enfants : un **ColorRect** noir plein écran, puis trois **Label** centrés nommés **Plan1** (« SEGA »), **Plan2** (« ARKHAM ASYLUM »), **Plan3** (« CLOCKWORK TORTOISE »), tous cachés. Et un **AnimationPlayer**.
3. Sélectionne l' **AnimationPlayer** : le panneau **Animation** s'ouvre. **Animation** → **Nouveau** : **intro**, longueur 9 s.
4. **Pistes de propriété** : sélectionne **Plan1**, place la tête de lecture à 0 s, coche **Visible** dans l'Inspecteur et clique sur l'icône de clé à côté : une clé est créée. À 2,5 s, décoche et crée une clé. Fais de même pour **Plan2** (3 → 6 s) et **Plan3** (6,5 → 9 s).
5. **Piste d'appel de méthode** : + **Ajouter une piste** → **Piste d'appel de méthode**, choisis le nœud **Intro**. Clic droit sur la piste à 3,5 s → **Insérer une clé** → méthode **lightning**. Ajoute-en une deuxième à 4,2 s.

TESTE

F6 sur l'intro : les trois plans s'enchaînent, deux éclairs (flashes de palette) frappent pendant « ARKHAM ASYLUM ». À la fin, la console signale que **title.tscn** n'existe pas encore : c'est la prochaine étape. **J** ou **Entrée** passe l'intro.

10.8 · L'écran titre



DANS LA VIDÉO

Le menu ne propose que **START** et **OPTIONS**, superposés au logo (**▶ 1:28**). Sans action, le jeu repart en boucle sur l'intro (**▶ 1:04**) : c'est l'*attract mode* des bornes d'arcade, qui montrait le jeu aux passants.

res://screens/title.gd

extends Control

Logo + menu START / OPTIONS. Retour à l'intro après un délai d'inactivité (attract mode).

@export var idle_to_attract := 20.0

@onready var anim: AnimationPlayer = \$AnimationPlayer ## animation « reveal » : yeux puis logo

@onready var menu: VBoxContainer = \$Menu

@onready var start_button: Button = \$Menu/Start

@onready var options_button: Button = \$Menu/Options

var _idle := 0.0

func _ready() -> void:

menu.hide()

start_button.pressed.connect(func() -> void: SceneFlow.goto("res://screens/player_select.tscn"))

options_button.pressed.connect(func() -> void: SceneFlow.goto("res://screens/options.tscn"))

anim.play(&"reveal")

await anim.animation_finished

menu.show()

start_button.grab_focus() # indispensable pour naviguer à la manette

func _process(delta: float) -> void:

_idle += delta

if _idle >= idle_to_attract:

_idle = -INF # une seule fois

SceneFlow.goto(SceneFlow.INTRO)

func _input(event: InputEvent) -> void:

if event.is_pressed():

_idle = 0.0

À FAIRE

- Scène `res://screens/title.tscn` : racine `Control` `Title` (Rect complet), script `title.gd`.
Enfants : `ColorRect` noir plein écran, `Label` `Logo` (« BATMAN & ROBIN », grand, centré, **Modulate** alpha 0), `VBoxContainer` nommé `Menu` (centré en bas) avec deux `Button` nommés `Start` et `Options` (textes « START », « OPTIONS », **Flat** coché), et un `AnimationPlayer`.
- Animation `reveal` de 1,5 s : une piste sur `Logo:modulate`, alpha 0 à 0 s → alpha 1 à 1,5 s. **Elle est obligatoire** : le script attend sa fin pour afficher le menu.
- ≡ **Projet** → **Paramètres du projet** → **Contrôles** : coche **Afficher les actions intégrées**, trouve `ui_accept` et ajoute-lui le bouton **Start** de la manette. « Start » validera ainsi les menus, comme sur la console.

LE FOCUS, LA SOURIS DE LA MANETTE

Les boutons Godot réagissent à `ui_accept` quand ils ont le **focus**. Sans `grab_focus()`, personne ne l'a et la manette ne fait rien. Avec le focus, les flèches passent d'un bouton à l'autre automatiquement : Godot déduit les voisins de la position des boutons.

10.9 · Les options et un réglage qui persiste



D'abord, `Game` apprend à lire et écrire le réglage des crédits. Ajoute à `game.gd` :

res://autoload/game.gd (ajouts)

```
const OPTIONS_PATH := "user://options.cfg"

func _ready() -> void:
    var cfg := ConfigFile.new()
    if cfg.load(OPTIONS_PATH) == OK:
        credits_setting = cfg.get_value("game", "credits", START_CREDITS)
        credits = credits_setting

func save_options() -> void:
    var cfg := ConfigFile.new()
    cfg.set_value("game", "credits", credits_setting)
    cfg.save(OPTIONS_PATH)
```

res://screens/options.gd

```
extends Control
## CREDITS / MUSIC TEST / SOUND TEST, comme l'original (vidéo 1:30). Start = sortir.
## Haut/bas : choisir la ligne ; gauche/droite : changer la valeur ; attaque : jouer le son.

@export var music_tracks: Array[AudioStream] = []
@export var sound_effects: Array[AudioStream] = []
@onready var rows: Array[Label] = [$List/Credits, $List/Music, $List/Sound]
var _row := 0
var _credits := 0
var _music := 1
var _sound := 1

func _ready() -> void:
    _credits = Game.credits_setting # Game a déjà lu user://options.cfg au démarrage
    _refresh()

func _unhandled_input(event: InputEvent) -> void:
    if event.is_action_pressed("move_up"):
        _row = posmod(_row - 1, rows.size())
    elif event.is_action_pressed("move_down"):
        _row = posmod(_row + 1, rows.size())
    elif event.is_action_pressed("move_left"):
        _change(-1)
    elif event.is_action_pressed("move_right"):
        _change(1)
    elif event.is_action_pressed("attack"):
        _play()
    elif event.is_action_pressed("start"):
        _save()
        SceneFlow.to_title()
        return
    else:
```

```

return
_refresh()

func _change(d: int) -> void:
    match _row:
        0: _credits = clampi(_credits + d, 1, 9)
        1: _music = clampi(_music + d, 1, maxi(music_tracks.size(), 1))
        2: _sound = clampi(_sound + d, 1, maxi(sound_effects.size(), 1))

func _play() -> void:
    if _row == 1 and _music <= music_tracks.size():
        Audio.play_music(music_tracks[_music - 1])
    elif _row == 2 and _sound <= sound_effects.size():
        Audio.sfx(sound_effects[_sound - 1], 0.0)

func _refresh() -> void:
    rows[0].text = "CREDITS: %d" % _credits
    rows[1].text = "MUSIC TEST: %d" % _music
    rows[2].text = "SOUND TEST: %d" % _sound
    for i in rows.size():
        rows[i].modulate = Color.WHITE if i == _row else Color(0.55, 0.55, 0.7)

func _save() -> void:
    Game.credits_setting = _credits    # utilisé par Game.new_game() à la prochaine partie
    Game.save_options()

```

À FAIRE

Scène `res://screens/options.tscn` : racine `Control` (script `options.gd`), fond noir, un `VBoxContainer` nommé `List` centré, avec trois `Label` nommés `Credits`, `Music`, `Sound`.

CONCEPT : UNE DONNÉE, UN PROPRIÉTAIRE

L'écran des options ne modifie pas directement `Game.credits` : la valeur serait écrasée par `new_game()` au lancement de la partie. On distingue le **réglage** (`credits_setting`, enregistré sur le disque) de l'**état de la partie** (`credits`, consommé par les continues). Le réglage a un seul propriétaire, `Game`, qui sait le lire, l'écrire et l'appliquer.

`USER://` ET `CONFIGFILE`

`res://` est en lecture seule une fois le jeu exporté. `user://` désigne un dossier propre au jeu dans le profil de l'utilisateur (sur macOS : `~/Library/Application Support/Godot/app_userdata/<projet>/`). `ConfigFile` lit et écrit des fichiers texte au format INI : idéal pour des réglages. **≡** **Projet → Ouvrir le dossier de données utilisateur** t'y emmène.

DANS LA VIDÉO

« PLAYER SELECT » : l'écran est coupé en deux bandes, bleue en haut avec Batman qui court, rouge en bas avec Robin (▶ 1:33 — ▶ 1:38).

res://screens/player_select.gd

```
extends Control
## Deux bandes : Batman (haut), Robin (bas). Haut/bas pour choisir, Start pour valider.

@onready var batman_band: Control = $Batman
@onready var robin_band: Control = $Robin
@onready var batman_runner: AnimatedSprite2D = $Batman/Runner
@onready var robin_runner: AnimatedSprite2D = $Robin/Runner
var _choice := Game.Hero.BATMAN # valeur de l'enum déclaré dans game.gd

func _ready() -> void:
    _refresh()

func _unhandled_input(event: InputEvent) -> void:
    if event.is_action_pressed("move_up") or event.is_action_pressed("move_down"):
        _choice = Game.Hero.ROBIN if _choice == Game.Hero.BATMAN else Game.Hero.BATMAN
        _refresh()
    elif event.is_action_pressed("start") or event.is_action_pressed("attack"):
        Game.new_game(_choice)
        SceneFlow.goto("res://screens/batmobile.tscn")

func _refresh() -> void:
    var is_batman := _choice == Game.Hero.BATMAN
    batman_runner.play(&"run" if is_batman else &"idle")
    robin_runner.play(&"run" if not is_batman else &"idle")
    batman_band.modulate = Color.WHITE if is_batman else Color(0.45, 0.45, 0.45)
    robin_band.modulate = Color.WHITE if not is_batman else Color(0.45, 0.45, 0.45)
```

À FAIRE

- Scène `res://screens/player_select.tscn` : racine `Control` (script). Deux `ColorRect` nommés `Batman` (bleu, moitié haute) et `Robin` (rouge, moitié basse), chacun avec un `Label` (« BATMAN », « ROBIN ») et un `AnimatedSprite2D` nommé `Runner`.
- Chaque `Runner` a besoin des animations `run` et `idle`. Avec les formes de substitution : duplique `placeholder_frames.tres` en `runner_placeholder.tres`, renomme `default` en `idle` et ajoute une animation `run` avec la même image.

TESTE

Haut/bas : la bande choisie s'éclaire, l'autre s'assombrit. Valider appelle `Game.new_game()` : le héros est mémorisé, les crédits sont remis au réglage des options, et `player.gd` chargera `robin.tres` si tu as choisi Robin (étape 05).

Batmobile, carte-titre et fin de niveau : trois écrans fixes de quelques secondes. Un seul script suffit.

```
res://screens/interlude.gd
```

```
extends Control
```

```
## Écran fixe ou animé : Batmobile (1:39), carte-titre « LEVEL ONE » (1:43), fin de niveau (14:37).
```

```
## Passe à next_scene après « duration » secondes, ou sur Start.
```

```
@export_file("*.tscn") var next_scene: String
```

```
@export var duration := 3.0
```

```
@export var music: AudioStream
```

```
var _t := 0.0
```

```
var _leaving := false
```

```
func _ready() -> void:
```

```
    if music:
```

```
        Audio.play_music(music)
```

```
func _process(delta: float) -> void:
```

```
    _t += delta
```

```
    var skip := _t > 0.5 and Input.is_action_just_pressed("start")
```

```
    if not _leaving and (_t >= duration or skip):
```

```
        _leaving = true
```

```
        SceneFlow.goto(next_scene)
```

Scène	Contenu	next_scene
res://screens/batmobile.tscn	La Batmobile file dans la nuit (ou un <code>Label</code> « BATMOBILE » en attendant)	res://screens/title_card.tscn
res://screens/title_card.tscn	« LEVEL ONE / HAPPY BIRTHDAY TO ME! » (planche « Title Cards » du dépôt, via une <code>AtlasTexture</code>)	res://sections/s1_streets.tscn
res://screens/level_end.tscn	Batmobile, puis « À suivre : niveau 2 »	res://screens/title.tscn

res://screens/continue.gd

extends Control

« CONTINUE ? 9... 0 ». Start : consomme un crédit et reprend la section en cours.

@onready var label: Label = \$Label

var _count := 9.99

func _process(delta: float) -> void:

_count -= delta

label.text = "CONTINUE ? %d\nCREDIT %02d" % [maxi(int(_count), 0), Game.credits]

if _count <= 0.0:

set_process(false)

SceneFlow.to_title()

func _unhandled_input(event: InputEvent) -> void:

if event.is_action_pressed("start") **and** Game.continue_game():

set_process(false)

SceneFlow.goto(Game.current_section)

À FAIRE

1. Crée les trois scènes d'interlude : racine Control plein écran avec interlude.gd, un fond noir et un Label ou un TextureRect centré. Renseigne **Next Scene** selon le tableau.
2. Crée res://screens/continue.tscn : Control + fond noir + un Label nommé Label, centré.
3. Dans s1_streets.tscn, mets enfin **Next Scene** = res://screens/level_end.tscn : en attendant les sections 2 et 3, finir le boss termine le « niveau ».

NON OBSERVÉ

La partie analysée ne contient aucun game over : la présentation exacte de l'écran de continue est inconnue. Le décompte de 9 à 0 est une convention de l'époque, pas une observation.

10.12 · Le jeu de bout en bout



À FAIRE

1. ≡ Projet → Paramètres du projet → Application → Exécuter → Scène principale = res://screens/intro.tscn.
2. F5 et joue tout le parcours.

TESTE

- Intro (Start pour passer) → titre → START → sélection → Batmobile → carte-titre → section 1, avec des fondus en paliers partout.
- Sans toucher au titre pendant 20 s, l'intro repart.
- Options : passe les crédits à 3, sors avec Start, relance le jeu : le réglage est conservé. Lance une partie : « CREDIT 03 » dans la zone de droite du HUD.
- Perds toutes tes vies : écran de continue ; Start reprend la section (crédit -1, score remis à zéro). Laisse le décompte finir : retour au titre.
- Bats la machine d'Harley : Batmobile de fin, puis retour au titre.

MÉTHODE : LA TRANCHE VERTICALE

Tu as maintenant une **tranche verticale** (*vertical slice*) : une petite partie du jeu, mais complète de haut en bas, de l'intro au générique, avec toutes ses couches (menus, HUD, gameplay, effets, son). C'est le jalon qu'un studio présente pour prouver qu'un jeu fonctionne. Les sections 2 et 3 ne sont plus « le jeu » : ce sont du contenu ajouté à un jeu qui marche déjà.

POINT DE CONTRÔLE

TU AS FINI L'ÉTAPE QUAND

- ☐ Au lancement : intro → titre → sélection → Batmobile → carte-titre → section 1, avec des fondus en paliers.
- ☐ Le HUD affiche le portrait, l'anneau de vie, les vies, le score, la jauge colorée, et la pastille 99 → 00 pendant le boss.
- ☐ Le réglage des crédits survit au redémarrage et s'applique à la partie suivante.
- ☐ En perdant toutes les vies : écran de continue ; Start reprend la section tant qu'il reste des crédits.
- ☐ Après le boss : écran de fin, puis retour au titre.

```
git git add -A && git commit -m "Étape 10 : HUD, écrans, flux complet, audio"
```

CE QUE TU RETIENS

? Pourquoi le HUD attend-il une image avant de se brancher sur le joueur ?

Le HUD et le joueur sont dans la même scène, et l'ordre de leurs `_ready()` dépend de leur place dans l'arbre. En attendant une image (`await get_tree().process_frame`), le HUD est sûr que le joueur a fini son `_ready()` (fiche du héros chargée, système d'armes en place).

? Pourquoi le menu titre appelle-t-il `grab_focus()` ?

Parce qu'un bouton ne réagit au clavier ou à la manette que s'il a le focus. Sans focus initial, aucune touche ne fait rien ; avec, les flèches naviguent et `ui_accept` valide.

? Quelle différence entre `credits` et `credits_setting` ?

`credits_setting` est un réglage choisi dans les options et enregistré sur le disque. `credits` est l'état de la partie en cours, initialisé depuis le réglage par `new_game()` et consommé par les continues.

? Qu'apporte une piste d'appel de méthode dans un `AnimationPlayer` ?

Elle appelle une fonction à un instant précis de la frise (un flash, un son, un changement de plan). La cinématique entière se règle visuellement, sans minuteries dans le code.

POUR ALLER PLUS LOIN

- Enrichis l'intro avec les plans du [tome 10](#) : fourgon de police, façade en contre-plongée, moniteurs, main gelée.
- Ajoute un bruitage aux tirs : un `@export var shot_sound: AudioStream` dans `WeaponSystem` et `Audio.sfx(shot_sound)` dans `try_fire()`.
- Ajoute un écran de pause : `get_tree().paused = true`, et un menu dont le `process_mode` est `ALWAYS`.

SECTION 2 : GEM EXPO ET GRAPPIN

Une exposition de bijoux construite en salles réutilisables, un entrepôt peint en tuiles, une apparition d'Harley pour annoncer la suite, puis le morceau de bravoure technique du niveau : le grappin de grue en sphères qui plonge dans la profondeur.

~5 h

Prérequis : étape 10

Guide : tome 08

CE QUE TU VAS APPRENDRE

GODOT 4

- Scènes comme modules de niveau
- `Parallax2D` en pratique
- `TileSet` et `TileMapLayer` avec collisions
- `Tween` chaînés et parallèles
- `z_index`, `Vector3` en 2D

DÉVELOPPEMENT DE JEU

- Construire avec des modules
- L'annonce d'un boss
- La projection perspective
- L'algorithme du peintre
- La vulnérabilité liée à la profondeur

LE GEM EXPO

11.1 · Observer la répétition



Le Gem Expo est fait de salles réutilisées (modules)



Des salles réutilisées. La salle « bannière » revient à l'identique ; elle alterne avec la salle « baies vitrées ». Chaque salle est une scène instanciée plusieurs fois.

DANS LA VIDÉO

La même salle revient plusieurs fois : colonnes de marbre, bannière « GEM EXPO », vitrines, plantes en pot (▶ 6:24 , ▶ 7:28 , ▶ 8:32). Elle alterne avec une salle à grandes baies vitrées sur la ville (▶ 7:51).

CONCEPT : CONSTRUIRE AVEC DES MODULES

Une cartouche de 1995 n'avait pas la place pour un décor unique de plusieurs milliers de pixels : les niveaux étaient faits de blocs et de salles réutilisés. Dans Godot, une salle est une **scène** qu'on instancie bout à bout. Corriger un détail dans la salle le corrige partout. Les arènes et les ennemis restent dans la scène de section : chaque instance de salle peut accueillir un combat différent.

11.2 · Deux salles



À FAIRE

1. **Salle « bannière »** : nouvelle scène 2D `res://sections/s2/expo_hall.tscn`, racine `ExpoHall`. Elle fait 640 px de large. Contenu, en formes de substitution :
 - un mur (`ColorRect` 640×200, beige) de y = 0 à 200 ;
 - trois colonnes (`ColorRect` 32×200, blanc cassé) à x = 80, 300, 520 ;
 - une bannière (`Label` « GEM EXPO » sur un `ColorRect` rouge) en haut ;
 - deux vitrines basses : chacune un `StaticBody2D` (couche 2, forme one-way 48×8) avec un `ColorRect` vitré de 48×24, posées sur le sol.
2. **Salle « baies vitrées »** : `≡ Scène → Enregistrer la scène sous → expo_windows.tscn`. Remplace le mur plein par deux murs de part et d'autre d'une grande ouverture (x = 160 à 480, y = 40 à 160) : on verra la ville à travers. Dans l'ouverture, deux vitres `Breakable` (Health 1, Hurtbox couche 6, **Broken Texture** = une image de vitre étoilée, ou rien).

MÉTHODE : UN SEUL SOL POUR TOUTE LA SECTION

Les salles ne contiennent **pas** de sol. Mettre bout à bout des sols séparés crée des jointures sur lesquelles un personnage peut accrocher. Le sol est un seul long `StaticBody2D` dans la scène de section ; les salles n'apportent que le décor et les meubles.

LES COLONNES

Mesure du guide : les colonnes défilent exactement à la vitesse du mur et Batman passe devant elles. Ce sont des éléments du décor, au même plan que le mur : pas besoin de couche à part, ni de parallaxe. Mesurer avant de coder évite d'ajouter un effet qui n'existe pas.

11.3 · La section et la skyline



À FAIRE

1. Nouvelle scène 2D `res://sections/s2_gem_expo.tscn`, racine `S2GemExpo` (script `section.gd`), en (0, 0).
2. Construis l'arbre. **Skyline** doit être le **premier enfant** : les nœuds sont dessinés dans l'ordre de l'arbre, donc elle sera derrière les salles.

```
S2GemExpo (Node2D, section.gd)
├── Skyline (Parallax2D)
│   └── Buildings (Node2D)
├── World (Node2D)
│   ├── Ground (StaticBody2D)
│   ├── Hall1 (expo_hall.tscn)
│   ├── Windows1 (expo_windows.tscn)
│   ├── Hall2 (expo_hall.tscn)
│   ├── Windows2 (expo_windows.tscn)
│   ├── Hall3 (expo_hall.tscn)
│   ├── ArenaA, ArenaB (WaveArena)
│   ├── Warehouse (Node2D)
│   ├── HarleyFlyby (Area2D)
│   └── BossArena → CraneHarley
└── EnemyBullets (Pool) · Player · Camera (camera_rig.gd) · Hud
```

next_scene = level_end.tscn (provisoire) · player = Player
scroll_scale (0.3, 0) · repeat_size (640, 0)
quelques ColorRect sombres de hauteurs variées, sur 640 px
couche 1 · rectangle 4400×16 en (2200, 208)
x = 0
x = 640
x = 1280
x = 1920
x = 2560
dans les salles
à partir de x = 3200 (11.5)
(11.6)
(11.8)

3. Dans la section 1, remplace **Next Scene** par `res://sections/s2_gem_expo.tscn`.

TESTE

F6 sur la section 2 : en courant, les salles défilent avec le sol, et la ville, visible par les baies, défile trois fois moins vite. Les vitres se brisent sous les tirs. Monte sur les vitrines.

11.4 · Les vagues de têtes à hélice



Vague de têtes à hélice. ▶ 6:58

À FAIRE

- Dans `ArenaA` (vers $x = 450$), place deux `Formation` enfants :
 - « **Chenille** » : une courbe qui descend à la verticale depuis le haut de l'écran, puis serpente vers la gauche. 8 membres, `delay` 0,3 s.
 - « **Ligne** » : une courbe horizontale en haut de l'écran, de droite à gauche. 6 membres, `delay` 0,25 s.
- Vague 1 de `ArenaA` : quelques clowns (quota 4) + **Formation** = Chenille. Vague 2 : quota 4 + **Formation** = Ligne.
- `ArenaB` (vers $x = 2\ 000$) : clowns, costauds et tireurs, comme une arène de la section 1.

TESTE

La chenille descend puis serpente, la ligne traverse le haut de l'écran, et toutes deux bombardent pendant que les clowns arrivent. La première vague lance sa formation dès l'entrée dans l'arène sans erreur dans la console : c'est le démarrage différé de `WaveArena` (étape 07) qui le permet.

L'ENTREPÔT

11.5 · Peindre des caisses avec une `TileMapLayer`



L'entrepôt. Les pyramides de caisses sont des blocs pleins à franchir en sautant.

Une `TileMapLayer` peint une grille de tuiles prises dans un `TileSet`. Chaque tuile peut porter une forme de collision : peindre une caisse, c'est aussi créer son sol et ses murs. C'est l'outil standard pour construire des niveaux 2D (et l'héritier direct des métatuiles de la Mega Drive). `TileMapLayer` remplace l'ancien `TileMap` depuis Godot 4.3.

D'abord, des images de test pour cette section : des caisses, et les sphères du grappin (11.7).

```
res://tools/make_debug_s2.gd
```

```
@tool
extends EditorScript

## Génère les images de test de la section 2 :
## - crates.png : 2 tuiles 16x16 (caisse claire, caisse bleue) pour la TileMapLayer ;
## - sphere_orange.png : 4 tailles de sphère orange (bande de 4 cases de 20x20) ;
## - sphere_dark.png : 2 tailles de sphère sombre (bande de 2 cases de 12x12).

const DIR := "res://art/s2/"

func _run() -> void:
    DirAccess.make_dir_recursive_absolute(DIR)
    _crates().save_png(DIR + "crates.png")
    _spheres(20, [4.0, 6.0, 8.0, 9.5], Color("e0782c")).save_png(DIR + "sphere_orange.png")
    _spheres(12, [3.5, 5.5], Color("3a3550")).save_png(DIR + "sphere_dark.png")
    EditorInterface.get_resource_filesystem().scan()
    print("Images de la section 2 écrites dans ", DIR)

func _crates() -> Image:
    var img := Image.create_empty(32, 16, false, Image.FORMAT_RGBA8)
    var colors := [Color("b07a3c"), Color("3c5ea8")]
    for t in 2:
        for y in 16:
            for x in 16:
                var edge := x == 0 or y == 0 or x == 15 or y == 15 or x == y or x == 15 - y
                var c: Color = colors[t]
                img.set_pixel(t * 16 + x, y, c.darkened(0.4) if edge else c)
    return img

func _spheres(cell: int, radii: Array, base: Color) -> Image:
    var img := Image.create_empty(cell * radii.size(), cell, false, Image.FORMAT_RGBA8)
    for i in radii.size():
        var r: float = radii[i]
        var center := Vector2(i * cell + cell / 2.0, cell / 2.0)
        for y in cell:
            for x in cell:
                var p := Vector2(i * cell + x + 0.5, y + 0.5)
                if p.distance_to(center) <= r:
                    # Reflet en haut à gauche : plus clair près de ce point.
                    var hl := p.distance_to(center + Vector2(-r, -r) / 3.0) / r
                    img.set_pixel(i * cell + x, y, base.lightened(clampf(0.6 - hl, 0.0, 0.6)))
    return img
```

À FAIRE

1. Exécute `make_debug_s2.gd`.
2. Sous `Warehouse`, ajoute une `TileMapLayer` nommée `Crates`. Inspecteur → **Tile Set** → **Nouveau TileSet**, puis clique dessus : **Tile Size** = 16×16.
3. Toujours dans le `TileSet` : **Physics Layers** → **Ajouter un élément**. Règle sa **Collision Layer** sur la couche 1 seule (« monde »).
4. Panneau **TileSet** (en bas) : glisse `crates.png` dans la liste des sources et accepte la création automatique des tuiles.
5. Onglet **Sélectionner** du panneau TileSet, choisis une tuile → **Physics** → **Physics Layer 0** : dans l'éditeur de polygone, appuie sur `F` pour remplir toute la case. Recommence pour l'autre tuile.
6. Onglet **TileMap** du panneau : choisis une tuile et peins dans la vue 2D (clic gauche peint, clic droit efface). Dessine trois ou quatre pyramides de caisses posées sur le sol, de x = 3 200 à 3 900. Une case fait 16 px : le sol est à y = 200, donc la rangée du bas va de y = 184 à 200.

TESTE

Batman monte de marche en marche (une marche de 16 px se franchit en sautant), les clowns butent contre les caisses. Active les formes de collision visibles : chaque caisse a son carré.

11.6 · Harley passe : une annonce en pleine partie



DANS LA VIDÉO

À `9:25` – `9:29`, Harley traverse l'écran en diagonale dans son fauteuil-fusée, sans combattre. Le joueur sait désormais qui l'attend.

CONCEPT : ANNONCER UN BOSS

Montrer l'adversaire avant le combat crée de l'attente et donne du sens au parcours. C'est une technique de mise en scène vieille comme le cinéma, et presque gratuite à réaliser : un sprite et une trajectoire.

res://sections/s2/harley_flyby.gd

extends Area2D

Déclencheur (masque 3) : Harley traverse l'écran sans combattre (annonce du mini-boss).

@export var harley_scene: PackedScene *## juste un Sprite2D animé du fauteuil-fusée*

@export var duration := 3.0

var _done := false

func _ready() -> void:

body_entered.connect(_on_body_entered)

func _on_body_entered(body: Node2D) -> void:

if _done or not body is Player:

return

_done = true

var cam := (body as Player).camera.view_rect()

var h: Node2D = harley_scene.instantiate()

Fx.spawn(h, Vector2(cam.position.x - 30, cam.end.y - 60))

var tw := h.create_tween()

tw.tween_property(h, "global_position", Vector2(cam.end.x + 40, cam.position.y + 20), duration)

\

.set_trans(Tween.TRANS_SINE).set_ease(Tween.EASE_IN_OUT)

tw.tween_callback(h.queue_free)

À FAIRE

1. Crée `res://bosses/crane_harley/harley_flyby_sprite.tscn` : un `Sprite2D` (`white.png`, scale (1,5 ; 1,2), modulate rose).
2. Sous `World`, une `Area2D` `HarleyFlyby` (masque 3, forme 8×224) vers x = 3 500, script `harley_flyby.gd`, **Harley Scene** = la scène du sprite.

LE TWEEN

Un `Tween` anime une propriété d'une valeur à une autre sur une durée, avec une courbe d'accélération (`TRANS_SINE`, `EASE_IN_OUT` : départ et arrivée en douceur). Les appels s'enchaînent : `tween_callback` s'exécute une fois l'animation précédente terminée. Créé sur le nœud animé (`h.create_tween()`), il s'arrête avec lui.

TESTE

En arrivant vers x = 3 500, un rectangle rose traverse l'écran en diagonale, de bas en haut, en accélérant puis en ralentissant, et disparaît.

LE GRAPPIN EN SPHÈRES

11.7 · La projection perspective



Le grappin en sphères (secret #3) : la profondeur = taille + luminosité



$z \approx 0$: grand, clair, dangereux



z moyen : plus petit



z grand : petit, sombre, inoffensif

La profondeur du grappin, dans le jeu. Au premier plan, il est grand et clair, et il blesse ; plus il s'enfonce, plus il est petit et sombre, et il devient inoffensif.

CE QU'EXPLIQUE JON BURTON (SECRET #3)

- Des **sphères** : sous n'importe quel angle, une sphère reste un disque avec son reflet au même endroit. Pas besoin de rotation.
- Environ **20 sphères**, chacune un sprite.
- **4 tailles** de sphère orange et **2** de sphère noire, pré-calculées : pas de mise à l'échelle.
- Une vraie position 3D, projetée à l'écran.
- Quand le bras part vers le fond, il **s'assombrit**.

CONCEPT : DE LA 3D SUR UN ÉCRAN 2D

On décrit le bras en **3D** (x à droite, y vers le bas, z vers le fond). Pour l'afficher, on divise par la profondeur : un point deux fois plus loin paraît deux fois plus petit et deux fois plus proche du centre. Avec une « focale » f : $s = f / (f + z)$, puis $\text{écran} = (x, y) \times s$. Le facteur s donne aussi la taille apparente et la luminosité. C'est toute la « 3D » du jeu.

res://bosses/crane_harley/sphere_arm.gd

```
class_name SphereArm
extends Node2D

## Secret #3 : grappin de grue en sphères, pseudo-3D.
## Positions en 3D -> projection perspective -> image de la bonne taille + luminosité + tri.
```

```

enum S { TRACK, DROP, HOLD, RAISE, SWING }

@export var orange_sphere: Texture2D  ## bande horizontale de 4 tailles, de la plus petite à la plus grande
@export var dark_sphere: Texture2D    ## bande horizontale de 2 tailles
@export var chain_count := 10
@export var focal := 260.0
@export var ground_y := 150.0          ## y local du moyeu quand le grappin touche le sol
@export var raised_y := 36.0
@export var track_speed := 70.0
@export var drop_speed := 420.0
@export var raise_speed := 90.0
@export var swing_depth := 140.0
@export var target: Node2D

## 4 « palettes » de plus en plus sombres, comme la CRAM mise à jour en direct.
const BRIGHTNESS := [1.0, 0.8, 0.62, 0.45]

var hub := Vector3(0, 36, 0)  ## moyeu du grappin, relatif au point d'accroche
var open := 1.0               ## 1 = pince ouverte, 0 = fermée
var spin := 0.0
var _s := S.TRACK
var _t := 0.0
var _chain: Array[Sprite2D] = []
var _claw: Array[Sprite2D] = []  ## [0] = moyeu, puis 4 doigts × 3 sphères

@onready var claw_hitbox: Hitbox = $ClawHitbox

func _ready() -> void:
    for i in chain_count:
        _chain.append(_make_sphere(dark_sphere, 2))
    for i in 13:
        _claw.append(_make_sphere(orange_sphere, 4))
    claw_hitbox.monitoring = false
    _layout()

func _make_sphere(tex: Texture2D, sizes: int) -> Sprite2D:
    var s := Sprite2D.new()
    s.texture = tex
    s.hframes = sizes
    add_child(s)
    return s

func _physics_process(delta: float) -> void:
    _t += delta
    match _s:
        S.TRACK:
            # Se place au-dessus de Batman, pince ouverte, au premier plan.
            open = move_toward(open, 1.0, delta * 2.0)
            hub.y = move_toward(hub.y, raised_y, raise_speed * delta)
            hub.z = move_toward(hub.z, 0.0, 120.0 * delta)
            if target:
                hub.x = move_toward(hub.x, target.global_position.x - global_position.x, track_speed * delta)
            if _t > 1.6:

```

```

        _go(S.DROP)
S.DROP:
    hub.y = move_toward(hub.y, ground_y, drop_speed * delta)
    if hub.y >= ground_y:
        _impact()
        _go(S.HOLD)
S.HOLD:
    open = move_toward(open, 0.0, delta * 4.0) # la pince se referme
    if _t > 0.5:
        _go(S.RAISE)
S.RAISE:
    hub.y = move_toward(hub.y, raised_y, raise_speed * delta)
    if hub.y <= raised_y:
        _go(S.SWING if randf() < 0.5 else S.TRACK)
S.SWING:
    # Passe dans le fond : plus petit, plus sombre, inoffensif.
    hub.z = (1.0 - cos(_t * 1.6)) * 0.5 * swing_depth
    hub.x = sin(_t * 0.8) * 60.0
    if _t > TAU / 1.6:
        _go(S.TRACK)
spin += delta * 0.8
# La pince ne blesse que dans le plan du joueur, pendant la frappe.
claw_hitbox.monitoring = (_s == S.DROP or _s == S.HOLD) and absf(hub.z) < 12.0
_layout()

func _go(next: S) -> void:
    _s = next
    _t = 0.0
    if next == S.DROP:
        claw_hitbox.reset_hits() # chaque frappe peut toucher à nouveau

func _impact() -> void:
    Fx.explosion(to_global(project(hub + Vector3(0, 14, 0))))
    get_tree().call_group("camera", "shake", 0.35) # tremblement (tome 11), sans effet sinon

func _layout() -> void:
    # Chaîne : sphères sombres régulièrement réparties de l'accroche (0,0,0) au moyeu.
    for i in chain_count:
        var k := (i + 1.0) / (chain_count + 1.0)
        _place(_chain[i], Vector3.ZERO.lerp(hub, k), 2)
    # Moyeu, puis 4 doigts de 3 sphères qui s'écartent (ouvert) ou se referment.
    _place(_claw[0], hub, 4)
    for f in 4:
        var a := spin + f * TAU / 4.0
        for j in 3:
            var r := open * (j + 1) * 7.0 + (1.0 - open) * (3.0 - j * 2.0)
            var drop := (j + 1) * 6.0
            var p := hub + Vector3(cos(a) * r, drop, sin(a) * r)
            _place(_claw[1 + f * 3 + j], p, 4)
    claw_hitbox.position = project(hub + Vector3(0, 12, 0))

func project(p: Vector3) -> Vector2:
    var s := focal / (focal + p.z)
    return Vector2(p.x, p.y) * s

```

```

func _place(sprite: Sprite2D, p: Vector3, sizes: int) -> void:
    var s := focal / (focal + p.z)          # ~1 au premier plan, < 1 au fond
    sprite.position = Vector2(p.x, p.y) * s
    # Taille : l'image pré-calculée la plus proche (le VDP ne sait pas agrandir).
    var size01 := clampf(inverse_lerp(0.55, 1.12, s), 0.0, 1.0)
    sprite.frame = int(round(size01 * (sizes - 1)))
    # Luminosité : 4 paliers selon la profondeur (palette assombrie).
    var level := clampi(int(inverse_lerp(-20.0, swing_depth, p.z) * 4.0), 0, 3)
    var v: float = BRIGHTNESS[level]
    sprite.modulate = Color(v, v, v)
    # Tri : le plus proche est dessiné par-dessus (algorithme du peintre).
    sprite.z_index = -int(p.z)

```

Comprendre le code

- **Une seule vérité, la position 3D** : les états (`TRACK`, `DROP` ...) ne modifient que `hub` (un `Vector3`) et `open`. `_layout()` en déduit tout le reste à chaque image : position, taille, luminosité et ordre de dessin des 23 sphères.
- `_place()` fait trois choses avec le même facteur `s` : la **position** (projection), la **taille** (choix parmi les images pré-calculées, via `frame`), la **luminosité** (4 paliers, comme 4 palettes de plus en plus sombres).
- `z_index = -z` : **l'algorithme du peintre**. Sans tampon de profondeur, on dessine du plus lointain au plus proche. Godot trie les enfants par `z_index` ; sur la console, le programme triait la table des sprites à chaque image.
- `claw_hitbox.monitoring` n'est vrai que pendant la frappe **et** dans le plan du joueur ($|z| < 12$). Ce qui a l'air loin ne touche pas.
- `call_group("camera", "shake", ...)` : le tremblement n'existe qu'à l'étape 13. D'ici là, personne n'est dans le groupe `camera`, et l'appel ne fait rien.

11.8 · Harley, son fauteuil et le mini-boss



res://bosses/crane_harley/harley_chair.gd

```
class_name HarleyChair
extends Node2D
## Harley vole dans la moitié haute de l'écran et lobe des bombes sur Batman.

@export var bomb_scene: PackedScene          ## bomb.tscn, bounces = 0
@export var flight_area := Rect2(-120, 20, 240, 50) ## relatif au boss
@export var throw_interval := 1.5
@export var ko_texture: Texture2D            ## Harley assommée au sol
var player: Node2D
var _t := 0.0
var _throw := 1.0
@onready var sprite: Sprite2D = $Sprite2D

func tick(delta: float) -> void:
    _t += delta
    var c := flight_area.get_center()
    position = c + Vector2(sin(_t * 0.7) * flight_area.size.x / 2.0, sin(_t * 1.9) *
        flight_area.size.y / 2.0)
    if player:
        sprite.flip_h = player.global_position.x < global_position.x
    _throw -= delta
    if _throw <= 0.0 and bomb_scene and player:
        _throw = throw_interval
        var b := bomb_scene.instantiate() as Bomb
        b.velocity = Fx.ballistic(global_position, player.global_position + Vector2(0, -12),
            b.fall_gravity, 0.8)
        Fx.spawn(b, global_position)

func show_ko() -> void:
    rotation = 0.0
    if ko_texture:
        sprite.texture = ko_texture
```

```
res://bosses/crane_harley/crane_harley.gd
```

```
class_name CraneHarley
extends Boss

## Mini-boss de la section 2 (vidéo 9:48-10:43) : le grappin frappe, Harley est la cible.

@export var ground_local_y := 172.0 ## y local du sol, pour la chute d'Harley
@onready var arm: SphereArm = $Arm
@onready var harley: HarleyChair = $Harley

func _ready() -> void:
    super._ready()
    arm.target = player
    harley.player = player
    arm.set_physics_process(false)

func start() -> void:
    super.start()
    arm.set_physics_process(true)

func _physics_process(delta: float) -> void:
    if active:
        harley.tick(delta)

func die_sequence() -> void:
    arm.set_physics_process(false)
    arm.claw_hitbox.set_deferred("monitoring", false)
    # Harley part en vrille et s'écrase (vidéo 10:36-10:39).
    var tw := create_tween()
    tw.tween_property(harley, "rotation", TAU * 3.0, 1.2)
    tw.parallel().tween_property(harley, "position:y", ground_local_y, 1.2) \
        .set_trans(Tween.TRANS_QUAD).set_ease(Tween.EASE_IN)
    await tw.finished
    await explode_area(Rect2(harley.position - Vector2(16, 16), Vector2(32, 32)), 5, 0.08)
    harley.show_ko()
    # Le grappin remonte hors de l'écran.
    create_tween().tween_property(arm, "position:y", arm.position.y - 220.0, 1.5)
```

À FAIRE

1. Nouvelle scène `res://bosses/crane_harley/crane_harley.tscn`, racine `Node2D` `CraneHarley`, script `crane_harley.gd` :

```
CraneHarley (Node2D, crane_harley.gd)
├── Health
├── Arm (Node2D, sphere_arm.gd)
│   ├── ClawHitbox (Hitbox)
│   │   └── CollisionShape2D
│   └── Harley (Node2D, harley_chair.gd)
│       ├── Sprite2D
│       ├── Hurtbox
│       │   └── CollisionShape2D
│       └── HitFlash
score_value = 5000
max_health = 80 (À CALIBRER)
origine = point d'accroche de la chaîne (haut de l'écran)
couche vide · masque 5 · damage 2
cercle de rayon 14
white.png scale (1,5 ; 1,2), rose · matériau hit_flash (Local To Scene)
couche 6 · health = Health
rectangle 22x18
target = Sprite2D · hurtbox = Hurtbox
```

2. **Arm** : **Orange Sphere** = `sphere_orange.png`, **Dark Sphere** = `sphere_dark.png`. **Harley** : **Bomb Scene** = `harley_bomb.tscn` (la variante sans rebond de l'étape 09).
3. Dans la section, une `BossArena` en (4 200, 112), avec `CraneHarley` comme enfant en position locale (0, -112) : le point d'accroche est en haut de l'écran (y = 0). Exports : **Boss, Section**.
4. Prolonge le sol jusqu'à x = 4 500 si besoin.

TESTE

- À l'entrée de l'arène, la pastille « 99 » apparaît. Le grappin suit Batman, s'abat (explosion au sol), se referme, remonte.
- Parfois, il part dans le fond : il rapetisse, s'assombrit, et passe derrière la chaîne. Pendant ce temps, il ne fait pas mal.
- Harley vole en huit dans le haut de l'écran et lobe des bombes là où se trouvait Batman. Il faut la viser en diagonale haute.
- À 00 : Harley part en vrille, s'écrase, explosions, le grappin remonte, pose de victoire, +5 000, puis la scène suivante.

SI ÇA NE MARCHE PAS

- **Les sphères sont toutes de la même taille** : la texture n'a pas le bon nombre de cases (4 pour l'orange, 2 pour le sombre), ou le générateur n'a pas été exécuté.
- **Le grappin ne bouge pas** : c'est normal avant le début du combat (`set_physics_process(false)`) ; vérifie que l'arène lance bien le boss.
- **Le grappin ne touche jamais** : masque 5 sur `ClawHitbox`, et vérifie que `ground_y` amène la pince jusqu'à la hauteur de Batman (le sol est à 200 px sous le point d'accroche).

CONCEPT : LA VULNÉRABILITÉ LIÉE À LA PROFONDEUR

Quand le grappin passe dans le fond, il est plus petit, plus sombre *et* inoffensif : le visuel dit la vérité sur le gameplay. Règle d'or de toute pseudo-3D : ce qui a l'air loin ne doit pas toucher. Tu retrouveras la même règle avec la montgolfière du Joker.

POINT DE CONTRÔLE

TU AS FINI L'ÉTAPE QUAND

- ☐ Le Gem Expo s'assemble avec deux salles réutilisées ; la ville défile plus lentement derrière les baies.
- ☐ Une chenille de têtes à hélice descend puis serpente ; une ligne traverse le haut de l'écran.
- ☐ L'entrepôt est peint en tuiles avec collisions ; Harley passe en diagonale sans combattre.
- ☐ Le grappin suit Batman, s'abat, se referme, remonte et parfois s'enfonce dans le fond, plus petit et plus sombre.
- ☐ Harley vole et lobe des bombes. À 00 : vrille, crash, K.O., +5 000, section suivante.

```
git add -A && git commit -m "Étape 11 : section 2, Gem Expo, entrepôt et grappin"
```

CE QUE TU RETIENS

? Pourquoi les salles n'ont-elles pas leur propre sol ?

Des sols séparés bout à bout créent des jointures où un personnage peut accrocher. Un seul sol continu dans la section évite le problème ; les salles n'apportent que le décor et les meubles.

? Que donne la formule $s = f / (f + z)$?

Le facteur de réduction d'un point situé à la profondeur z : 1 au premier plan ($z = 0$), de plus en plus petit en s'éloignant. On l'utilise pour la position à l'écran, pour choisir l'image de la bonne taille et pour la luminosité.

? À quoi sert $z_index = -z$?

À trier l'affichage : les sphères les plus proches (z petit) ont un z_index plus grand et sont dessinées par-dessus les lointaines. C'est l'algorithme du peintre.

? Pourquoi peindre les caisses avec une `TileMapLayer` plutôt qu'avec des `StaticBody2D` ?

Parce que chaque tuile porte son image et sa collision : peindre une pyramide crée à la fois le décor et le sol. On construit, modifie et efface un niveau à la souris, en quelques secondes, sans gérer des dizaines de nœuds.

POUR ALLER PLUS LOIN

- Remplace `modulate` par des sphères indexées et `indexed.gdshader` avec une palette de 4 lignes (étape 08) : la fidélité totale.
- Ajoute une ombre au sol sous le grappin (une ellipse sombre dont la taille dépend de `hub.y`) : elle aide le joueur à anticiper l'impact.
- Lis le tome 08 du guide, et regarde la vidéo de Jon Burton sur le bras de grue.

SECTION 3 : CONVOI ET JOKER

Une rue qui file à toute vitesse, des camions dont les écarts s'ouvrent et se ferment, des ballons-mines qui montent, puis le Joker dans sa montgolfière, qui fond sur Batman depuis le fond de l'écran. C'est la fin du niveau 1.

~4 h

Prérequis : étape 10 (l'étape 11 n'est pas nécessaire)

Guide : [tome 09](#)

CE QUE TU VAS APPRENDRE

GODOT 4

- `AnimatableBody2D` et `sync_to_physics`
- `Parallax2D.autoscroll`
- `use_parent_material`
- Composer des `Tween` pour une séquence de fin

DÉVELOPPEMENT DE JEU

- Le référentiel : la vitesse sans bouger
- Les plateformes mobiles
- La pseudo-3D par l'échelle
- Un boss qui alterne présence et absence
- Terminer un niveau

LE CONVOI

12.1 · Le monde ne bouge pas, la rue si



Le convoi. Les véhicules restent en place dans le monde, c'est la rue qui défile derrière eux ; chaque véhicule est une plateforme mobile.

DANS LA VIDÉO

De ▶ 10:44 à ▶ 13:03 : les vitrines et les réverbères défilent en continu, les véhicules se suivent (plateau à caisses, fourgon violet, plateau nu, voiture de police), et les écarts entre eux varient légèrement.

CONCEPT : LE RÉFÉRENTIEL

Faire réellement rouler des camions à 80 km/h en suivant le joueur compliquerait tout : caméra, ennemis, projectiles. On inverse le point de vue : **le convoi est immobile** dans le monde, et c'est **la rue qui défile** derrière lui. Le joueur ne voit aucune différence ; le code reste celui d'un niveau normal. Sur Mega Drive, c'était pareil : le « mouvement » n'est qu'une valeur de défilement du plan de fond.

12.2 · La section et la rue qui défile

À FAIRE

1. Nouvelle scène 2D `res://sections/s3_convoy.tscn`, racine `S3Convoy` (script `section.gd`), en (0, 0).
2. En premiers enfants (donc dessinés derrière), deux `Parallax2D` :
 - `Street` : **Scroll Scale** (1, 0), **Repeat Size** (640, 0), **Autoscroll** (-300, 0). Enfants : un fond sombre (`ColorRect` 640×170), des vitrines roses et des réverbères (`ColorRect`) répartis sur 640 px.
 - `Road` : mêmes réglages mais **Autoscroll** (-420, 0). Enfants : une chaussée grise de 640×54 en y = 170, avec des marquages blancs tous les 80 px.
3. Complète l'arbre :

```
S3Convoy (Node2D, section.gd)
├─ Street (Parallax2D)
├─ Road (Parallax2D)
├─ World
│   ├── Convoy (Node2D)           les véhicules (12.3)
│   ├── Gift (Breakable)         paquet géant → spawn_enemy = clown (12.5)
│   ├── KillZone (Area2D)        sous la chaussée (12.4)
│   ├── ArenaBalloons (WaveArena) (12.5)
│   └─ BossArena → JokerBalloon sur le dernier plateau (12.6)
└─ EnemyBullets (Pool) · Player · Camera (camera_rig.gd) · Hud
```

next_scene = res://screens/level_end.tscn · player = Player
autoscroll (-300, 0) · repeat_size (640, 0)
autoscroll (-420, 0) · plus rapide : plus proche

4. Dans la section 2, mets **Next Scene** = `res://sections/s3_convoy.tscn` (ou dans la section 1, si tu as sauté l'étape 11).

AUTOSCROLL

`autoscroll` fait défiler la couche toute seule, en px/s, en plus de son mouvement lié à la caméra. Avec `repeat_size`, son contenu boucle à l'infini ; il doit faire au moins la largeur de l'écran (ici $640 \geq 320$). Deux couches à des vitesses différentes donnent la profondeur : la chaussée, plus proche, défile plus vite que les vitrines.

12.3 · Le véhicule, une plateforme mobile



Les écarts entre véhicules changent et obligent à sauter.

res://sections/s3/convoy_vehicle.gd

```
class_name ConvoyVehicle
extends AnimatableBody2D
## Véhicule du convoi. Il dérive un peu en avant et en arrière pour ouvrir et fermer les écarts.
## AnimatableBody2D + sync_to_physics : Batman posé dessus est emporté proprement.

@export var drift_amplitude := 12.0    ## px
@export var drift_period := 5.0        ## s
@export var wheel_speed := 14.0        ## rad/s
var _origin := Vector2.ZERO
var _t := 0.0
@onready var wheels: Node2D = $Wheels

func _ready() -> void:
    sync_to_physics = true
    _origin = position
    _t = randf() * drift_period

func _physics_process(delta: float) -> void:
    _t += delta
    position = _origin + Vector2(sin(_t * TAU / drift_period) * drift_amplitude, 0.0)
    for w in wheels.get_children():
        (w as Node2D).rotation += wheel_speed * delta
```

CONCEPT : POURQUOI `AnimatableBody2D`

C'est le corps physique prévu pour les plateformes déplacées par script ou par animation. Avec `sync_to_physics`, son déplacement est calculé au rythme de la physique, et un `CharacterBody2D` posé dessus reçoit automatiquement sa vitesse (*platform velocity*) : Batman avance avec le camion sans une ligne de code. Essaie avec un `StaticBody2D` qu'on déplace : Batman resterait sur place et le camion glisserait sous ses pieds.

À FAIRE

1. Nouvelle scène, racine `AnimatableBody2D` nommée `ConvoyVehicle`, script `convoy_vehicle.gd`, couche 2 seule, masque vide. Enregistre en `res://sections/s3/convoy_vehicle.tscn`.
2. Enfants : `Body` (`Sprite2D` ou `ColorRect` : cabine + plateau, environ 160×40), un `CollisionShape2D` pour le plateau (rectangle 140×8, **One Way Collision** coché), et un `Node2D` nommé `Wheels` avec trois roues (`Sprite2D` `white.png` gris, avec un trait pour voir la rotation).
3. Variantes : à partir de cette scène, crée des **scènes héritées** : `truck_crates.tscn` (avec des caisses : formes rectangles **non** one-way, posées sur le plateau), `van.tscn` (un toit one-way plus haut), `police_car.tscn` (plus court et plus bas).
4. Sous `Convoy`, aligne une douzaine de véhicules de x = 0 à environ 2 800, plateaux vers y = 160, avec des écarts de 24 à 40 px. Répète la séquence plateau à caisses → fourgon → plateau nu → voiture de police. Batman démarre sur le premier.

TESTE

La rue file derrière ; les roues tournent ; les véhicules avancent et reculent doucement. Posé sur un plateau immobile, Batman est emporté par le mouvement du camion. Saute de véhicule en véhicule ; monte sur les caisses.

BAS + SAUT SUR LES CAISSES

Les caisses sont sur la couche 2 comme les plateaux. « Bas + saut » coupe toute la couche 2 pendant 0,25 s : debout sur une caisse, Batman la traverse et retombe sur le plateau. C'est acceptable (et pratique pour redescendre). Pour l'empêcher, mets les caisses sur la couche 1.

12.4 · Tomber du convoi



```
res://sections/s3/kill_zone.gd
```

```
extends Area2D
```

```
## Sous la chaussée (masque 3) : tomber entre deux véhicules coûte une vie.
```

```
func _ready() -> void:
```

```
    body_entered.connect(_on_body_entered)
```

```
func _on_body_entered(body: Node2D) -> void:
```

```
    if body is Player:
```

```
        # Health directement : la chute tue même pendant le clignotement d'invincibilité.
```

```
        # respawn() (tome 03) fera ensuite retomber Batman depuis le haut de l'écran.
```

```
        (body as Player).health.damage(999)
```

À FAIRE

Sous `World`, une `Area2D` `KillZone`, masque 3, avec un rectangle de 3 200×16 en (1 400, 240) : sous le bas de l'écran.

TESTE

Rate un saut : Batman tombe entre deux véhicules, passe en « Dead », puis réapparaît depuis le haut de l'écran, au-dessus du convoi, avec une vie de moins.

HYPOTHÈSE

La vidéo ne montre aucune chute. On suppose qu'elle est mortelle, comme dans la plupart des run & gun. Si tu préfères être indulgent, replace plutôt Batman sur le véhicule le plus proche avec un peu de dégâts.

12.5 · Ballons-mines et paquet géant



Ballons de couleur portant chacun une mine. ▶ 11:20

À FAIRE

1. Une `WaveArena` `ArenaBalloons` vers $x = 1\,000$, avec **Ground Y** = la hauteur des plateaux (≈ 160). Deux vagues de `balloon_mine.tscn` : `origins` = ["below", "right"], `max_alive` = 8, `spawn_interval` = 0,4, quotas 14 et 18.
2. Le paquet géant : une instance de `breakable.gd` (comme la poubelle, mais 32×32, rouge et vert), posée sur un plateau vers $x = 1\,600$, avec **Spawn Enemy** = `clown.tscn` et Health 3.
3. Un crâne sur un plateau juste avant l'arène des ballons (▶ 12:24 dans la vidéo).

TESTE

Dans l'arène, des ballons de quatre couleurs montent du bas en oscillant ; Batman les abat en diagonale haute ou à la verticale. Le crâne fait tout exploser d'un coup. Plus loin, trois tirs sur le paquet libèrent un clown.

LA MONTGOLFIÈRE DU JOKER

12.6 · Observer un passage



Un passage de la montgolfière : $\sin(u \cdot \pi)$ pilote taille, hauteur et vulnérabilité



Un passage complet de la montgolfière. Entrée petite et haute, approche, point le plus proche, éloignement, puis écran vide avant le passage suivant.

DANS LA VIDÉO

- Chaque « passage » : entrée par un bord, petite et haute ; elle grossit en descendant vers le plateau ; traverse ; sort par l'autre bord, puis quelques secondes d'écran vide (▶ 13:59 – ▶ 14:04).
- Certains passages sont géants (▶ 13:22), d'autres moyens.
- Des grappes de petites bombes noires, en ligne ou en colonne, qui rebondissent.
- Un flash pastel de tout le ballon quand il est touché ; la vitalité baisse surtout quand il est proche.

CONCEPT : LA PROFONDEUR PAR L'ÉCHELLE

Un seul paramètre, la **taille apparente**, raconte toute la 3D : petit = loin, grand = près. On le pilote par une courbe en cloche, $\sin(u \times \pi)$, qui monte de 0 à 1 puis redescend pendant le passage. La même valeur commande la taille, la hauteur (le ballon descend en approchant) et la vulnérabilité : on ne peut le toucher que lorsqu'il est assez proche. Comme sur la console, la taille est arrondie à des **paliers** (`Fx.snap_scale`, étape 08).

12.7 · `joker_balloon.gd`



res://bosses/joker_balloon/joker_balloon.gd

```
class_name JokerBalloon
extends Boss

## Boss final du niveau 1 (vidéo 13:04-14:36).
## Cycle : attente hors écran -> passage (approche puis éloignement) -> attente...

enum S { WAIT, PASS }

@export var bomb_scene: PackedScene
@export var pass_duration := Vector2(5.0, 7.0)

## bomb.tscn : bounces = 1
```

```

@export var wait_time := Vector2(1.5, 4.0)
@export var high_y := -150.0 ## relatif au plateau (y = 0)
@export var low_y := -70.0 ## nacelle à hauteur de tir au plus près
@export var half_width := 230.0 ## départ et arrivée hors écran
@export var sizes := PackedFloat32Array([0.35, 0.5, 0.65, 0.8, 1.0, 1.2, 1.4, 1.6])
@export var vulnerable_scale := 0.8
@export var drop_interval := 1.1
@export var ko_texture: Texture2D

var _s := S.WAIT
var _t := 0.0
var _dur := 2.0
var _side := 1.0
var _near := 1.0
var _drop := 0.0

@onready var visual: Node2D = $Visual
@onready var hurtbox: Hurtbox = $Visual/Hurtbox
@onready var drop_point: Marker2D = $Visual/DropPoint
@onready var joker_figure: Sprite2D = $JokerFigure

func _ready() -> void:
    super._ready()
    visual.hide()
    joker_figure.hide()
    hurtbox.monitorable = false

func start() -> void:
    super.start()
    _go_wait(1.0)

func _physics_process(delta: float) -> void:
    if not active:
        return
    _t += delta
    match _s:
        S.WAIT:
            if _t >= _dur:
                _go_pass()
        S.PASS:
            var u := clampf(_t / _dur, 0.0, 1.0)
            var bump := sin(u * PI) # 0 -> 1 -> 0 : approche puis éloignement
            var sc := Fx.snap_scale(lerpf(sizes[0], _near, bump), sizes)
            visual.scale = Vector2(sc, sc)
            visual.position = Vector2(lerpf(-_side * half_width, _side * half_width, u),
                lerp(high_y, low_y, bump))
            hurtbox.monitorable = sc >= vulnerable_scale # ce qui est loin ne se touche pas
            _drop -= delta
            if _drop <= 0.0 and sc >= 0.6:
                _drop = drop_interval
                _drop_cluster()
            if u >= 1.0:
                _go_wait(randf_range(wait_time.x, wait_time.y))

```

```

func _go_wait(duration: float) -> void:
    _s = S.WAIT
    _t = 0.0
    _dur = duration
    visual.hide()
    hurtbox.monitorable = false

func _go_pass() -> void:
    _s = S.PASS
    _t = 0.0
    _dur = randf_range(pass_duration.x, pass_duration.y)
    _side = -_side # alterne les côtés d'entrée
    _near = sizes[sizes.size() - 1] if randf() < 0.5 else 1.0 # passage géant ou moyen
    _drop = 0.8
    visual.show()

func _drop_cluster() -> void:
    if bomb_scene == null:
        return
    var column := randf() < 0.4
    for i in 5:
        var b := bomb_scene.instantiate() as Bomb
        if column:
            b.velocity = Vector2(0.0, 20.0 + i * 28.0) # colonne : chutes étagées
        else:
            b.velocity = Vector2((i - 2) * 32.0, -40.0) # ligne : éventail
        Fx.spawn(b, drop_point.global_position)

func die_sequence() -> void:
    hurtbox.set_deferred("monitorable", false)
    visual.show()
    # Le ballon se fige en haut, au centre...
    var tw := create_tween()
    tw.tween_property(visual, "position", Vector2(0.0, high_y + 30.0), 0.6)
    tw.parallel().tween_property(visual, "scale", Vector2.ONE, 0.6)
    await tw.finished
    # ...et l'enveloppe se couvre d'explosions (vidéo 14:24-14:29).
    await explode_area(Rect2(-60.0, high_y - 30.0, 120.0, 100.0), 30, 0.12)
    visual.hide()
    # Le Joker est éjecté puis retombe, assommé, sur le plateau (vidéo 14:30-14:31).
    joker_figure.position = visual.position + Vector2(0.0, 30.0)
    joker_figure.show()
    var tw2 := create_tween()
    tw2.tween_property(joker_figure, "position:y", joker_figure.position.y - 40.0, 0.35) \
        .set_trans(Tween.TRANS_QUAD).set_ease(Tween.EASE_OUT)
    tw2.tween_property(joker_figure, "position", Vector2(30.0, -8.0), 0.6) \
        .set_trans(Tween.TRANS_QUAD).set_ease(Tween.EASE_IN)
    await tw2.finished
    if ko_texture:
        joker_figure.texture = ko_texture

```

Comprendre le code

- **Deux états seulement** : **WAIT** (hors écran) et **PASS**. Toute la richesse du combat vient d'une variable continue, **u** (avancement du passage de 0 à 1), dont on dérive la cloche **bump**.

- `bump` commande à la fois l'échelle (petit → grand → petit), la hauteur (haut → bas → haut) et, via l'échelle, la vulnérabilité et le droit de lâcher des bombes. Une seule cause, plusieurs effets cohérents.
- `Fx.snap_scale` arrondit l'échelle à 8 paliers : le ballon « saute » de taille en taille, comme des sprites pré-calculés.
- `_near` tiré au hasard : un passage sur deux est géant. `_side` alterne : le ballon entre une fois par la gauche, une fois par la droite.
- La séquence de mort enchaîne trois `Tween` et des explosions avec `await` : c'est une petite cinématique.

12.8 · Monter la montgolfière



À FAIRE

1. Nouvelle scène `res://bosses/joker_balloon/joker_balloon.tscn` :

```
JokerBalloon (Node2D, joker_balloon.gd)
├── Health
├── Visual (Node2D)
│   ├── Envelope (Sprite2D)
│   ├── Basket (Sprite2D)
│   ├── Hurtbox
│   │   └── CollisionShape2D
│   └── DropPoint (Marker2D)
├── JokerFigure (Sprite2D, caché)
└── HitFlash

score_value = 10000 · origine = centre du plateau
max_health = 120 (À CALIBRER)
matériau hit_flash (Local To Scene)
white.png scale (4, 4) en (0, -40), rouge · Use Parent Material
white.png scale (2, 1) en (0, 0), vert · Use Parent Material
couche 6 · health = Health · sur la nacelle
rectangle 32×16
(0, 12) : sous la nacelle
white.png scale (1, 1.5), violet : le Joker éjecté
target = Visual · hurtbox = Visual/Hurtbox · amount = 0.55
```

2. **Bomb Scene** = `bomb.tscn` (1 rebond).
3. Le dernier véhicule du convoi : un plateau nu, **Drift Amplitude** = 0 (sol stable). Une `BossArena` au centre de ce plateau, avec `JokerBalloon` comme enfant placé exactement à la surface du plateau : `high_y` et `low_y` sont relatifs à ce point.
4. Exports de la `BossArena` : **Boss** et **Section**.

USE_PARENT_MATERIAL

Le shader de flash est posé sur `Visual`, un simple `Node2D`. En cochant **Use Parent Material** sur l'enveloppe et la nacelle, ils utilisent le matériau de leur parent : un seul paramètre `flash` fait clignoter tout le ballon d'un coup.

TESTE

- La caméra se fige au centre du plateau, la rue continue de défiler derrière, la pastille « 99 » apparaît.
- Le ballon entre petit et haut, grossit par paliers en descendant, lâche des grappes de bombes (en ligne ou en colonne), traverse et ressort. Écran vide, puis il revient par l'autre côté.
- Tes tirs le traversent quand il est petit ; quand il est grand, il clignote en pastel et la vitalité baisse.
- À 00 : il se fige au centre, explosions en chaîne, le Joker est éjecté et retombe sur le plateau, pose de victoire, +10 000, fondu, écran de fin.

CE QUI N'EST PAS DANS LA VIDÉO

D'autres descriptions du jeu évoquent une montgolfière qui rebondit au sol et charge à l'horizontale. Ces attaques n'apparaissent pas de 13:04 à 14:36. Si tu les vois dans un émulateur, ajoute un état `CHARGE` : échelle fixée à 1, `y = low_y`, traversée rapide (1,5 s) et Hurtbox active. Un excellent exercice pour vérifier que tu maîtrises les machines à états.

12.9 · Le niveau complet



À FAIRE

1. Vérifie la chaîne des **Next Scene** : `title_card` → `s1_streets` → `s2_gem_expo` → `s3_convoy` → `level_end` → `title`.
2. `F5` et joue le niveau en entier, de l'intro à la Batmobile finale. Chronomètre : la vidéo de référence dure 14:37.

MÉTHODE : FINIR

Tu viens de terminer un niveau complet. C'est l'étape que la plupart des projets amateurs n'atteignent jamais : ils ajoutent des fonctionnalités sans jamais boucler la boucle. Un jeu court mais fini vaut plus qu'un jeu long à moitié fait. Fais un commit, et prends le temps de le faire essayer à quelqu'un.

POINT DE CONTRÔLE

TU AS FINI L'ÉTAPE QUAND

- ☐ La rue défile en continu ; Batman saute de camion en camion, emporté par leur mouvement ; les écarts varient.
- ☐ Tomber entre deux véhicules coûte une vie.
- ☐ Des ballons-mines montent du bas en oscillant ; un paquet géant libère un sbire.
- ☐ La montgolfière entre petite et haute, grossit par paliers, lâche des grappes de bombes, ressort ; elle n'est vulnérable que lorsqu'elle est grande.
- ☐ À 00 : explosions, Joker éjecté et assommé, pose de victoire, +10 000, écran de fin, retour au titre.

```
git add -A && git commit -m "Étape 12 : section 3, convoi et montgolfière du Joker – niveau 1 complet"
```

CE QUE TU RETIENS

? Pourquoi le convoi est-il immobile dans le monde ?

Parce que seul le mouvement relatif compte pour le joueur. Faire défiler le décor plutôt que déplacer le convoi garde une caméra, des ennemis et des projectiles normaux, sans compenser une vitesse partout dans le code.

? Qu'apporte `AnimatableBody2D` par rapport à un `StaticBody2D` qu'on déplace ?

Il transmet sa vitesse aux corps posés dessus : Batman est emporté par le camion. Un `StaticBody2D` déplacé glisserait sous ses pieds.

? Comment la montgolfière sait-elle si elle est vulnérable ?

Par son échelle courante : `hurtbox.monitorable = sc >= vulnerable_scale`. Petite (loin), les tirs la traversent ; grande (proche), elle encaisse. Le visuel dit la vérité sur le gameplay.

POUR ALLER PLUS LOIN

- Ajoute l'état `CHARGE` décrit plus haut.
- Fais vibrer légèrement les véhicules (± 1 px vertical à haute fréquence) pour renforcer l'impression de vitesse.
- Lis le tome 09 du guide.

FINITIONS, EXPORT ET SUITE

Le jeu est complet. Il reste ce qui sépare un projet d'un jeu fini : le ressenti, la comparaison méthodique avec l'original, le ménage, la mesure des performances, l'export. Puis un regard en arrière sur ce que tu as appris, et des aide-mémoires à garder sous la main.

~3 h

Prérequis : étape 12

Guide : tome 11

CE QUE TU VAS APPRENDRE

GODOT 4

- `Camera2D.offset`, `Engine.time_scale`
- Le profileur et les moniteurs
- Les modèles d'exportation et les pré-réglages
- Les filtres d'export

DÉVELOPPEMENT DE JEU

- Le ressenti (*game feel*) et sa mesure
- La passe de fidélité
- Le profilage avant l'optimisation
- La rétrospective (*post-mortem*)

LE RESENTI

Le « juice » à la Mega Drive : des dizaines de sprites d'explosion, pas de tremblement d'écran



Le ressenti du jeu original. Pas de tremblement d'écran ni de hit stop visibles : l'impact passe par la quantité d'explosions et les flashes de palette.

13.1 - Le tremblement de caméra



CONCEPT : LE « TRAUMA »

Une valeur de 0 à 1 qui augmente à chaque choc et décroît avec le temps ; l'amplitude du tremblement est proportionnelle à son **carré** : fort au début, il s'éteint en douceur. En 320×224, 4 à 6 pixels suffisent. On décale `offset` plutôt que la position, pour ne pas perturber le suivi du joueur.

Ajoute à `camera_rig.gd` :

res://level/camera_rig.gd (ajouts)

```
@export var max_shake := Vector2(5, 4)  ## pixels
@export var shake_decay := 1.6          ## trauma perdu par seconde
var trauma := 0.0

# Dans _ready(), ajoute : add_to_group("camera")

## Appelée par call_group("camera", "shake", 0.35) : grappin, explosions de boss...
func shake(amount: float) -> void:
    trauma = minf(trauma + amount, 1.0)

func _process(delta: float) -> void:
    trauma = maxf(trauma - shake_decay * delta, 0.0)
    var k := trauma * trauma  # le carré : fort au début, s'éteint en douceur
    offset = Vector2(roundf(max_shake.x * k * randf_range(-1, 1)), roundf(max_shake.y * k *
        randf_range(-1, 1)))
```

TESTE

Le grappin de la section 2 appelait déjà `call_group("camera", "shake", 0.35)` : maintenant que la caméra est dans le groupe, chaque impact secoue l'écran. Aucune autre ligne à changer. Ajoute aussi un `shake(0.6)` dans les `die_sequence()` des boss si tu veux.

13.2 · Le hit stop



res://fx/fx.gd (ajout)

```
## Fige le jeu un instant pour donner du poids à un impact.
## Le 4e argument de create_timer (ignore_time_scale = true) est indispensable :
## sans lui, la minuterie est figée elle aussi et le jeu ne repart jamais.
static func hit_stop(tree: SceneTree, duration := 0.05) -> void:
    Engine.time_scale = 0.0
    await tree.create_timer(duration, true, false, true).timeout
    Engine.time_scale = 1.0
```

EXERCICE

1. Au début de `HarleyMachine._go_phase()`, ajoute `Fx.hit_stop(get_tree(), 0.1)`.
2. Joue la transition de phase avec et sans. Puis essaie 0,3 s : trop long, le jeu paraît planté.

FIDÉLITÉ

La vidéo ne montre ni tremblement d'écran évident ni hit stop : ce sont des ajouts modernes. Utilise-les avec parcimonie, ou désactive-les pour un remake strict. C'est un choix de direction artistique : note-le dans ton journal, avec ta raison.

MÉTHODE : RÉGLER LE RESSENTI PAR COMPARAISON

Le ressenti ne se juge pas en lisant du code. Enregistre deux courtes vidéos de ton jeu (avec et sans l'effet), regarde-les côte à côte, et compare-les à la vidéo de référence. L'œil voit tout de suite ce qui « tombe juste ».

LA PASSE DE FIDÉLITÉ

13.3 · Comparer à l'original, section par section



Planche de référence pour comparer ton remake à l'original, section par section.

À FAIRE

1. Ouvre la [checklist de fidélité du tome 11](#) dans un onglet, et la vidéo dans un autre.
2. Joue une section, puis regarde la même section dans la vidéo. Coche ce qui est conforme, note ce qui ne l'est pas.
3. Classe tes notes en trois colonnes : **à corriger** (bug ou écart gênant), **à calibrer** (un chiffre à ajuster), **assumé** (un écart volontaire, avec sa raison).
4. Traite d'abord la colonne « à corriger », puis les réglages les plus visibles.

MÉTHODE : LA LISTE PRIORISÉE

À la fin d'un projet, la liste des améliorations possibles est infinie. On n'essaie pas de tout faire : on trie par impact sur le joueur, et on s'arrête quand le reste ne se remarque plus. Un jeu n'est jamais terminé, il est publié.

LE MÉNAGE

13.4 · Ranger les outils de débogage



À FAIRE

1. Dans `player.tscn`, cache le `StateDebug` (icône d'œil). Ne le supprime pas : il resservira.
2. Cache le `Placeholder` si tu as des sprites.
3. Retire les raccourcis de test de `test_room.gd` du jeu final : ils ne sont que dans la salle de test, qui n'est jamais chargée par le flux du jeu. Vérifie-le.
4. Cherche les `print()` restants : **Script → Rechercher dans les fichiers** (`Ctrl Maj F` / `⌘ Maj F`). Garde ceux des scripts d'éditeur, retire les autres.
5. Lance le jeu complet et vérifie que le panneau **Débogueur → Erreurs** reste vide. Chaque avertissement est un bug en attente.

13.5 · Mesurer les performances



CONCEPT : MESURER AVANT D'OPTIMISER

On n'optimise jamais « au feeling ». On mesure, on trouve le vrai goulot, on corrige celui-là, et on remesure. Le niveau 1 est très léger pour une machine moderne ; l'exercice sert surtout à apprendre les outils.

À FAIRE

1. Pendant une arène chargée, ouvre **Débogueur → Moniteurs : Object → Nodes, Time → FPS, Time → Physics Process**. Le nombre de nœuds doit revenir au même niveau après chaque vague. S'il grimpe sans cesse, quelque chose n'est jamais libéré (une fuite).
2. **Débogueur → Profileur → Démarrer**, joue 30 s, **Arrêter**. Trie par temps : quelles fonctions coûtent le plus ? Normalement, aucune de tes fonctions ne dépasse quelques dixièmes de milliseconde.
3. Si tu veux retrouver la vraie limite de la console, plafonne le nombre total d'ennemis actifs vers 60. Au-delà, le jeu original aurait fait clignoter des sprites.

L'EXPORT

13.6 · Exporter ton jeu



À FAIRE

1. ≡ Éditeur → Gérer les modèles d'exportation → **Télécharger et installer** (une seule fois par version de Godot).
2. ≡ **Projet** → Exporter → **Ajouter...** → choisis ta plateforme (macOS, Windows Desktop, Linux, Web).
3. Onglet **Ressources** : dans « Filtres pour exclure des fichiers », saisis `tools/*`, `sandbox/*`. Les scripts d'éditeur et les essais jetables ne doivent pas partir avec le jeu.
4. ≡ **Projet** → Paramètres du projet → Application → **Boot Splash** : fond noir, ton propre logo, filtre désactivé.
5. **Exporter le projet**, choisis un dossier hors du projet, lance l'exécutable obtenu.

SI ÇA NE MARCHE PAS

- **Le jeu exporté plante au chargement d'une image** : un script charge un fichier par `Image.load_from_file("res://...")` au lieu de `load()`. Seuls les scripts d'éditeur (exclus) le font ici.
- **Les options ne sont pas sauvegardées** : vérifie que tu écris dans `user://` et non dans `res://` (en lecture seule une fois exporté).
- **macOS refuse de lancer l'application** : clic droit → Ouvrir, comme pour Godot lui-même.

RAPPEL

Un build qui contient les sprites, la musique ou les logos originaux ne doit pas être publié. Avec tes formes de substitution et tes propres dessins, en revanche, tout t'appartient.

REGARDER EN ARRIÈRE

13.7 · Ta rétrospective



MÉTHODE : LE POST-MORTEM

Les studios terminent chaque projet par une rétrospective écrite. Elle transforme une expérience en connaissance réutilisable. Prends une heure, relis ton journal de bord, et réponds par écrit :

1. Quelles sont les trois choses qui ont **bien marché** dans ta façon de travailler ?
2. Quelles sont les trois choses qui t'ont **coûté le plus de temps** ? Auraient-elles pu être évitées ?
3. Quelle erreur as-tu rencontrée le plus souvent ? Que vérifies-tu maintenant en premier ?
4. Quel concept te semblait obscur au début et te paraît évident maintenant ?
5. Si tu recommençais demain, que ferais-tu différemment ?

13.8 · Et après ?



- **Le mode deux joueurs** : le jeu original se joue à deux. Le [tome 11](#) décrit la marche à suivre (actions préfixées `p1_` / `p2_`, caméra entre les deux joueurs, arrivée en cours de partie). C'est un excellent exercice : il touche les entrées, la caméra, le HUD et les sections.
- **Le niveau 2** : tu as tous les outils. Regarde la suite de la vidéo, découpe le niveau comme le tome 01 l'a fait pour le niveau 1, puis construis-le avec ta méthode.
- **Ton propre jeu** : prends un concept simple, tiens-toi aux six principes, et fais d'abord une tranche verticale. Tu sais désormais construire un jeu de bout en bout.
- **Le filtre CRT** (optionnel) : le shader de l'ancienne doc B (tome 4) fonctionne tel quel, dans un `CanvasLayer` de couche 101, donc au-dessus de `PaletteFx`.

POINT DE CONTRÔLE FINAL

TU AS FINI LE PARCOURS QUAND

- ☐ Le jeu se lance depuis un exécutable exporté et se joue de l'intro à la Batmobile finale.
- ☐ Le panneau des erreurs reste vide pendant toute une partie.
- ☐ Ta checklist de fidélité est passée, avec ses écarts assumés.
- ☐ Ta rétrospective est écrite.

```
git git add -A && git commit -m "Étape 13 : finitions, export" && git tag v1.0-niveau-1
```

ANNEXE A - LA BOÎTE À OUTILS DE DÉBOGAGE

Outil	Où	Quand s'en servir
<code>print()</code> , <code>print_debug()</code>	dans le code	Savoir si une ligne est atteinte et avec quelles valeurs. <code>print_debug</code> ajoute le fichier et la ligne.
Point d'arrêt	clic dans la marge du script (point rouge), ou <code>breakpoint</code>	Figurer le jeu sur une ligne et inspecter toutes les variables, puis avancer pas à pas (<code>F10</code> , <code>F11</code>).
Arbre distant	onglet Distant au-dessus de l'arbre de scène, jeu lancé	Voir l'arbre réel du jeu en cours, et modifier les propriétés en direct.
Formes de collision visibles	≡ Déboguer → Formes de collision visibles	Toute question « pourquoi ça ne touche pas ? ».
Moniteurs	Débogueur → Moniteurs	FPS, nombre de nœuds, mémoire, temps physique.
Profileur	Débogueur → Profileur	Trouver la fonction qui coûte cher.
Affichage de débogage	un <code>Label</code> au-dessus d'un personnage (étape 04)	Voir un état interne (état de la machine, vitesse) en jouant.
Raccourcis de test	<code>_unhandled_input</code> de la salle de test (étape 05)	Provoquer un cas rare à volonté.
Salle de test et bancs d'essai	<code>sandbox/</code> , <code>test_room.tscn</code>	Isoler une brique pour savoir si c'est elle qui est en cause.
<code>git diff</code> , <code>git stash</code>	terminal	« Ça marchait hier » : voir ce qui a changé, ou mettre les changements de côté pour vérifier.

ANNEXE B · LES ERREURS FRÉQUENTES

Message	Cause habituelle	Correction
Invalid get index 'x' (on base: 'null instance')	Un <code>\$Chemin</code> ou un <code>@export</code> de nœud ne désigne rien	Vérifier le nom exact du nœud, la place dans l'arbre, ou assigner l'export dans l'Inspecteur
Could not find type "X" in the current scope	Le fichier de la classe n'existe pas encore, n'est pas enregistré, ou contient une erreur	Créer et enregistrer le fichier ; corriger l'erreur dans l'autre script
Function blocked during in/out signal	Modifier <code>monitoring</code> ou une forme pendant un signal de collision	<code>set_deferred("monitoring", ...)</code>
Can't change this state while flushing queries	Ajouter un corps physique ou une zone pendant un signal de collision	<code>add_child.call_deferred()</code> , <code>Fx.spawn(..., true)</code> , ou différer la fonction
État inconnu : X (message du parcours)	Le nœud d'état ne s'appelle pas exactement X	Renommer le nœud (majuscules comprises)
Un objet apparaît à des centaines de pixels de sa place	<code>global_position</code> réglé avant <code>add_child()</code> sous un parent décalé	<code>Fx.spawn()</code>
Rien ne touche rien	Couches et masques	Relire le tableau des couches (étape 02), afficher les formes de collision
Une ressource modifiée change partout	Les ressources sont partagées entre instances	« Rendre unique », ou « Local To Scene », ou <code>duplicate()</code> dans le code
Le personnage tombe à travers le sol	Masque sans la couche 1, ou sol sans forme	Vérifier le masque du corps et la forme du sol
Le jeu se fige après un hit stop	Minuterie soumise à <code>time_scale = 0</code>	<code>create_timer(d, true, false, true)</code>

ANNEXE C · AIDE-MÉMOIRE GDSCRIPT

Aide-mémoire (à ne pas créer)

```

class_name MonType           # type global, visible partout
extends CharacterBody2D      # hérite d'un nœud

signal touche(degats: int)   # déclarer un signal...
# ...l'émettre : touche.emit(3) ; l'écouter : noeud.touche.connect(ma_fonction)

enum Etat { REPOS, COURSE }  # Etat.COURSE vaut 1
const VITESSE := 117.0       # constante, type déduit

@export var vie := 8          # réglable dans l'Inspecteur
@export_range(0.0, 1.0) var ratio := 0.5 # avec un curseur
@export var cible: Node2D     # désigner un nœud dans l'Inspecteur
@export_file("*.tscn") var scene_suivante: String
@onready var sprite: Sprite2D = $Sprite2D # récupéré juste avant _ready()

var etat := Etat.REPOS

```

```

var points: int = 0                                # type explicite
var _prive := 0.0                                  # convention : _ = interne

var vies := 2:                                     # propriété avec setter
    set(v):
        vies = v
        queue_redraw()

func _ready() -> void:                             # une fois, quand le nœud et ses enfants sont prêts
    add_to_group("ennemis")

func _physics_process(delta: float) -> void:      # 60 fois par seconde, à pas fixe
    match etat:
        Etat.REPOS:
            pass
        Etat.COURSE:
            velocity.x = VITESSE
    move_and_slide()

func _unhandled_input(event: InputEvent) -> void:
    if event.is_action_pressed("jump"):
        pass

static func utilitaire(a: float) -> float:        # appelée par MonType.utilitaire(...)
    return a * 2.0

func sequence() -> void:                           # une coroutine
    await get_tree().create_timer(1.0).timeout
    var t := create_tween()
    t.tween_property(self, "modulate:a", 0.0, 0.5)
    await t.finished
    queue_free()

```

ANNEXE D - CE QUE TU AS APPRIS

Concept	Où tu l'as pratiqué
Boucle de jeu, pas de temps fixe, <code>delta</code>	Étapes 01, 02
Résolution virtuelle, pixel net	Étape 01
Touches contre actions	Étape 02
Couches et masques de collision	Étape 02, puis partout
Composition par composants (<code>Health</code> , <code>Hitbox</code> , <code>Hurtbox</code> , <code>HitFlash</code>)	Étapes 02, 05, 08
Patron Observateur (signaux), découplage	Étapes 01, 02, 06, 10
Machine à états finis (par nœuds, par <code>enum</code> , à l'échelle du jeu)	Étapes 02, 04, 05, 07, 10
Prototype jetable, observer et mesurer la référence	Étape 03
Physique arcade, saut variable, coyote time	Étapes 03, 04
Caméra lissée, lissage exponentiel	Étape 04
Préparation, images actives, récupération ; télégraphie	Étapes 05, 07, 09
Conception pilotée par les données (<code>Resource</code>)	Étapes 05, 06, 07
Réserve d'objets (<i>pooling</i>)	Étape 06
Directeur de vagues, tension et détente	Étapes 07, 09
Shaders, effets raster, couleurs indexées	Étape 08
Événement scripté, coroutines	Étape 09
Grammaire d'un boss (phases, point faible, transitions)	Étapes 09, 11, 12
Interfaces, focus, thème, dessin personnalisé	Étape 10
Modules de niveau, tuiles	Étape 11
Pseudo-3D : projection perspective, algorithme du peintre, échelle par paliers	Étapes 11, 12
Plateformes mobiles, référentiel	Étape 12
Ressenti, profilage, export, rétrospective	Étape 13

LES SIX PRINCIPES, UNE DERNIÈRE FOIS

1. Toujours avoir un jeu qui tourne.
2. Des formes de substitution d'abord.
3. Mesurer la référence avant de coder.
4. Un nœud, une responsabilité.
5. Les réglages dans des données, pas dans le code.
6. Des petits pas vérifiés.

SOURCES

- [Le guide vérifié](#), référence de tout le code de ce parcours.
- [The Adventures of Batman & Robin \(Genesis\) Playthrough — NintendoComplete](#).
- Jon Burton, *Coding Secrets* : [partie 1](#) et [partie 2](#).
- [Documentation officielle de Godot 4](#) (en français).

The Adventures of Batman & Robin © DC Comics / Warner Bros. / Sega, 1995. Ce parcours est un support d'apprentissage personnel.